

Table of Contents

1. Binary Exponentiation
2. Euclidean algorithm for computing the greatest common divisor
3. Extended Euclidean Algorithm
4. Linear Diophantine Equations
5. Fibonacci Numbers
6. Sieve of Eratosthenes
7. Linear Sieve
8. Primality tests
9. Integer factorization
10. Euler's totient function
11. Number of divisors / sum of divisors
12. Modular Inverse
13. Linear Congruence Equation
14. Chinese Remainder Theorem
15. Garner's Algorithm
16. Factorial modulo p
17. Discrete Log
18. Primitive Root
19. Discrete Root
20. Montgomery Multiplication
21. Balanced Ternary
22. Gray code
23. Bit manipulation
24. Enumerating submasks of a bitmask
25. Arbitrary-Precision Arithmetic
26. Fast Fourier transform
27. Operations on polynomials and series
28. Continued fractions
29. Factoring Exponentiation

Translated From: e-maxx.ru

Binary Exponentiation

Binary exponentiation (also known as exponentiation by squaring) is a trick which allows to calculate a^n using only $O(\log n)$ multiplications (instead of $O(n)$ multiplications required by the naive approach).

It also has important applications in many tasks unrelated to arithmetic, since it can be used with any operations that have the property of **associativity**:

$$(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$$

Most obviously this applies to modular multiplication, to multiplication of matrices and to other problems which we will discuss below.

Algorithm

Raising a to the power of n is expressed naively as multiplication by a done $n - 1$ times: $a^n = a \cdot a \cdot \dots \cdot a$. However, this approach is not practical for large a or n .

$$a^{b+c} = a^b \cdot a^c \text{ and } a^{2b} = a^b \cdot a^b = (a^b)^2.$$

The idea of binary exponentiation is, that we split the work using the binary representation of the exponent.

Let's write n in base 2, for example:

$$3^{13} = 3^{1101_2} = 3^8 \cdot 3^4 \cdot 3^1$$

Since the number n has exactly $\lfloor \log_2 n \rfloor + 1$ digits in base 2, we only need to perform $O(\log n)$ multiplications, if we know the powers $a^1, a^2, a^4, a^8, \dots, a^{2^{\lfloor \log n \rfloor}}$.

So we only need to know a fast way to compute those. Luckily this is very easy, since an element in the sequence is just the square of the previous element.

$$\begin{aligned} 3^1 &= 3 \\ 3^2 &= (3^1)^2 = 3^2 = 9 \\ 3^4 &= (3^2)^2 = 9^2 = 81 \\ 3^8 &= (3^4)^2 = 81^2 = 6561 \end{aligned}$$

So to get the final answer for 3^{13} , we only need to multiply three of them (skipping 3^2 because the corresponding bit in n is not set):

$$3^{13} = 6561 \cdot 81 \cdot 3 = 1594323$$

The final complexity of this algorithm is $O(\log n)$: we have to compute $\log n$ powers of a , and then have to do at most $\log n$ multiplications to get the final answer from them.

The following recursive approach expresses the same idea:

$$a^n = \begin{cases} 1 & \text{if } n == 0 \\ (a^{\frac{n}{2}})^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ (a^{\frac{n-1}{2}})^2 \cdot a & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases}$$

Implementation

First the recursive approach, which is a direct translation of the recursive formula:

```
long long binpow(long long a, long long b) {
    if (b == 0)
        return 1;
    long long res = binpow(a, b / 2);
    if (b % 2)
        return res * res * a;
    else
        return res * res;
}
```

The second approach accomplishes the same task without recursion. It computes all the powers in a loop, and multiplies the ones with the corresponding set bit in n . Although the complexity of both approaches is identical, this approach will be faster in practice since we don't have the overhead of the recursive calls.

```
long long binpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

Applications

Effective computation of large exponents modulo a number

Problem: Compute $x^n \bmod m$. This is a very common operation. For instance it is used in computing the [modular multiplicative inverse](#).

Solution: Since we know that the modulo operator doesn't interfere with multiplications ($a \cdot b \equiv (a \bmod m) \cdot (b \bmod m) \pmod m$), we can directly use the same code, and just replace every multiplication with a modular multiplication:

```
long long binpow(long long a, long long b, long long m) {
    a %= m;
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
```

Note: It's possible to speed this algorithm for large $b \gg m$. If m is a prime number $x^n \equiv x^{n \bmod (m-1)} \pmod m$ for prime m , and $x^n \equiv x^{n \bmod \phi(m)} \pmod m$ for composite m . This follows directly from Fermat's little theorem and Euler's theorem, see the article about [Modular Inverses](#) for more details.

Effective computation of Fibonacci numbers

Problem: Compute n -th Fibonacci number F_n .

Solution: For more details, see the [Fibonacci Number](#) article. We will only go through an overview of the algorithm. To compute the next Fibonacci number, only the two previous ones are needed, as $F_n = F_{n-1} + F_{n-2}$. We can build a 2×2 matrix that describes this transformation: the transition from F_i and F_{i+1} to F_{i+1} and F_{i+2} . For example, applying this transformation to the pair F_0 and F_1 would change it into F_1 and F_2 . Therefore, we can raise this transformation matrix to the n -th power to find F_n in time complexity $O(\log n)$.

Applying a permutation k times

Problem: You are given a sequence of length n . Apply to it a given permutation k times.

Solution: Simply raise the permutation to k -th power using binary exponentiation, and then apply it to the sequence. This will give you a time complexity of $O(n \log k)$.

```
vector<int> applyPermutation(vector<int> sequence, vector<int> permutation) {
    vector<int> newSequence(sequence.size());
    for(int i = 0; i < sequence.size(); i++) {
        newSequence[i] = sequence[permutation[i]];
    }
    return newSequence;
}

vector<int> permute(vector<int> sequence, vector<int> permutation, long long k) {
    while (k > 0) {
        if (k & 1) {
            sequence = applyPermutation(sequence, permutation);
        }
        permutation = applyPermutation(permutation, permutation);
        k >>= 1;
    }
    return sequence;
}
```

Note: This task can be solved more efficiently in linear time by building the permutation graph and considering each cycle independently. You could then compute k modulo the size of the cycle and find the final position for each number which is part of this cycle.

Fast application of a set of geometric operations to a set of points

Problem: Given n points p_i , apply m transformations to each of these points. Each transformation can be a shift, a scaling or a rotation around a given axis by a given angle. There is also a "loop" operation which applies a given list of transformations k times ("loop" operations can be nested). You should apply all transformations faster than $O(n \cdot \text{length})$, where length is the total number of transformations to be applied (after unrolling "loop" operations).

Solution: Let's look at how the different types of transformations change the coordinates:

- Shift operation: adds a different constant to each of the coordinates.
- Scaling operation: multiplies each of the coordinates by a different constant.
- Rotation operation: the transformation is more complicated (we won't go in details here), but each of the new coordinates still can be represented as a linear combination of the old ones.

As you can see, each of the transformations can be represented as a linear operation on the coordinates. Thus, a transformation can be written as a 4×4 matrix of the form:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}$$

that, when multiplied by a vector with the old coordinates and a unit gives a new vector with the new coordinates and a unit:

$$\begin{pmatrix} x & y & z & 1 \end{pmatrix} \cdot \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} = \begin{pmatrix} x' & y' & z' & 1 \end{pmatrix}$$

(Why introduce a fictitious fourth coordinate, you ask? That is the beauty of [homogeneous coordinates](#), which find great application in computer graphics. Without this, it would not be possible to implement affine operations like the shift operation as a single matrix multiplication, as it requires us to *add* a constant to the coordinates. The affine transformation becomes a linear transformation in the higher dimension!)

Here are some examples of how transformations are represented in matrix form:

- Shift operation: shift x coordinate by 5, y coordinate by 7 and z coordinate by 9.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 5 & 7 & 9 & 1 \end{pmatrix}$$

- Scaling operation: scale the x coordinate by 10 and the other two by 5.

$$\begin{pmatrix} 10 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Rotation operation: rotate θ degrees around the x axis following the right-hand rule (counter-clockwise direction).

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Now, once every transformation is described as a matrix, the sequence of transformations can be described as a product of these matrices, and a "loop" of k repetitions can be described as the matrix raised to the power of k (which can be calculated using binary exponentiation in $O(\log k)$). This way, the matrix which represents all transformations can be calculated first in $O(m \log k)$, and then it can be applied to each of the n points in $O(n)$ for a total complexity of $O(n + m \log k)$.

Number of paths of length k in a graph

Problem: Given a directed unweighted graph of n vertices, find the number of paths of length k from any vertex u to any other vertex v .

Solution: This problem is considered in more detail in [a separate article](#). The algorithm consists of raising the adjacency matrix M of the graph (a matrix where $m_{ij} = 1$ if there is an edge from i to j , or 0 otherwise) to the k -th power. Now m_{ij} will be the number of paths of length k from i to j . The time complexity of this solution is $O(n^3 \log k)$.

Note: In that same article, another variation of this problem is considered: when the edges are weighted and it is required to find the minimum weight path containing exactly k edges. As shown in that article, this problem is also solved by exponentiation of the adjacency matrix. The matrix would have the weight of the edge from i to j , or ∞ if there is no such edge. Instead of the usual operation of multiplying two matrices, a modified one should be used: instead of multiplication, both values are added, and instead of a summation, a minimum is taken. That is:

$$\text{result}_{ij} = \min_{1 \leq k \leq n} (a_{ik} + b_{kj}).$$

Variation of binary exponentiation: multiplying two numbers modulo m

Problem: Multiply two numbers a and b modulo m . a and b fit in the built-in data types, but their product is too big to fit in a 64-bit integer. The idea is to compute $a \cdot b \pmod{m}$ without using bignum arithmetics.

Solution: We simply apply the binary construction algorithm described above, only performing additions instead of multiplications. In other words, we have "expanded" the multiplication of two numbers to $O(\log m)$ operations of addition and multiplication by two (which, in essence, is an addition).

$$a \cdot b = \begin{cases} 0 & \text{if } a = 0 \\ 2 \cdot \frac{a}{2} \cdot b & \text{if } a > 0 \text{ and } a \text{ is even} \\ 2 \cdot \frac{a-1}{2} \cdot b + b & \text{if } a > 0 \text{ and } a \text{ is odd} \end{cases}$$

Note: You can solve this task in a different way by using floating-point operations. First compute the expression $\frac{a \cdot b}{m}$ using floating-point numbers and cast it to an unsigned integer q . Subtract $q \cdot m$ from $a \cdot b$ using unsigned integer arithmetics and take it modulo m to find the answer. This solution looks rather unreliable, but it is very fast, and very easy to implement. See [here](#) for more information.

Practice Problems

- [UVa 1230 - MODEX](#)
- [UVa 374 - Big Mod](#)
- [UVa 11029 - Leading and Trailing](#)
- [Codeforces - Parking Lot](#)
- [leetcode - Count good numbers](#)
- [Codechef - Chef and Ruffles](#)
- [Codeforces - Decoding Genome](#)
- [Codeforces - Neural Network Country](#)
- [Codeforces - Magic Gems](#)
- [SPOJ - The last digit](#)
- [SPOJ - Locker](#)
- [LA - 3722 Jewel-eating Monsters](#)
- [SPOJ - Just add it](#)
- [Codeforces - Stairs and Lines](#)

Contributors:

[Jakob Kogler](#) (41.95%) [Gabriel Silva Simões](#) (17.98%) [Mariia Mykhailova](#) (11.24%) [David Zhang](#) (5.99%) [Rodion Gorkovenko](#) (5.62%) [Mirco Paul](#) (3.0%)
[Oleksandr Kulkov](#) (2.62%) [shark_s](#) (2.25%) [Morass](#) (1.87%) [Turfa Auliarachman](#) (1.87%) [Zarif Muhtasim](#) (1.5%) [wilwxk](#) (0.75%) [Anton Ålgmyr](#) (0.37%)
[Arjun Patel](#) (0.37%) [ContronThePanda](#) (0.37%) [Rhythm Bansal](#) (0.37%) [Vatsal Sharma](#) (0.37%) [ahmedibrahim404](#) (0.37%) [snymm2](#) (0.37%)
[wiktorkuchta](#) (0.37%) [zhangdavid275](#) (0.37%)

Translated

From: e-maxx.ru

Euclidean algorithm for computing the greatest common divisor

Given two non-negative integers a and b , we have to find their **GCD** (greatest common divisor), i.e. the largest number which is a divisor of both a and b . It's commonly denoted by $\gcd(a, b)$. Mathematically it is defined as:

$$\gcd(a, b) = \max \{k > 0 : (k \mid a) \text{ and } (k \mid b)\}$$

(here the symbol " $\mid$ " denotes divisibility, i.e. " $k \mid a$ " means " k divides a ")

When one of the numbers is zero, while the other is non-zero, their greatest common divisor, by definition, is the second number. When both numbers are zero, their greatest common divisor is undefined (it can be any arbitrarily large number), but it is convenient to define it as zero as well to preserve the associativity of $\gcd$. Which gives us a simple rule: if one of the numbers is zero, the greatest common divisor is the other number.

The Euclidean algorithm, discussed below, allows to find the greatest common divisor of two numbers a and b in $O(\log \min(a, b))$. Since the function is **associative**, to find the GCD of **more than two numbers**, we can do $\gcd(a, b, c) = \gcd(a, \gcd(b, c))$ and so forth.

The algorithm was first described in Euclid's "Elements" (circa 300 BC), but it is possible that the algorithm has even earlier origins.

Algorithm

Originally, the Euclidean algorithm was formulated as follows: subtract the smaller number from the larger one until one of the numbers is zero. Indeed, if g divides a and b , it also divides $a - b$. On the other hand, if g divides $a - b$ and b , then it also divides $a = b + (a - b)$, which means that the sets of the common divisors of $\{a, b\}$ and $\{b, a - b\}$ coincide.

Note that a remains the larger number until b is subtracted from it at least $\lfloor \frac{a}{b} \rfloor$ times. Therefore, to speed things up, $a - b$ is substituted with $a - \lfloor \frac{a}{b} \rfloor b = a \bmod b$. Then the algorithm is formulated in an extremely simple way:

$$\gcd(a, b) = \begin{cases} a, & \text{if } b = 0 \\ \gcd(b, a \bmod b), & \text{otherwise.} \end{cases}$$

Implementation

```
int gcd (int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd (b, a % b);
}
```

Using the ternary operator in C++, we can write it as a one-liner.

```
int gcd (int a, int b) {
    return b ? gcd (b, a % b) : a;
}
```

And finally, here is a non-recursive implementation:

```
int gcd (int a, int b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
}
```

```
    return a;
}
```

Note that since C++17, `gcd` is implemented as a [standard function](#) in C++.

Time Complexity

The running time of the algorithm is estimated by Lamé's theorem, which establishes a surprising connection between the Euclidean algorithm and the Fibonacci sequence:

If $a > b \geq 1$ and $b < F_n$ for some n , the Euclidean algorithm performs at most $n - 2$ recursive calls.

Moreover, it is possible to show that the upper bound of this theorem is optimal. When $a = F_n$ and $b = F_{n-1}$, $\text{gcd}(a, b)$ will perform exactly $n - 2$ recursive calls. In other words, consecutive Fibonacci numbers are the worst case input for Euclid's algorithm.

Given that Fibonacci numbers grow exponentially, we get that the Euclidean algorithm works in $O(\log \min(a, b))$.

Another way to estimate the complexity is to notice that $a \bmod b$ for the case $a \geq b$ is at least 2 times smaller than a , so the larger number is reduced at least in half on each iteration of the algorithm. Applying this reasoning to the case when we compute the GCD of the set of numbers $a_1, \dots, a_n \leq C$, this also allows us to estimate the total runtime as $O(n + \log C)$, rather than $O(n \log C)$, since every non-trivial iteration of the algorithm reduces the current GCD candidate by at least a factor of 2.

Least common multiple

Calculating the least common multiple (commonly denoted **LCM**) can be reduced to calculating the GCD with the following simple formula:

$$\text{lcm}(a, b) = \frac{a \cdot b}{\text{gcd}(a, b)}$$

Thus, LCM can be calculated using the Euclidean algorithm with the same time complexity:

A possible implementation, that cleverly avoids integer overflows by first dividing a with the GCD, is given here:

```
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}
```

Binary GCD

The Binary GCD algorithm is an optimization to the normal Euclidean algorithm.

The slow part of the normal algorithm are the modulo operations. Modulo operations, although we see them as $O(1)$, are a lot slower than simpler operations like addition, subtraction or bitwise operations. So it would be better to avoid those.

It turns out, that you can design a fast GCD algorithm that avoids modulo operations. It's based on a few properties:

- If both numbers are even, then we can factor out a two of both and compute the GCD of the remaining numbers:
 $\text{gcd}(2a, 2b) = 2 \cdot \text{gcd}(a, b)$.
- If one of the numbers is even and the other one is odd, then we can remove the factor 2 from the even one: $\text{gcd}(2a, b) = \text{gcd}(a, b)$ if b is odd.
- If both numbers are odd, then subtracting one number of the other one will not change the GCD: $\text{gcd}(a, b) = \text{gcd}(b, a - b)$

Using only these properties, and some fast bitwise functions from GCC, we can implement a fast version:

```
int gcd(int a, int b) {
    if (!a || !b)
        return a | b;
    unsigned shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    b >>= __builtin_ctz(b);
```

```
    if (a > b)
        swap(a, b);
    b -= a;
} while (b);
return a << shift;
}
```

Notice, that such an optimization is usually not necessary, and most programming languages already have a GCD function in their standard libraries. E.g. C++17 has such a function `std::gcd` in the `numeric` header.

Practice Problems

- [CSAcademy - Greatest Common Divisor](#)
- [Codeforces 1916B - Two Divisors](#)

Contributors:

[Nalin Bhardwaj](#) (42.64%) [Jakob Kogler](#) (38.76%) [Oleksandr Kulkov](#) (10.85%) [Mariia Mykhailova](#) (3.88%) [Pedro](#) (1.55%) [Anikett](#) (0.78%)
[Sachin Kumar](#) (0.78%) [jlnlk](#) (0.78%)

Translated From: e-maxx.ru

Extended Euclidean Algorithm

While the [Euclidean algorithm](#) calculates only the greatest common divisor (GCD) of two integers a and b , the extended version also finds a way to represent GCD in terms of a and b , i.e. coefficients x and y for which:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

It's important to note that by [Bézout's identity](#) we can always find such a representation. For instance, $\gcd(55, 80) = 5$, therefore we can represent 5 as a linear combination with the terms 55 and 80: $55 \cdot 3 + 80 \cdot (-2) = 5$

A more general form of that problem is discussed in the article about [Linear Diophantine Equations](#). It will build upon this algorithm.

Algorithm

We will denote the GCD of a and b with g in this section.

The changes to the original algorithm are very simple. If we recall the algorithm, we can see that the algorithm ends with $b = 0$ and $a = g$. For these parameters we can easily find coefficients, namely $g \cdot 1 + 0 \cdot 0 = g$.

Starting from these coefficients $(x, y) = (1, 0)$, we can go backwards up the recursive calls. All we need to do is to figure out how the coefficients x and y change during the transition from (a, b) to $(b, a \bmod b)$.

Let us assume we found the coefficients (x_1, y_1) for $(b, a \bmod b)$:

$$b \cdot x_1 + (a \bmod b) \cdot y_1 = g$$

and we want to find the pair (x, y) for (a, b) :

$$a \cdot x + b \cdot y = g$$

We can represent $a \bmod b$ as:

$$a \bmod b = a - \left\lfloor \frac{a}{b} \right\rfloor \cdot b$$

Substituting this expression in the coefficient equation of (x_1, y_1) gives:

$$g = b \cdot x_1 + (a \bmod b) \cdot y_1 = b \cdot x_1 + \left(a - \left\lfloor \frac{a}{b} \right\rfloor \cdot b \right) \cdot y_1$$

and after rearranging the terms:

$$g = a \cdot y_1 + b \cdot \left(x_1 - y_1 \cdot \left\lfloor \frac{a}{b} \right\rfloor \right)$$

We found the values of x and y :

$$\begin{cases} x = y_1 \\ y = x_1 - y_1 \cdot \lfloor \frac{a}{b} \rfloor \end{cases}$$

Implementation

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

The recursive function above returns the GCD and the values of coefficients to `x` and `y` (which are passed by reference to the function).

This implementation of extended Euclidean algorithm produces correct results for negative integers as well.

Iterative version

It's also possible to write the Extended Euclidean algorithm in an iterative way. Because it avoids recursion, the code will run a little bit faster than the recursive one.

```
int gcd(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}
```

If you look closely at the variables `a1` and `b1`, you can notice that they take exactly the same values as in the iterative version of the normal [Euclidean algorithm](#). So the algorithm will at least compute the correct GCD.

To see why the algorithm computes the correct coefficients, consider that the following invariants hold at any given time (before the while loop begins and at the end of each iteration):

$$x \cdot a + y \cdot b = a_1$$

$$x_1 \cdot a + y_1 \cdot b = b_1$$

Let the values at the end of an iteration be denoted by a prime ($'$), and assume $q = \frac{a_1}{b_1}$. From the [Euclidean algorithm](#), we have:

$$a'_1 = b_1$$

$$b'_1 = a_1 - q \cdot b_1$$

For the first invariant to hold, the following should be true:

$$x' \cdot a + y' \cdot b = a'_1 = b_1$$

$$x' \cdot a + y' \cdot b = x_1 \cdot a + y_1 \cdot b$$

Similarly for the second invariant, the following should hold:

$$x'_1 \cdot a + y'_1 \cdot b = a_1 - q \cdot b_1$$

$$x'_1 \cdot a + y'_1 \cdot b = (x - q \cdot x_1) \cdot a + (y - q \cdot y_1) \cdot b$$

By comparing the coefficients of a and b , the update equations for each variable can be derived, ensuring that the invariants are maintained throughout the algorithm.

At the end we know that a_1 contains the GCD, so $x \cdot a + y \cdot b = g$. Which means that we have found the required coefficients.

You can even optimize the code more, and remove the variable a_1 and b_1 from the code, and just reuse a and b . However if you do so, you lose the ability to argue about the invariants.

Practice Problems

- [UVA - 10104 - Euclid Problem](#)
- [GYM - \(J\) Once Upon A Time](#)
- [UVA - 12775 - Gift Dilemma](#)

Contributors:

[Jakob Kogler](#) (43.7%) [Silap Aliyev](#) (23.7%) [Kshitij Sharma](#) (19.26%) [Oleksandr Kulkov](#) (4.44%) [Morass](#) (2.96%) [Anton Ålgmyr](#) (1.48%)
[Mariia Mykhailova](#) (1.48%) [boxlesscat](#) (1.48%) [Rohit Choudhary](#) (0.74%) [jxu](#) (0.74%)

Translated From: e-maxx.ru

Linear Diophantine Equation

A Linear Diophantine Equation (in two variables) is an equation of the general form:

$$ax + by = c$$

where a , b , c are given integers, and x , y are unknown integers.

In this article, we consider several classical problems on these equations:

- finding one solution
- finding all solutions
- finding the number of solutions and the solutions themselves in a given interval
- finding a solution with minimum value of $x + y$

The degenerate case

A degenerate case that need to be taken care of is when $a = b = 0$. It is easy to see that we either have no solutions or infinitely many solutions, depending on whether $c = 0$ or not. In the rest of this article, we will ignore this case.

Analytic solution

When $a \neq 0$ and $b \neq 0$, the equation $ax + by = c$ can be equivalently treated as either of the following:

$$\begin{aligned} ax &\equiv c \pmod{b} \\ by &\equiv c \pmod{a} \end{aligned}$$

Without loss of generality, assume that $b \neq 0$ and consider the first equation. When a and b are co-prime, the solution to it is given as

$$x \equiv ca^{-1} \pmod{b},$$

where a^{-1} is the [modular inverse](#) of a modulo b .

When a and b are not co-prime, values of ax modulo b for all integer x are divisible by $g = \gcd(a, b)$, so the solution only exists when c is divisible by g . In this case, one of solutions can be found by reducing the equation by g :

$$(a/g)x \equiv (c/g) \pmod{b/g}.$$

By the definition of g , the numbers a/g and b/g are co-prime, so the solution is given explicitly as

$$\begin{cases} x \equiv (c/g)(a/g)^{-1} \pmod{b/g}, \\ y = \frac{c-ax}{b}. \end{cases}$$

Algorithmic solution

Bézout's lemma (also called Bézout's identity) is a useful result that can be used to understand the following solution.

Let $g = \gcd(a, b)$. Then there exist integers x, y such that $ax + by = g$.

Moreover, g is the least such positive integer that can be written as $ax + by$; all integers of the form $ax + by$ are multiples of g .

To find one solution of the Diophantine equation with 2 unknowns, you can use the [Extended Euclidean algorithm](#). First, assume that a and b are non-negative. When we apply Extended Euclidean algorithm for a and b , we can find their greatest common divisor g and 2 numbers x_g and y_g such that:

$$ax_g + by_g = g$$

If c is divisible by $g = \gcd(a, b)$, then the given Diophantine equation has a solution, otherwise it does not have any solution. The proof is straight-forward: a linear combination of two numbers is divisible by their common divisor.

Now suppose that c is divisible by g , then we have:

$$a \cdot x_g \cdot \frac{c}{g} + b \cdot y_g \cdot \frac{c}{g} = c$$

Therefore one of the solutions of the Diophantine equation is:

$$x_0 = x_g \cdot \frac{c}{g},$$

$$y_0 = y_g \cdot \frac{c}{g}.$$

The above idea still works when a or b or both of them are negative. We only need to change the sign of x_0 and y_0 when necessary.

Finally, we can implement this idea as follows (note that this code does not consider the case $a = b = 0$):

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
```

Getting all solutions

From one solution (x_0, y_0) , we can obtain all the solutions of the given equation.

Let $g = \gcd(a, b)$ and let x_0, y_0 be integers which satisfy the following:

$$a \cdot x_0 + b \cdot y_0 = c$$

Now, we should see that adding b/g to x_0 , and, at the same time subtracting a/g from y_0 will not break the equality:

$$a \cdot \left(x_0 + \frac{b}{g}\right) + b \cdot \left(y_0 - \frac{a}{g}\right) = a \cdot x_0 + b \cdot y_0 + a \cdot \frac{b}{g} - b \cdot \frac{a}{g} = c$$

Obviously, this process can be repeated again, so all the numbers of the form:

$$x = x_0 + k \cdot \frac{b}{g}$$

$$y = y_0 - k \cdot \frac{a}{g}$$

are solutions of the given Diophantine equation.

Since the equation is linear, all solutions lie on the same line, and by the definition of g this is the set of all possible solutions of the given Diophantine equation.

Finding the number of solutions and the solutions in a given interval

From previous section, it should be clear that if we don't impose any restrictions on the solutions, there would be infinite number of them. So in this section, we add some restrictions on the interval of x and y , and we will try to count and enumerate all the solutions.

Let there be two intervals: $[\min_x; \max_x]$ and $[\min_y; \max_y]$ and let's say we only want to find the solutions in these two intervals.

Note that if a or b is 0, then the problem only has one solution. We don't consider this case here.

First, we can find a solution which has minimum value of x , such that $x \geq \min_x$. To do this, we first find any solution of the Diophantine equation. Then, we shift this solution to get $x \geq \min_x$ (using what we know about the set of all solutions in previous section). This can be done in $O(1)$. Denote this minimum value of x by l_{x1} .

Similarly, we can find the maximum value of x which satisfies $x \leq \max_x$. Denote this maximum value of x by r_{x1} .

Similarly, we can find the minimum value of y ($y \geq \min_y$) and maximum value of y ($y \leq \max_y$). Denote the corresponding values of x by l_{x2} and r_{x2} .

The final solution is all solutions with x in intersection of $[l_{x1}, r_{x1}]$ and $[l_{x2}, r_{x2}]$. Let denote this intersection by $[l_x, r_x]$.

Following is the code implementing this idea. Notice that we divide a and b at the beginning by g . Since the equation $ax + by = c$ is equivalent to the equation $\frac{a}{g}x + \frac{b}{g}y = \frac{c}{g}$, we can use this one instead and have $\gcd(\frac{a}{g}, \frac{b}{g}) = 1$, which simplifies the formulas.

```
void shift_solution(int &x, int &y, int a, int b, int cnt) {
    x += cnt * b;
    y -= cnt * a;
}

int find_all_solutions(int a, int b, int c, int minx, int maxx, int miny, int maxy) {
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
```

```

    shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    int rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}

```

Once we have l_x and r_x , it is also simple to enumerate through all the solutions. Just need to iterate through $x = l_x + k \cdot \frac{b}{g}$ for all $k \geq 0$ until $x = r_x$, and find the corresponding y values using the equation $ax + by = c$.

Find the solution with minimum value of $x + y$

Here, x and y also need to be given some restriction, otherwise, the answer may become negative infinity.

The idea is similar to previous section: We find any solution of the Diophantine equation, and then shift the solution to satisfy some conditions.

Finally, use the knowledge of the set of all solutions to find the minimum:

$$x' = x + k \cdot \frac{b}{g},$$

$$y' = y - k \cdot \frac{a}{g}.$$

Note that $x + y$ change as follows:

$$x' + y' = x + y + k \cdot \left(\frac{b}{g} - \frac{a}{g} \right) = x + y + k \cdot \frac{b - a}{g}$$

If $a < b$, we need to select smallest possible value of k . If $a > b$, we need to select the largest possible value of k . If $a = b$, all solution will have the same sum $x + y$.

Practice Problems

- [Spoj - Crucial Equation](#)
- [SGU 106](#)
- [Codeforces - Ebony and Ivory](#)
- [Codechef - Get AC in one go](#)
- [LightOj - Solutions to an equation](#)
- [Atcoder - F - S = 1](#)

Contributors:

[thanhtnguyen](#) (57.64%)
 [Jakob Kogler](#) (17.47%)
 [Oleksandr Kulkov](#) (12.67%)
 [jxu](#) (4.8%)
 [Mariia Mykhailova](#) (3.06%)
 [Bhuvnesh Jain](#) (2.18%)
[boxlesscat](#) (1.31%)
 [Arjun Patel](#) (0.44%)
 [traxex2](#) (0.44%)

Translated From: e-maxx.ru

Fibonacci Numbers

The Fibonacci sequence is defined as follows:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

The first elements of the sequence ([OEIS A000045](#)) are:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots$$

Properties

Fibonacci numbers possess a lot of interesting properties. Here are a few of them:

- Cassini's identity:

$$F_{n-1}F_{n+1} - F_n^2 = (-1)^n$$

This can be proved by induction. A one-line proof by Knuth comes from taking the determinant of the 2x2 matrix form below.

- The "addition" rule:

$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$$

- Applying the previous identity to the case $k = n$, we get:

$$F_{2n} = F_n(F_{n+1} + F_{n-1})$$

- From this we can prove by induction that for any positive integer k , F_{nk} is multiple of F_n .
- The inverse is also true: if F_m is multiple of F_n , then m is multiple of n .
- GCD identity:

$$\text{GCD}(F_m, F_n) = F_{\text{GCD}(m, n)}$$

- Fibonacci numbers are the worst possible inputs for Euclidean algorithm (see [Lame's theorem](#) in [Euclidean algorithm](#))

Fibonacci Coding

We can use the sequence to encode positive integers into binary code words. According to Zeckendorf's theorem, any natural number n can be uniquely represented as a sum of Fibonacci numbers:

$$N = F_{k_1} + F_{k_2} + \dots + F_{k_r}$$

such that $k_1 \geq k_2 + 2$, $k_2 \geq k_3 + 2$, $\dots$, $k_r \geq 2$ (i.e.: the representation cannot use two consecutive Fibonacci numbers).

It follows that any number can be uniquely encoded in the Fibonacci coding. And we can describe this representation with binary codes $d_0 d_1 d_2 \dots d_s 1$, where d_i is 1 if F_{i+2} is used in the representation. The code will be appended by a 1 to indicate the end of the code word. Notice that this is the only occurrence where two consecutive 1-bits appear.

$$\begin{aligned}
 1 &= 1 &= F_2 &= (11)_F \\
 2 &= 2 &= F_3 &= (011)_F \\
 6 &= 5 + 1 &= F_5 + F_2 &= (10011)_F \\
 8 &= 8 &= F_6 &= (000011)_F \\
 9 &= 8 + 1 &= F_6 + F_2 &= (100011)_F \\
 19 &= 13 + 5 + 1 = F_7 + F_5 + F_2 &= (1001011)_F
 \end{aligned}$$

The encoding of an integer n can be done with a simple greedy algorithm:

1. Iterate through the Fibonacci numbers from the largest to the smallest until you find one less than or equal to n .
2. Suppose this number was F_i . Subtract F_i from n and put a 1 in the $i - 2$ position of the code word (indexing from 0 from the leftmost to the rightmost bit).
3. Repeat until there is no remainder.
4. Add a final 1 to the codeword to indicate its end.

To decode a code word, first remove the final 1. Then, if the i -th bit is set (indexing from 0 from the leftmost to the rightmost bit), sum F_{i+2} to the number.

Formulas for the n^{th} Fibonacci number

Closed-form expression

There is a formula known as "Binet's formula", even though it was already known by Moivre:

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

This formula is easy to prove by induction, but it can be deduced with the help of the concept of generating functions or by solving a functional equation.

You can immediately notice that the second term's absolute value is always less than 1, and it also decreases very rapidly (exponentially). Hence the value of the first term alone is "almost" F_n . This can be written strictly as:

$$F_n = \left\lceil \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n}{\sqrt{5}} \right\rceil$$

where the square brackets denote rounding to the nearest integer.

As these two formulas would require very high accuracy when working with fractional numbers, they are of little use in practical calculations.

Fibonacci in linear time

The n -th Fibonacci number can be easily found in $O(n)$ by computing the numbers one by one up to n . However, there are also faster ways, as we will see.

We can start from an iterative approach, to take advantage of the use of the formula $F_n = F_{n-1} + F_{n-2}$, therefore, we will simply precalculate those values in an array. Taking into account the base cases for F_0 and F_1 .

```
int fib(int n) {
    int a = 0;
    int b = 1;
    for (int i = 0; i < n; i++) {
        int tmp = a + b;
        a = b;
    }
}
```

```

    b = tmp;
}
return a;
}

```

In this way, we obtain a linear solution, $O(n)$ time, saving all the values prior to n in the sequence.

Matrix form

To go from (F_n, F_{n-1}) to (F_{n+1}, F_n) , we can express the linear recurrence as a 2×2 matrix multiplication:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} F_n + F_{n-1} \\ F_n \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

This lets us treat iterating the recurrence as repeated matrix multiplication, which has nice properties. In particular,

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

where $F_1 = 1, F_0 = 0$. In fact, since

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix}$$

we can use the matrix directly:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

Thus, in order to find F_n in $O(\log n)$ time, we must raise the matrix to n . (See [Binary exponentiation](#))

```

struct matrix {
    long long mat[2][2];
    matrix friend operator *(const matrix &a, const matrix &b){
        matrix c;
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                c.mat[i][j] = 0;
                for (int k = 0; k < 2; k++) {
                    c.mat[i][j] += a.mat[i][k] * b.mat[k][j];
                }
            }
        }
        return c;
    }
};

matrix matpow(matrix base, long long n) {
    matrix ans{ {
        {1, 0},
        {0, 1}
    } };
    while (n) {
        if (n&1)
            ans = ans*base;
        base = base*base;
        n >>= 1;
    }
    return ans;
}

long long fib(int n) {
    matrix base{ {
        {1, 1},
        {1, 0}
    }

```

```

    } };
    return matpow(base, n).mat[0][1];
}

```

Fast Doubling Method

By expanding the above matrix expression for $n = 2 \cdot k$

$$\begin{pmatrix} F_{2k+1} & F_{2k} \\ F_{2k} & F_{2k-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{2k} = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}^2$$

we can find these simpler equations:

$$\begin{aligned} F_{2k+1} &= F_{k+1}^2 + F_k^2 \\ F_{2k} &= F_k(F_{k+1} + F_{k-1}) = F_k(2F_{k+1} - F_k) \end{aligned}$$

Thus using above two equations Fibonacci numbers can be calculated easily by the following code:

```

pair<int, int> fib (int n) {
    if (n == 0)
        return {0, 1};

    auto p = fib(n >> 1);
    int c = p.first * (2 * p.second - p.first);
    int d = p.first * p.first + p.second * p.second;
    if (n & 1)
        return {d, c + d};
    else
        return {c, d};
}

```

The above code returns F_n and F_{n+1} as a pair.

Periodicity modulo p

Consider the Fibonacci sequence modulo p . We will prove the sequence is periodic.

Let us prove this by contradiction. Consider the first $p^2 + 1$ pairs of Fibonacci numbers taken modulo p :

$$(F_0, F_1), (F_1, F_2), \dots, (F_{p^2}, F_{p^2+1})$$

There can only be p different remainders modulo p , and at most p^2 different pairs of remainders, so there are at least two identical pairs among them. This is sufficient to prove the sequence is periodic, as a Fibonacci number is only determined by its two predecessors. Hence if two pairs of consecutive numbers repeat, that would also mean the numbers after the pair will repeat in the same fashion.

We now choose two pairs of identical remainders with the smallest indices in the sequence. Let the pairs be (F_a, F_{a+1}) and (F_b, F_{b+1}) . We will prove that $a = 0$. If this was false, there would be two previous pairs (F_{a-1}, F_a) and (F_{b-1}, F_b) , which, by the property of Fibonacci numbers, would also be equal. However, this contradicts the fact that we had chosen pairs with the smallest indices, completing our proof that there is no pre-period (i.e the numbers are periodic starting from F_0).

Practice Problems

- [SPOJ - Euclid Algorithm Revisited](#)
- [SPOJ - Fibonacci Sum](#)
- [HackerRank - Is Fibo](#)
- [Project Euler - Even Fibonacci numbers](#)
- [DMOJ - Fibonacci Sequence](#)

- [DMOJ - Fibonacci Sequence \(Harder\)](#)
- [DMOJ UCLV - Numbered sequence of pencils](#)
- [DMOJ UCLV - Fibonacci 2D](#)
- [DMOJ UCLV - fibonacci calculation](#)
- [LightOJ - Number Sequence](#)
- [Codeforces - C. Fibonacci](#)
- [Codeforces - A. Hexadecimal's theorem](#)
- [Codeforces - B. Blackboard Fibonacci](#)
- [Codeforces - E. Fibonacci Number](#)

Contributors:

[Gabriel Silva Simões](#) (23.55%) [Jakob Kogler](#) (20.48%) [jxu](#) (18.77%) [Carlos Javier Blanco](#) (15.7%) [Madhur Chauhan](#) (6.14%) [Mariia Mykhailova](#) (5.12%)
[Oleksandr Kulkov](#) (2.73%) [Morass](#) (2.05%) [Md Shamim Ahmmed](#) (1.71%) [Anirudh Srikanth](#) (1.02%) [boxlesscat](#) (1.02%) [pasthec](#) (0.68%)
[Ankit Kumar Dwivedi](#) (0.34%) [Michael Hayter](#) (0.34%) [conlacda](#) (0.34%)

Translated From: e-maxx.ru

Sieve of Eratosthenes

Sieve of Eratosthenes is an algorithm for finding all the prime numbers in a segment $[1; n]$ using $O(n \log \log n)$ operations.

The algorithm is very simple: at the beginning we write down all numbers between 2 and n . We mark all proper multiples of 2 (since 2 is the smallest prime number) as composite. A proper multiple of a number x , is a number greater than x and divisible by x . Then we find the next number that hasn't been marked as composite, in this case it is 3. Which means 3 is prime, and we mark all proper multiples of 3 as composite. The next unmarked number is 5, which is the next prime number, and we mark all proper multiples of it. And we continue this procedure until we have processed all numbers in the row.

In the following image you can see a visualization of the algorithm for computing all prime numbers in the range $[1; 16]$. It can be seen, that quite often we mark numbers as composite multiple times.



The idea behind is this: A number is prime, if none of the smaller prime numbers divides it. Since we iterate over the prime numbers in order, we already marked all numbers, which are divisible by at least one of the prime numbers, as divisible. Hence if we reach a cell and it is not marked, then it isn't divisible by any smaller prime number and therefore has to be prime.

Implementation

```
int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i <= n; i++) {
    if (is_prime[i] && (long long)i * i <= n) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}
```

This code first marks all numbers except zero and one as potential prime numbers, then it begins the process of sifting composite numbers. For this it iterates over all numbers from 2 to n . If the current number i is a prime number, it marks all numbers that are multiples of i as composite numbers, starting from i^2 . This is already an optimization over naive way of implementing it, and is allowed as all smaller numbers that are multiples of i necessary also have a prime factor which is less than i , so all of them were already sifted earlier. Since i^2 can easily overflow the type `int`, the additional verification is done using type `long long` before the second nested loop.

Using such implementation the algorithm consumes $O(n)$ of the memory (obviously) and performs $O(n \log \log n)$ (see next section).

Asymptotic analysis

It's simple to prove a running time of $O(n \log n)$ without knowing anything about the distribution of primes - ignoring the `is_prime` check, the inner loop runs (at most) n/i times for $i = 2, 3, 4, \dots$, leading the total number of operations in the inner loop to be a harmonic sum like $n(1/2 + 1/3 + 1/4 + \dots)$, which is bounded by $O(n \log n)$.

Let's prove that algorithm's running time is $O(n \log \log n)$. The algorithm will perform $\frac{n}{p}$ operations for every prime $p \leq n$ in the inner loop. Hence, we need to evaluate the next expression:

$$\sum_{\substack{p \leq n, \\ p \text{ prime}}} \frac{n}{p} = n \cdot \sum_{\substack{p \leq n, \\ p \text{ prime}}} \frac{1}{p}.$$

Let's recall two known facts.

- The number of prime numbers less than or equal to n is approximately $\frac{n}{\ln n}$.
- The k -th prime number approximately equals $k \ln k$ (this follows from the previous fact).

Thus we can write down the sum in the following way:

$$\sum_{\substack{p \leq n, \\ p \text{ prime}}} \frac{1}{p} \approx \frac{1}{2} + \sum_{k=2}^{\frac{n}{\ln n}} \frac{1}{k \ln k}.$$

Here we extracted the first prime number 2 from the sum, because $k = 1$ in approximation $k \ln k$ is 0 and causes a division by zero.

Now, let's evaluate this sum using the integral of a same function over k from 2 to $\frac{n}{\ln n}$ (we can make such approximation because, in fact, the sum is related to the integral as its approximation using the rectangle method):

$$\sum_{k=2}^{\frac{n}{\ln n}} \frac{1}{k \ln k} \approx \int_2^{\frac{n}{\ln n}} \frac{1}{k \ln k} dk.$$

The antiderivative for the integrand is $\ln \ln k$. Using a substitution and removing terms of lower order, we'll get the result:

$$\int_2^{\frac{n}{\ln n}} \frac{1}{k \ln k} dk = \ln \ln \frac{n}{\ln n} - \ln \ln 2 = \ln(\ln n - \ln \ln n) - \ln \ln 2 \approx \ln \ln n.$$

Now, returning to the original sum, we'll get its approximate evaluation:

$$\sum_{\substack{p \leq n, \\ p \text{ is prime}}} \frac{n}{p} \approx n \ln \ln n + o(n).$$

You can find a more strict proof (that gives more precise evaluation which is accurate within constant multipliers) in the book authored by Hardy & Wright "An Introduction to the Theory of Numbers" (p. 349).

Different optimizations of the Sieve of Eratosthenes

The biggest weakness of the algorithm is, that it "walks" along the memory multiple times, only manipulating single elements. This is not very cache friendly. And because of that, the constant which is concealed in $O(n \log \log n)$ is comparably big.

Besides, the consumed memory is a bottleneck for big n .

The methods presented below allow us to reduce the quantity of the performed operations, as well as to shorten the consumed memory noticeably.

Sieving till root

Obviously, to find all the prime numbers until n , it will be enough just to perform the sifting only by the prime numbers, which do not exceed the root of n .

```
int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i * i <= n; i++) {
    if (is_prime[i]) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}
```

Such optimization doesn't affect the complexity (indeed, by repeating the proof presented above we'll get the evaluation $n \ln \ln \sqrt{n} + o(n)$, which is asymptotically the same according to the properties of logarithms), though the number of operations will reduce noticeably.

Sieving by the odd numbers only

Since all even numbers (except 2) are composite, we can stop checking even numbers at all. Instead, we need to operate with odd numbers only.

First, it will allow us to halve the needed memory. Second, it will reduce the number of operations performed by algorithm approximately in half.

Memory consumption and speed of operations

We should notice, that these two implementations of the Sieve of Eratosthenes use n bits of memory by using the data structure `vector<bool>`. `vector<bool>` is not a regular container that stores a series of `bool` (as in most computer architectures a `bool` takes one byte of memory). It's a memory-optimization specialization of `vector<T>`, that only consumes $\frac{N}{8}$ bytes of memory.

Modern processors architectures work much more efficiently with bytes than with bits as they usually cannot access bits directly. So underneath the `vector<bool>` stores the bits in a large continuous memory, accesses the memory in blocks of a few bytes, and extracts/sets the bits with bit operations like bit masking and bit shifting.

Because of that there is a certain overhead when you read or write bits with a `vector<bool>`, and quite often using a `vector<char>` (which uses 1 byte for each entry, so 8x the amount of memory) is faster.

However, for the simple implementations of the Sieve of Eratosthenes using a `vector<bool>` is faster. You are limited by how fast you can load the data into the cache, and therefore using less memory gives a big advantage. A benchmark ([link](#)) shows, that using a `vector<bool>` is between 1.4x and 1.7x faster than using a `vector<char>`.

The same considerations also apply to `bitset`. It's also an efficient way of storing bits, similar to `vector<bool>`, so it takes only $\frac{N}{8}$ bytes of memory, but is a bit slower in accessing the elements. In the benchmark above `bitset` performs a bit worse than `vector<bool>`. Another drawback from `bitset` is that you need to know the size at compile time.

Segmented Sieve

It follows from the optimization "sieving till root" that there is no need to keep the whole array `is_prime[1..n]` at all times. For sieving it is enough to just keep the prime numbers until the root of n , i.e. `prime[1..sqrt(n)]`, split the complete range into blocks, and sieve each block separately.

Let s be a constant which determines the size of the block, then we have $\lceil \frac{n}{s} \rceil$ blocks altogether, and the block k ($k = 0 \dots \lceil \frac{n}{s} \rceil$) contains the numbers in a segment $[ks; ks + s - 1]$. We can work on blocks by turns, i.e. for every block k we will go through all the prime numbers (from 1 to $\sqrt{n}$) and perform sieving using them. It is worth noting, that we have to modify the strategy a little bit when handling the first numbers: first, all the prime numbers from $[1; \sqrt{n}]$ shouldn't remove themselves; and second, the numbers 0 and 1 should be marked as non-prime numbers. While working on the last block it should not be forgotten that the last needed number n is not necessarily located at the end of the block.

As discussed previously, the typical implementation of the Sieve of Eratosthenes is limited by the speed how fast you can load data into the CPU caches. By splitting the range of potential prime numbers $[1; n]$ into smaller blocks, we never have to keep multiple blocks in memory at the same time, and all operations are much more cache-friendlier. As we are now no longer limited by the cache speeds, we can replace the `vector<bool>` with a `vector<char>`, and gain some additional performance as the processors can handle read and writes with bytes directly and don't need to rely on bit operations for extracting individual bits. The benchmark ([link](#)) shows, that using a `vector<char>` is about 3x faster

in this situation than using a `vector<bool>`. A word of caution: those numbers might differ depending on architecture, compiler, and optimization levels.

Here we have an implementation that counts the number of primes smaller than or equal to n using block sieving.

```
int count_primes(int n) {
    const int S = 10000;

    vector<int> primes;
    int nsqrt = sqrt(n);
    vector<char> is_prime(nsqrt + 2, true);
    for (int i = 2; i <= nsqrt; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (int j = i * i; j <= nsqrt; j += i)
                is_prime[j] = false;
        }
    }

    int result = 0;
    vector<char> block(S);
    for (int k = 0; k * S <= n; k++) {
        fill(block.begin(), block.end(), true);
        int start = k * S;
        for (int p : primes) {
            int start_idx = (start + p - 1) / p;
            int j = max(start_idx, p) * p - start;
            for (; j < S; j += p)
                block[j] = false;
        }
        if (k == 0)
            block[0] = block[1] = false;
        for (int i = 0; i < S && start + i <= n; i++) {
            if (block[i])
                result++;
        }
    }
    return result;
}
```

The running time of block sieving is the same as for regular sieve of Eratosthenes (unless the size of the blocks is very small), but the needed memory will shorten to $O(\sqrt{n} + S)$ and we have better caching results. On the other hand, there will be a division for each pair of a block and prime number from $[1; \sqrt{n}]$, and that will be far worse for smaller block sizes. Hence, it is necessary to keep balance when selecting the constant S . We achieved the best results for block sizes between 10^4 and 10^5 .

Find primes in range

Sometimes we need to find all prime numbers in a range $[L, R]$ of small size (e.g. $R - L + 1 \approx 1e7$), where R can be very large (e.g. $1e12$).

To solve such a problem, we can use the idea of the Segmented sieve. We pre-generate all prime numbers up to $\sqrt{R}$, and use those primes to mark all composite numbers in the segment $[L, R]$.

```
// BIG NOTE: In case when L = 0 and R > 0, algorithm will return 0 and 1 as primes.
vector<char> segmentedSieve(long long L, long long R) {
    // generate all primes up to sqrt(R)
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }

    vector<char> isPrime(R - L + 1, true);
    for (long long i : primes)
        for (long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
}
```

```
    return isPrime;
}
```

Time complexity of this approach is $O((R - L + 1) \log \log(R) + \sqrt{R} \log \log \sqrt{R})$.

It's also possible that we don't pre-generate all prime numbers:

```
vector<char> segmentedSieveNoPreGen(long long L, long long R) {
    vector<char> isPrime(R - L + 1, true);
    long long lim = sqrt(R);
    for (long long i = 2; i <= lim; ++i)
        for (long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    return isPrime;
}
```

Obviously, the complexity is worse, which is $O((R - L + 1) \log(R) + \sqrt{R})$. However, it still runs very fast in practice.

Linear time modification

We can modify the algorithm in a such a way, that it only has linear time complexity. This approach is described in the article [Linear Sieve](#).

However, this algorithm also has its own weaknesses.

Supposed bug fix

```
vector<char> segmented_sieve(long long L, long long R) {
    // generate all primes up to sqrt(R)
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }

    vector<char> isPrime(R - L + 1, true);
    for (long long i : primes)
        for (long long j = max(i, (L + i - 1) / i) * i; j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 0)
        isPrime[L] = false;
    if (L <= 1)
        isPrime[min(1 - L, R - L)] = false;
    return isPrime;
}
```

```
vector<char> segmented_sieve_no_pre_gen(long long L, long long R) {
    vector<char> isPrime(R - L + 1, true);
    long long lim = sqrt(R);
    for (long long i = 2; i <= lim; ++i)
        for (long long j = max(i, (L + i - 1) / i) * i; j <= R; j += i)
            isPrime[j - L] = false;
    if (L == 0)
        isPrime[L] = false;
    if (L <= 1)
        isPrime[min(1 - L, R - L)] = false;
    return isPrime;
}
```

Practice Problems

- [Leetcode - Four Divisors](#)
- [Leetcode - Count Primes](#)

- [SPOJ - Printing Some Primes](#)
- [SPOJ - A Conjecture of Paul Erdos](#)
- [SPOJ - Primal Fear](#)
- [SPOJ - Primes Triangle \(I\)](#)
- [Codeforces - Almost Prime](#)
- [Codeforces - Sherlock And His Girlfriend](#)
- [SPOJ - Namit in Trouble](#)
- [SPOJ - Bazinga!](#)
- [Project Euler - Prime pair connection](#)
- [SPOJ - N-Factorful](#)
- [SPOJ - Binary Sequence of Prime Numbers](#)
- [UVA 11353 - A Different Kind of Sorting](#)
- [SPOJ - Prime Generator](#)
- [SPOJ - Printing some primes \(hard\)](#)
- [Codeforces - Nodbach Problem](#)
- [Codeforces - Colliders](#)

Contributors:

[Jakob Kogler](#) (36.25%) [ellen-interpret](#) (20.0%) [Le Bao Hiep](#) (14.37%) [Your name](#) (13.75%) [Morass](#) (4.69%) [Mariia Mykhailova](#) (3.12%)
[Oleksandr Kulkov](#) (1.88%) [boxlesscat](#) (1.88%) [Michael Hayter](#) (0.94%) [Tarun Khullar](#) (0.94%) [TrietMinh](#) (0.62%) [jxu](#) (0.62%) [Anant Sharan Pandey](#) (0.31%)
[Joaquin Garmendia Cabrera](#) (0.31%) [zdr256](#) (0.31%)

Translated From: e-maxx.ru

Linear Sieve

Given a number n , find all prime numbers in a segment $[2; n]$.

The standard way of solving a task is to use [the sieve of Eratosthenes](#). This algorithm is very simple, but it has runtime $O(n \log \log n)$.

Although there are a lot of known algorithms with sublinear runtime (i.e. $o(n)$), the algorithm described below is interesting by its simplicity: it isn't any more complex than the classic sieve of Eratosthenes.

Besides, the algorithm given here calculates **factorizations of all numbers** in the segment $[2; n]$ as a side effect, and that can be helpful in many practical applications.

The weakness of the given algorithm is in using more memory than the classic sieve of Eratosthenes: it requires an array of n numbers, while for the classic sieve of Eratosthenes it is enough to have n bits of memory (which is 32 times less).

Thus, it makes sense to use the described algorithm only until for numbers of order 10^7 and not greater.

The algorithm is due to Paul Pritchard. It is a variant of Algorithm 3.3 in (Pritchard, 1987: see references in the end of the article).

Algorithm

Our goal is to calculate **minimum prime factor** $lp[i]$ for every number i in the segment $[2; n]$.

Besides, we need to store the list of all the found prime numbers - let's call it $pr[]$.

We'll initialize the values $lp[i]$ with zeros, which means that we assume all numbers are prime. During the algorithm execution this array will be filled gradually.

Now we'll go through the numbers from 2 to n . We have two cases for the current number i :

- $lp[i] = 0$ - that means that i is prime, i.e. we haven't found any smaller factors for it.
Hence, we assign $lp[i] = i$ and add i to the end of the list $pr[]$.
- $lp[i] \neq 0$ - that means that i is composite, and its minimum prime factor is $lp[i]$.

In both cases we update values of $lp[]$ for the numbers that are divisible by i . However, our goal is to learn to do so as to set a value $lp[]$ at most once for every number. We can do it as follows:

Let's consider numbers $x_j = i \cdot p_j$, where p_j are all prime numbers less than or equal to $lp[i]$ (this is why we need to store the list of all prime numbers).

We'll set a new value $lp[x_j] = p_j$ for all numbers of this form.

The proof of correctness of this algorithm and its runtime can be found after the implementation.

Implementation

```
const int N = 10000000;
vector<int> lp(N+1);
vector<int> pr;

for (int i=2; i <= N; ++i) {
    if (lp[i] == 0) {
        lp[i] = i;
        pr.push_back(i);
    }
    for (int j = 0; i * pr[j] <= N; ++j) {
```

```

        lp[i * pr[j]] = pr[j];
        if (pr[j] == lp[i]) {
            break;
        }
    }
}

```

Correctness Proof

We need to prove that the algorithm sets all values $lp[]$ correctly, and that every value will be set exactly once. Hence, the algorithm will have linear runtime, since all the remaining actions of the algorithm, obviously, work for $O(n)$.

Notice that every number i has exactly one representation in form:

$$i = lp[i] \cdot x,$$

where $lp[i]$ is the minimal prime factor of i , and the number x doesn't have any prime factors less than $lp[i]$, i.e.

$$lp[i] \leq lp[x].$$

Now, let's compare this with the actions of our algorithm: in fact, for every x it goes through all prime numbers it could be multiplied by, i.e. all prime numbers up to $lp[x]$ inclusive, in order to get the numbers in the form given above.

Hence, the algorithm will go through every composite number exactly once, setting the correct values $lp[]$ there. Q.E.D.

Runtime and Memory

Although the running time of $O(n)$ is better than $O(n \log \log n)$ of the classic sieve of Eratosthenes, the difference between them is not so big. In practice the linear sieve runs about as fast as a typical implementation of the sieve of Eratosthenes.

In comparison to optimized versions of the sieve of Eratosthenes, e.g. the segmented sieve, it is much slower.

Considering the memory requirements of this algorithm - an array $lp[]$ of length n , and an array of $pr[]$ of length $\frac{n}{\ln n}$, this algorithm seems to be worse than the classic sieve in every way.

However, its redeeming quality is that this algorithm calculates an array $lp[]$, which allows us to find factorization of any number in the segment $[2; n]$ in the time of the size order of this factorization. Moreover, using just one extra array will allow us to avoid divisions when looking for factorization.

Knowing the factorizations of all numbers is very useful for some tasks, and this algorithm is one of the few which allow to find them in linear time.

References

- Paul Pritchard, **Linear Prime-Number Sieves: a Family Tree**, Science of Computer Programming, vol. 9 (1987), pp.17-35.

Contributors:

[ellen-interpret](#) (47.96%) [Mariia Mykhailova](#) (28.57%) [Jakob Kogler](#) (10.2%) [Oleksandr Kulkov](#) (10.2%) [paulpritchard](#) (2.04%) [boxlesscat](#) (1.02%)

Original

Primality tests

This article describes multiple algorithms to determine if a number is prime or not.

Trial division

By definition a prime number doesn't have any divisors other than 1 and itself. A composite number has at least one additional divisor, let's call it d . Naturally $\frac{n}{d}$ is also a divisor of n . It's easy to see, that either $d \leq \sqrt{n}$ or $\frac{n}{d} \leq \sqrt{n}$, therefore one of the divisors d and $\frac{n}{d}$ is $\leq \sqrt{n}$. We can use this information to check for primality.

We try to find a non-trivial divisor, by checking if any of the numbers between 2 and $\sqrt{n}$ is a divisor of n . If it is a divisor, then n is definitely not prime, otherwise it is.

```
bool isPrime(int x) {
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0)
            return false;
    }
    return x >= 2;
}
```

This is the simplest form of a prime check. You can optimize this function quite a bit, for instance by only checking all odd numbers in the loop, since the only even prime number is 2. Multiple such optimizations are described in the article about [integer factorization](#).

Fermat primality test

This is a probabilistic test.

Fermat's little theorem (see also [Euler's totient function](#)) states, that for a prime number p and a coprime integer a the following equation holds:

$$a^{p-1} \equiv 1 \pmod{p}$$

In general this theorem doesn't hold for composite numbers.

This can be used to create a primality test. We pick an integer $2 \leq a \leq p-2$, and check if the equation holds or not. If it doesn't hold, e.g. $a^{p-1} \not\equiv 1 \pmod{p}$, we know that p cannot be a prime number. In this case we call the base a a *Fermat witness* for the compositeness of p .

However it is also possible, that the equation holds for a composite number. So if the equation holds, we don't have a proof for primality. We only can say that p is *probably prime*. If it turns out that the number is actually composite, we call the base a a *Fermat liar*.

By running the test for all possible bases a , we can actually prove that a number is prime. However this is not done in practice, since this is a lot more effort than just doing *trial division*. Instead the test will be repeated multiple times with random choices for a . If we find no witness for the compositeness, it is very likely that the number is in fact prime.

```
bool probablyPrimeFermat(int n, int iter=5) {
    if (n < 4)
        return n == 2 || n == 3;

    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (binpower(a, n - 1, n) != 1)
            return false;
    }
    return true;
}
```

We use [Binary Exponentiation](#) to efficiently compute the power a^{p-1} .

There is one bad news though: there exist some composite numbers where $a^{n-1} \equiv 1 \pmod n$ holds for all a coprime to n , for instance for the number $561 = 3 \cdot 11 \cdot 17$. Such numbers are called *Carmichael numbers*. The Fermat primality test can identify these numbers only, if we have immense luck and choose a base a with $\gcd(a, n) \neq 1$.

The Fermat test is still being used in practice, as it is very fast and Carmichael numbers are very rare. E.g. there only exist 646 such numbers below 10^9 .

Miller-Rabin primality test

The Miller-Rabin test extends the ideas from the Fermat test.

For an odd number n , $n - 1$ is even and we can factor out all powers of 2. We can write:

$$n - 1 = 2^s \cdot d, \text{ with } d \text{ odd.}$$

This allows us to factorize the equation of Fermat's little theorem:

$$\begin{aligned} a^{n-1} \equiv 1 \pmod n &\iff a^{2^s d} - 1 \equiv 0 \pmod n \\ &\iff (a^{2^{s-1}d} + 1)(a^{2^{s-1}d} - 1) \equiv 0 \pmod n \\ &\iff (a^{2^{s-1}d} + 1)(a^{2^{s-2}d} + 1)(a^{2^{s-2}d} - 1) \equiv 0 \pmod n \\ &\vdots \\ &\iff (a^{2^{s-1}d} + 1)(a^{2^{s-2}d} + 1) \cdots (a^d + 1)(a^d - 1) \equiv 0 \pmod n \end{aligned}$$

If n is prime, then n has to divide one of these factors. And in the Miller-Rabin primality test we check exactly that statement, which is a more stricter version of the statement of the Fermat test. For a base $2 \leq a \leq n - 2$ we check if either

$$a^d \equiv 1 \pmod n$$

holds or

$$a^{2^r d} \equiv -1 \pmod n$$

holds for some $0 \leq r \leq s - 1$.

If we found a base a which doesn't satisfy any of the above equalities, then we found a *witness* for the compositeness of n . In this case we have proven that n is not a prime number.

Similar to the Fermat test, it is also possible that the set of equations is satisfied for a composite number. In that case the base a is called a *strong liar*. If a base a satisfies the equations (one of them), n is only *strong probable prime*. However, there are no numbers like the Carmichael numbers, where all non-trivial bases lie. In fact it is possible to show, that at most $\frac{1}{4}$ of the bases can be strong liars. If n is composite, we have a probability of $\geq 75\%$ that a random base will tell us that it is composite. By doing multiple iterations, choosing different random bases, we can tell with very high probability if the number is truly prime or if it is composite.

Here is an implementation for 64 bit integer.

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
```

```

        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};

bool MillerRabin(u64 n, int iter=5) { // returns true if n is probably prime, else returns false.
    if (n < 4)
        return n == 2 || n == 3;

    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s++;
    }

    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s))
            return false;
    }
    return true;
}

```

Before the Miller-Rabin test you can test additionally if one of the first few prime numbers is a divisor. This can speed up the test by a lot, since most composite numbers have very small prime divisors. E.g. 88% of all numbers have a prime factor smaller than 100.

Deterministic version

Miller showed that it is possible to make the algorithm deterministic by only checking all bases $\leq O((\ln n)^2)$. Bach later gave a concrete bound, it is only necessary to test all bases $a \leq 2 \ln(n)^2$.

This is still a pretty large number of bases. So people have invested quite a lot of computation power into finding lower bounds. It turns out, for testing a 32 bit integer it is only necessary to check the first 4 prime bases: 2, 3, 5 and 7. The smallest composite number that fails this test is $3 \cdot 215 \cdot 031 \cdot 751 = 151 \cdot 751 \cdot 28351$. And for testing 64 bit integer it is enough to check the first 12 prime bases: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, and 37.

This results in the following deterministic implementation:

```

bool MillerRabin(u64 n) { // returns true if n is prime, else returns false.
    if (n < 2)
        return false;

    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }

    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}

```

It's also possible to do the check with only 7 bases: 2, 325, 9375, 28178, 450775, 9780504 and 1795265022. However, since these numbers (except 2) are not prime, you need to check additionally if the number you are checking is equal to any prime divisor of those bases: 2, 3, 5, 13, 19, 73, 193, 407521, 299210837.

Practice Problems

- [SPOJ - Prime or Not](#)
- [Project euler - Investigating a Prime Pattern](#)

Contributors:

[Jakob Kogler](#) (91.41%) [gampu](#) (3.17%) [Oleksandr Kulkov](#) (2.71%) [boxlesscat](#) (1.36%) [HKalbasi](#) (0.45%) [Jan Pokorný](#) (0.45%) [Siddharth](#) (0.45%)

Original

Integer factorization

In this article we list several algorithms for the factorization of integers, each of which can be either fast or varying levels of slow depending on their input.

Notice, if the number that you want to factorize is actually a prime number, most of the algorithms will run very slowly. This is especially true for Fermat's, Pollard's p-1 and Pollard's rho factorization algorithms. Therefore, it makes the most sense to perform a probabilistic (or a fast deterministic) [primality test](#) before trying to factorize the number.

Trial division

This is the most basic algorithm to find a prime factorization.

We divide by each possible divisor d . It can be observed that it is impossible for all prime factors of a composite number n to be bigger than $\sqrt{n}$. Therefore, we only need to test the divisors $2 \leq d \leq \sqrt{n}$, which gives us the prime factorization in $O(\sqrt{n})$. (This is [pseudo-polynomial time](#), i.e. polynomial in the value of the input but exponential in the number of bits of the input.)

The smallest divisor must be a prime number. We remove the factored number, and continue the process. If we cannot find any divisor in the range $[2; \sqrt{n}]$, then the number itself has to be prime.

```
vector<long long> trial_division1(long long n) {
    vector<long long> factorization;
    for (long long d = 2; d * d <= n; d++) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

Wheel factorization

This is an optimization of the trial division. Once we know that the number is not divisible by 2, we don't need to check other even numbers. This leaves us with only 50% of the numbers to check. After factoring out 2, and getting an odd number, we can simply start with 3 and only count other odd numbers.

```
vector<long long> trial_division2(long long n) {
    vector<long long> factorization;
    while (n % 2 == 0) {
        factorization.push_back(2);
        n /= 2;
    }
    for (long long d = 3; d * d <= n; d += 2) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

This method can be extended further. If the number is not divisible by 3, we can also ignore all other multiples of 3 in the future computations. So we only need to check the numbers 5, 7, 11, 13, 17, 19, 23, We can observe a pattern of these remaining numbers. We need to check all numbers with $d \bmod 6 = 1$ and $d \bmod 6 = 5$. So this leaves us with only 33.3% percent of the numbers to check. We can implement this by factoring out the primes 2 and 3 first, after which we start with 5 and only count remainders 1 and 5 modulo 6.

Here is an implementation for the prime number 2, 3 and 5. It is convenient to store the skipping strides in an array.

```
vector<long long> trial_division3(long long n) {
    vector<long long> factorization;
    for (int d : {2, 3, 5}) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    static array<int, 8> increments = {4, 2, 4, 2, 4, 6, 2, 6};
    int i = 0;
    for (long long d = 7; d * d <= n; d += increments[i++]) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
        if (i == 8)
            i = 0;
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

If we continue extending this method to include even more primes, better percentages can be reached, but the skip lists will become larger.

Precomputed primes

Extending the wheel factorization method indefinitely, we will only be left with prime numbers to check. A good way of checking this is to precompute all prime numbers with the [Sieve of Eratosthenes](#) until $\sqrt{n}$, and test them individually.

```
vector<long long> primes;

vector<long long> trial_division4(long long n) {
    vector<long long> factorization;
    for (long long d : primes) {
        if (d * d > n)
            break;
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if (n > 1)
        factorization.push_back(n);
    return factorization;
}
```

Fermat's factorization method

We can write an odd composite number $n = p \cdot q$ as the difference of two squares $n = a^2 - b^2$:

$$n = \left(\frac{p+q}{2} \right)^2 - \left(\frac{p-q}{2} \right)^2$$

Fermat's factorization method tries to exploit this fact by guessing the first square a^2 , and checking if the remaining part, $b^2 = a^2 - n$, is also a square number. If it is, then we have found the factors $a - b$ and $a + b$ of n .

```
int fermat(int n) {
    int a = ceil(sqrt(n));
    int b2 = a*a - n;
    int b = round(sqrt(b2));
    while (b * b != b2) {
        a = a + 1;
        b2 = a*a - n;
        b = round(sqrt(b2));
    }
}
```

```
    return a - b;
}
```

This factorization method can be very fast if the difference between the two factors p and q is small. The algorithm runs in $O(|p - q|)$ time. In practice though, this method is rarely used. Once factors become further apart, it is extremely slow.

However, there are still a large number of optimization options regarding this approach. By looking at the squares a^2 modulo a fixed small number, it can be observed that certain values a don't have to be viewed, since they cannot produce a square number $a^2 - n$.

Pollard's $p - 1$ method

It is very likely that a number n has at least one prime factor p such that $p - 1$ is **B-powersmooth** for small B . An integer m is said to be B-powersmooth if every prime power dividing m is at most B . Formally, let $B \geq 1$ and let m be any positive integer. Suppose the prime factorization of m is $m = \prod q_i^{e_i}$, where each q_i is a prime and $e_i \geq 1$. Then m is B-powersmooth if, for all i , $q_i^{e_i} \leq B$. E.g. the prime factorization of 4817191 is $1303 \cdot 3697$. And the values, $1303 - 1$ and $3697 - 1$, are 31-powersmooth and 16-powersmooth respectively, because $1303 - 1 = 2 \cdot 3 \cdot 7 \cdot 31$ and $3697 - 1 = 2^4 \cdot 3 \cdot 7 \cdot 11$. In 1974 John Pollard invented a method to extract factors p , s.t. $p - 1$ is B-powersmooth, from a composite number.

The idea comes from [Fermat's little theorem](#). Let a factorization of n be $n = p \cdot q$. It says that if a is coprime to p , the following statement holds:

$$a^{p-1} \equiv 1 \pmod{p}$$

This also means that

$$(a^{p-1})^k \equiv a^{k(p-1)} \equiv 1 \pmod{p}.$$

So for any M with $p - 1 \mid M$ we know that $a^M \equiv 1$. This means that $a^M - 1 = p \cdot r$, and because of that also $p \mid \gcd(a^M - 1, n)$.

Therefore, if $p - 1$ for a factor p of n divides M , we can extract a factor using [Euclid's algorithm](#).

It is clear, that the smallest M that is a multiple of every B-powersmooth number is $\text{lcm}(1, 2, 3, 4, \dots, B)$. Or alternatively:

$$M = \prod_{\text{prime } q \leq B} q^{\lfloor \log_q B \rfloor}$$

Notice, if $p - 1$ divides M for all prime factors p of n , then $\gcd(a^M - 1, n)$ will just be n . In this case we don't receive a factor. Therefore, we will try to perform the gcd multiple times, while we compute M .

Some composite numbers don't have factors p s.t. $p - 1$ is B-powersmooth for small B . For example, for the composite number $100\,000\,000\,000\,493 = 763\,013 \cdot 131\,059\,365\,961$, values $p - 1$ are 190 753-powersmooth and 1 092 161 383-powersmooth correspondingly. We will have to choose $B \geq 190\,753$ to factorize the number.

In the following implementation we start with $B = 10$ and increase B after each iteration.

```
long long pollards_p_minus_1(long long n) {
    int B = 10;
    long long g = 1;
    while (B <= 1000000 && g < n) {
        long long a = 2 + rand() % (n - 3);
        g = gcd(a, n);
        if (g > 1)
            return g;

        // compute a^M
        for (int p : primes) {
            if (p >= B)
                continue;
            long long p_power = 1;
            while (p_power * p <= B)
                p_power *= p;
        }
        B *= 2;
    }
    return g;
}
```

```

    a = power(a, p_power, n);

    g = gcd(a - 1, n);
    if (g > 1 && g < n)
        return g;
    }
    B *= 2;
}
return 1;
}

```

Observe that this is a probabilistic algorithm. A consequence of this is that there is a possibility of the algorithm being unable to find a factor at all.

The complexity is $O(B \log B \log^2 n)$ per iteration.

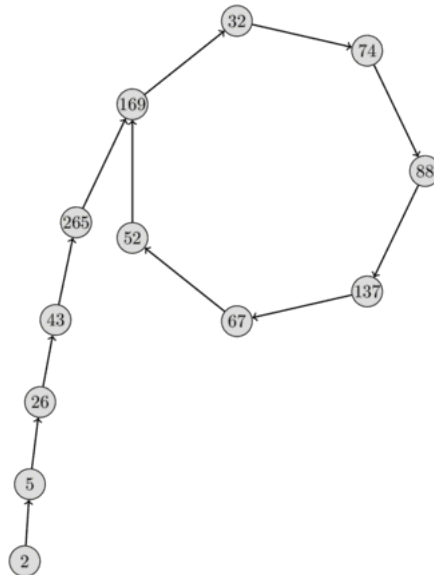
Pollard's rho algorithm

Pollard's Rho Algorithm is yet another factorization algorithm from John Pollard.

Let the prime factorization of a number be $n = pq$. The algorithm looks at a pseudo-random sequence $\{x_i\} = \{x_0, f(x_0), f(f(x_0)), \dots\}$ where f is a polynomial function, usually $f(x) = (x^2 + c) \bmod n$ is chosen with $c = 1$.

In this instance, we are not interested in the sequence $\{x_i\}$. We are more interested in the sequence $\{x_i \bmod p\}$. Since f is a polynomial function, and all the values are in the range $[0; p)$, this sequence will eventually converge into a loop. The **birthday paradox** actually suggests that the expected number of elements is $O(\sqrt{p})$ until the repetition starts. If p is smaller than $\sqrt{n}$, the repetition will likely start in $O(\sqrt{n})$.

Here is a visualization of such a sequence $\{x_i \bmod p\}$ with $n = 2206637$, $p = 317$, $x_0 = 2$ and $f(x) = x^2 + 1$. From the form of the sequence you can see very clearly why the algorithm is called Pollard's ρ algorithm.



Yet, there is still an open question. How can we exploit the properties of the sequence $\{x_i \bmod p\}$ to our advantage without even knowing the number p itself?

It's actually quite easy. There is a cycle in the sequence $\{x_i \bmod p\}_{i \leq j}$ if and only if there are two indices $s, t \leq j$ such that $x_s \equiv x_t \bmod p$. This equation can be rewritten as $x_s - x_t \equiv 0 \bmod p$ which is the same as $p \mid \gcd(x_s - x_t, n)$.

Therefore, if we find two indices s and t with $g = \gcd(x_s - x_t, n) > 1$, we have found a cycle and also a factor g of n . It is possible that $g = n$. In this case we haven't found a proper factor, so we must repeat the algorithm with a different parameter (different starting value x_0 , different constant c in the polynomial function f).

To find the cycle, we can use any common cycle detection algorithm.

Floyd's cycle-finding algorithm

This algorithm finds a cycle by using two pointers moving over the sequence at differing speeds. During each iteration, the first pointer will advance one element over, while the second pointer advances to every other element. Using this idea it is easy to observe that if there is a cycle, at some point the second pointer will come around to meet the first one during the loops. If the cycle length is λ and the μ is the first index at which the cycle starts, then the algorithm will run in $O(\lambda + \mu)$ time.

This algorithm is also known as the [Tortoise and Hare algorithm](#), based on the tale in which a tortoise (the slow pointer) and a hare (the faster pointer) have a race.

It is actually possible to determine the parameter λ and μ using this algorithm (also in $O(\lambda + \mu)$ time and $O(1)$ space). When a cycle is detected, the algorithm will return 'True'. If the sequence doesn't have a cycle, then the function will loop endlessly. However, using Pollard's Rho Algorithm, this can be prevented.

```
function floyd(f, x0):
    tortoise = x0
    hare = f(x0)
    while tortoise != hare:
        tortoise = f(tortoise)
        hare = f(f(hare))
    return true
```

Implementation

First, here is an implementation using the **Floyd's cycle-finding algorithm**. The algorithm generally runs in $O(\sqrt{n} \log(n))$ time.

```
long long mult(long long a, long long b, long long mod) {
    return (__int128)a * b % mod;
}

long long f(long long x, long long c, long long mod) {
    return (mult(x, x, mod) + c) % mod;
}

long long rho(long long n, long long x0=2, long long c=1) {
    long long x = x0;
    long long y = x0;
    long long g = 1;
    while (g == 1) {
        x = f(x, c, n);
        y = f(y, c, n);
        y = f(y, c, n);
        g = gcd(abs(x - y), n);
    }
    return g;
}
```

The following table shows the values of x and y during the algorithm for $n = 2206637$, $x_0 = 2$ and $c = 1$.

i	$x_i \bmod n$	$x_{2i} \bmod n$	$x_i \bmod 317$	$x_{2i} \bmod 317$	$\gcd(x_i - x_{2i}, n)$
0	2	2	2	2	—
1	5	26	5	26	1
2	26	458330	26	265	1
3	677	1671573	43	32	1
4	458330	641379	265	88	1
5	1166412	351937	169	67	1
6	1671573	1264682	32	169	1
7	2193080	2088470	74	74	317

The implementation uses a function `mult`, that multiplies two integers $\leq 10^{18}$ without overflow by using a GCC's type `__int128` for 128-bit integer. If GCC is not available, you can using a similar idea as [binary exponentiation](#).

```

long long mult(long long a, long long b, long long mod) {
    long long result = 0;
    while (b) {
        if (b & 1)
            result = (result + a) % mod;
        a = (a + a) % mod;
        b >>= 1;
    }
    return result;
}

```

Alternatively you can also implement the [Montgomery multiplication](#).

As stated previously, if n is composite and the algorithm returns n as factor, you have to repeat the procedure with different parameters x_0 and c . E.g. the choice $x_0 = c = 1$ will not factor $25 = 5 \cdot 5$. The algorithm will return 25. However, the choice $x_0 = 1, c = 2$ will factor it.

Brent's algorithm

Brent implements a similar method to Floyd, using two pointers. The difference being that instead of advancing the pointers by one and two places respectively, they are advanced by powers of two. As soon as 2^i is greater than λ and μ , we will find the cycle.

```

function floyd(f, x0):
    tortoise = x0
    hare = f(x0)
    l = 1
    while tortoise != hare:
        tortoise = hare
        repeat l times:
            hare = f(hare)
            if tortoise == hare:
                return true
        l *= 2
    return true

```

Brent's algorithm also runs in linear time, but is generally faster than Floyd's, since it uses less evaluations of the function f .

Implementation

The straightforward implementation of Brent's algorithm can be sped up by omitting the terms $x_1 - x_k$ if $k < \frac{3 \cdot l}{2}$. In addition, instead of performing the gcd computation at every step, we multiply the terms and only actually check gcd every few steps and backtrack if overshoot.

```

long long brent(long long n, long long x0=2, long long c=1) {
    long long x = x0;
    long long g = 1;
    long long q = 1;
    long long xs, y;

    int m = 128;
    int l = 1;
    while (g == 1) {
        y = x;
        for (int i = 1; i < l; i++)
            x = f(x, c, n);
        int k = 0;
        while (k < l && g == 1) {
            xs = x;
            for (int i = 0; i < m && i < l - k; i++) {
                x = f(x, c, n);
                q = mult(q, abs(y - x), n);
            }
            g = gcd(q, n);
            k += m;
        }
        l *= 2;
    }
    if (g == n) {
        do {
            xs = f(xs, c, n);
            g = gcd(abs(xs - y), n);
        } while (g == 1);
    }
}

```

```
    return g;  
}
```

The combination of a trial division for small prime numbers together with Brent's version of Pollard's rho algorithm makes a very powerful factorization algorithm.

Practice Problems

- [SPOJ - FACT0](#)
- [SPOJ - FACT1](#)
- [SPOJ - FACT2](#)
- [GCPC 15 - Divisions](#)

Contributors:

[Jakob Kogler](#) (82.98%) [Chloe](#) (9.56%) [Oleksandr Kulkov](#) (3.27%) [t0wbo2t](#) (1.86%) [Michael Hayter](#) (0.7%) [godspeedfw](#) (0.7%) [Jan Pokorný](#) (0.47%)
[jxu](#) (0.23%) [kyomukyomupurin](#) (0.23%)

Translated

From: e-maxx.ru

Euler's totient function

Euler's totient function, also known as **phi-function** $\phi(n)$, counts the number of integers between 1 and n inclusive, which are coprime to n . Two numbers are coprime if their greatest common divisor equals 1 (1 is considered to be coprime to any number).

Here are values of $\phi(n)$ for the first few positive integers:

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
$\phi(n)$	1	1	2	2	4	2	6	4	6	4	10	4	12	6	8	8	16	6	18	8	12

Properties

The following properties of Euler totient function are sufficient to calculate it for any number:

- If p is a prime number, then $\gcd(p, q) = 1$ for all $1 \leq q < p$. Therefore we have:

$$\phi(p) = p - 1.$$

- If p is a prime number and $k \geq 1$, then there are exactly p^k/p numbers between 1 and p^k that are divisible by p . Which gives us:

$$\phi(p^k) = p^k - p^{k-1}.$$

- If a and b are relatively prime, then:

$$\phi(ab) = \phi(a) \cdot \phi(b).$$

This relation is not trivial to see. It follows from the [Chinese remainder theorem](#). The Chinese remainder theorem guarantees, that for each $0 \leq x < a$ and each $0 \leq y < b$, there exists a unique $0 \leq z < ab$ with $z \equiv x \pmod{a}$ and $z \equiv y \pmod{b}$. It's not hard to show that z is coprime to ab if and only if x is coprime to a and y is coprime to b . Therefore the amount of integers coprime to ab is equal to product of the amounts of a and b .

- In general, for not coprime a and b , the equation

$$\phi(ab) = \phi(a) \cdot \phi(b) \cdot \frac{d}{\phi(d)}$$

with $d = \gcd(a, b)$ holds.

Thus, using the first three properties, we can compute $\phi(n)$ through the factorization of n (decomposition of n into a product of its prime factors). If $n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$, where p_i are prime factors of n ,

$$\begin{aligned}
 \phi(n) &= \phi(p_1^{a_1}) \cdot \phi(p_2^{a_2}) \cdots \phi(p_k^{a_k}) \\
 &= (p_1^{a_1} - p_1^{a_1-1}) \cdot (p_2^{a_2} - p_2^{a_2-1}) \cdots (p_k^{a_k} - p_k^{a_k-1}) \\
 &= p_1^{a_1} \cdot \left(1 - \frac{1}{p_1}\right) \cdot p_2^{a_2} \cdot \left(1 - \frac{1}{p_2}\right) \cdots p_k^{a_k} \cdot \left(1 - \frac{1}{p_k}\right) \\
 &= n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right)
 \end{aligned}$$

Implementation

Here is an implementation using factorization in $O(\sqrt{n})$:

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

Euler totient function from 1 to n in $O(n \log \log n)$

If we need the totient of all numbers between 1 and n, then factorizing all n numbers is not efficient. We can use the same idea as the [Sieve of Eratosthenes](#). It is still based on the property shown above, but instead of updating the temporary result for each prime factor for each number, we find all prime numbers and for each one update the temporary results of all numbers that are divisible by that prime number.

Since this approach is basically identical to the Sieve of Eratosthenes, the complexity will also be the same: $O(n \log \log n)$

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

Divisor sum property

This interesting property was established by Gauss:

$$\sum_{d|n} \phi(d) = n$$

Here the sum is over all positive divisors d of n.

For instance the divisors of 10 are 1, 2, 5 and 10. Hence $\phi(1) + \phi(2) + \phi(5) + \phi(10) = 1 + 1 + 4 + 4 = 10$.

Finding the totient from 1 to n using the divisor sum property

The divisor sum property also allows us to compute the totient of all numbers between 1 and n. This implementation is a little simpler than the previous implementation based on the Sieve of Eratosthenes, however also has a slightly worse complexity: $O(n \log n)$

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    phi[0] = 0;
    phi[1] = 1;
    for (int i = 2; i <= n; i++)
        phi[i] = i - 1;

    for (int i = 2; i <= n; i++)
        for (int j = 2 * i; j <= n; j += i)
            phi[j] -= phi[i];
}
```

Application in Euler's theorem

The most famous and important property of Euler's totient function is expressed in **Euler's theorem**:

$$a^{\phi(m)} \equiv 1 \pmod{m} \quad \text{if } a \text{ and } m \text{ are relatively prime.}$$

In the particular case when m is prime, Euler's theorem turns into **Fermat's little theorem**:

$$a^{m-1} \equiv 1 \pmod{m}$$

Euler's theorem and Euler's totient function occur quite often in practical applications, for example both are used to compute the [modular multiplicative inverse](#).

As immediate consequence we also get the equivalence:

$$a^n \equiv a^{n \bmod \phi(m)} \pmod{m}$$

This allows computing $x^n \bmod m$ for very big n , especially if n is the result of another computation, as it allows to compute n under a modulo.

Group Theory

$\phi(n)$ is the [order of the multiplicative group mod n](#) $(\mathbb{Z}/n\mathbb{Z})^\times$, that is the group of units (elements with multiplicative inverses). The elements with multiplicative inverses are precisely those coprime to n .

The [multiplicative order](#) of an element $a \bmod n$, denoted $\text{ord}_n(a)$, is the smallest $k > 0$ such that $a^k \equiv 1 \pmod{m}$. $\text{ord}_n(a)$ is the size of the subgroup generated by a , so by Lagrange's Theorem, the multiplicative order of any a must divide $\phi(n)$. If the multiplicative order of a is $\phi(n)$, the largest possible, then a is a [primitive root](#) and the group is cyclic by definition.

Generalization

There is a less known version of the last equivalence, that allows computing $x^n \bmod m$ efficiently for not coprime x and m . For arbitrary x , m and $n \geq \log_2 m$:

$$x^n \equiv x^{\phi(m) + [n \bmod \phi(m)]} \pmod{m}$$

Proof:

Let $p_1, \dots, p_t$ be common prime divisors of x and m , and k_i their exponents in m . With those we define $a = p_1^{k_1} \dots p_t^{k_t}$, which makes $\frac{m}{a}$ coprime to x . And let k be the smallest number such that a divides x^k . Assuming $n \geq k$, we can write:

$$\begin{aligned}
 x^n \bmod m &= \frac{x^k}{a} a x^{n-k} \bmod m \\
 &= \frac{x^k}{a} (a x^{n-k} \bmod m) \bmod m \\
 &= \frac{x^k}{a} (a x^{n-k} \bmod a \frac{m}{a}) \bmod m \\
 &= \frac{x^k}{a} a (x^{n-k} \bmod \frac{m}{a}) \bmod m \\
 &= x^k (x^{n-k} \bmod \frac{m}{a}) \bmod m
 \end{aligned}$$

The equivalence between the third and forth line follows from the fact that $ab \bmod ac = a(b \bmod c)$. Indeed if $b = cd + r$ with $r < c$, then $ab = acd + ar$ with $ar < ac$.

Since x and $\frac{m}{a}$ are coprime, we can apply Euler's theorem and get the efficient (since k is very small; in fact $k \leq \log_2 m$) formula:

$$x^n \bmod m = x^k (x^{n-k \bmod \phi(\frac{m}{a})} \bmod \frac{m}{a}) \bmod m.$$

This formula is difficult to apply, but we can use it to analyze the behavior of $x^n \bmod m$. We can see that the sequence of powers $(x^1 \bmod m, x^2 \bmod m, x^3 \bmod m, \dots)$ enters a cycle of length $\phi(\frac{m}{a})$ after the first k (or less) elements. $\phi(\frac{m}{a})$ divides $\phi(m)$ (because a and $\frac{m}{a}$ are coprime we have $\phi(a) \cdot \phi(\frac{m}{a}) = \phi(m)$), therefore we can also say that the period has length $\phi(m)$. And since $\phi(m) \geq \log_2 m \geq k$, we can conclude the desired, much simpler, formula:

$$x^n \equiv x^{\phi(m)} x^{(n - \phi(m)) \bmod \phi(m)} \bmod m \equiv x^{\phi(m) + [n \bmod \phi(m)]} \bmod m.$$

Practice Problems

- [SPOJ #4141 "Euler Totient Function" \[Difficulty: CakeWalk\]](#)
- [UVA #10179 "Irreducible Basic Fractions" \[Difficulty: Easy\]](#)
- [UVA #10299 "Relatives" \[Difficulty: Easy\]](#)
- [UVA #11327 "Enumerating Rational Numbers" \[Difficulty: Medium\]](#)
- [TIMUS #1673 "Admission to Exam" \[Difficulty: High\]](#)
- [UVA 10990 - Another New Function](#)
- [Codechef - Golu and Sweetness](#)
- [SPOJ - LCM Sum](#)
- [GYM - Simple Calculations \(F\)](#)
- [UVA 13132 - Laser Mirrors](#)
- [SPOJ - GCDEX](#)
- [UVA 12995 - Farey Sequence](#)
- [SPOJ - Totient in Permutation \(easy\)](#)
- [LOJ - Mathematically Hard](#)
- [SPOJ - Totient Extreme](#)
- [SPOJ - Playing with GCD](#)
- [SPOJ - G Force](#)
- [SPOJ - Smallest Inverse Euler Totient Function](#)
- [Codeforces - Power Tower](#)
- [Kattis - Exponential](#)
- [LeetCode - 372. Super Pow](#)

- [Codeforces - The Holmes Children](#)
- [Codeforces - Small GCD](#)

Contributors:

[Jakob Kogler](#) (38.76%) [Tanmay Sinha](#) (22.97%) [samsidx](#) (11.48%) [Morass](#) (5.74%) [Alexander Kulkov](#) (4.31%) [Mariia Mykhailova](#) (3.35%)
[Oleksandr Kulkov](#) (2.87%) [Rodion Gorkovenko](#) (2.39%) [jxu](#) (2.39%) [Anton Algmyr](#) (0.96%) [wiktorkuchta](#) (0.96%) [Angkur Mondal](#) (0.48%)
[Aniket Manoj Mahabole](#) (0.48%) [Bhuvnesh Jain](#) (0.48%) [Harpreet Singh](#) (0.48%) [Harsh Kumar Bhartiya](#) (0.48%) [Luke Videckis](#) (0.48%)
[Rohit Kharsan](#) (0.48%) [SiddharthEEE](#) (0.48%)

Original

Number of divisors / sum of divisors

In this article we discuss how to compute the number of divisors $d(n)$ and the sum of divisors $\sigma(n)$ of a given number n .

Number of divisors

It should be obvious that the prime factorization of a divisor d has to be a subset of the prime factorization of n , e.g. $6 = 2 \cdot 3$ is a divisor of $60 = 2^2 \cdot 3 \cdot 5$. So we only need to find all different subsets of the prime factorization of n .

Usually the number of subsets is 2^x for a set with x elements. However this is no longer true, if there are repeated elements in the set. In our case some prime factors may appear multiple times in the prime factorization of n .

If a prime factor p appears e times in the prime factorization of n , then we can use the factor p up to e times in the subset. Which means we have $e + 1$ choices.

Therefore if the prime factorization of n is $p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, where p_i are distinct prime numbers, then the number of divisors is:

$$d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdots (e_k + 1)$$

A way of thinking about it is the following:

- If there is only one distinct prime divisor $n = p_1^{e_1}$, then there are obviously $e_1 + 1$ divisors $(1, p_1, p_1^2, \dots, p_1^{e_1})$.
- If there are two distinct prime divisors $n = p_1^{e_1} \cdot p_2^{e_2}$, then you can arrange all divisors in form of a tabular.

	1	p_2	p_2^2	...	$p_2^{e_2}$
1	1	p_2	p_2^2	...	$p_2^{e_2}$
p_1	p_1	$p_1 \cdot p_2$	$p_1 \cdot p_2^2$	...	$p_1 \cdot p_2^{e_2}$
p_1^2	p_1^2	$p_1^2 \cdot p_2$	$p_1^2 \cdot p_2^2$	...	$p_1^2 \cdot p_2^{e_2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$p_1^{e_1}$	$p_1^{e_1}$	$p_1^{e_1} \cdot p_2$	$p_1^{e_1} \cdot p_2^2$	...	$p_1^{e_1} \cdot p_2^{e_2}$

So the number of divisors is trivially $(e_1 + 1) \cdot (e_2 + 1)$.

- A similar argument can be made if there are more then two distinct prime factors.

```
long long numberOfDivisors(long long num) {
    long long total = 1;
    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);
            total *= e + 1;
        }
    }
}
```

```

    }
    if (num > 1) {
        total *= 2;
    }
    return total;
}

```

Sum of divisors

We can use the same argument of the previous section.

- If there is only one distinct prime divisor $n = p_1^{e_1}$, then the sum is:

$$1 + p_1 + p_1^2 + \dots + p_1^{e_1} = \frac{p_1^{e_1+1} - 1}{p_1 - 1}$$

- If there are two distinct prime divisors $n = p_1^{e_1} \cdot p_2^{e_2}$, then we can make the same table as before. The only difference is that now we now want to compute the sum instead of counting the elements. It is easy to see, that the sum of each combination can be expressed as:

$$(1 + p_1 + p_1^2 + \dots + p_1^{e_1}) \cdot (1 + p_2 + p_2^2 + \dots + p_2^{e_2})$$

$$= \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1}$$

- In general, for $n = p_1^{e_1} \cdot p_2^{e_2} \dots p_k^{e_k}$ we receive the formula:

$$\sigma(n) = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1} \dots \frac{p_k^{e_k+1} - 1}{p_k - 1}$$

```

long long SumOfDivisors(long long num) {
    long long total = 1;

    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);

            long long sum = 0, pow = 1;
            do {
                sum += pow;
                pow *= i;
            } while (e-- > 0);
            total *= sum;
        }
    }
    if (num > 1) {
        total *= (1 + num);
    }
    return total;
}

```

Multiplicative functions

A multiplicative function is a function $f(x)$ which satisfies

$$f(a \cdot b) = f(a) \cdot f(b)$$

if a and b are coprime.

Both $d(n)$ and $\sigma(n)$ are multiplicative functions.

Multiplicative functions have a huge variety of interesting properties, which can be very useful in number theory problems. For instance the Dirichlet convolution of two multiplicative functions is also multiplicative.

Practice Problems

- [SPOJ - COMDIV](#)
- [SPOJ - DIVSUM](#)
- [SPOJ - DIVSUM2](#)

Contributors:

[Jakob Kogler](#) (55.81%) [Md Shamim Ahmmed](#) (20.16%) [Md Shamim Ahmad](#) (17.83%) [Oleksandr Kulkov](#) (5.43%) [Siddharth S](#) (0.78%)

Translated From: e-maxx.ru

Modular Multiplicative Inverse

Definition

A [modular multiplicative inverse](#) of an integer a is an integer x such that $a \cdot x$ is congruent to 1 modular some modulus m . To write it in a formal way: we want to find an integer x so that

$$a \cdot x \equiv 1 \pmod{m}.$$

We will also denote x simply with a^{-1} .

We should note that the modular inverse does not always exist. For example, let $m = 4$, $a = 2$. By checking all possible values modulo m , it should become clear that we cannot find a^{-1} satisfying the above equation. It can be proven that the modular inverse exists if and only if a and m are relatively prime (i.e. $\gcd(a, m) = 1$).

In this article, we present two methods for finding the modular inverse in case it exists, and one method for finding the modular inverse for all numbers in linear time.

Finding the Modular Inverse using Extended Euclidean algorithm

Consider the following equation (with unknown x and y):

$$a \cdot x + m \cdot y = 1$$

This is a [Linear Diophantine equation in two variables](#). As shown in the linked article, when $\gcd(a, m) = 1$, the equation has a solution which can be found using the [extended Euclidean algorithm](#). Note that $\gcd(a, m) = 1$ is also the condition for the modular inverse to exist.

Now, if we take modulo m of both sides, we can get rid of $m \cdot y$, and the equation becomes:

$$a \cdot x \equiv 1 \pmod{m}$$

Thus, the modular inverse of a is x .

The implementation is as follows:

```
int x, y;
int g = extended_euclidean(a, m, x, y);
if (g != 1) {
    cout << "No solution!";
}
else {
    x = (x % m + m) % m;
    cout << x << endl;
}
```

Notice that the way we modify x . The resulting x from the extended Euclidean algorithm may be negative, so $x \% m$ might also be negative, and we first have to add m to make it positive.

Finding the Modular Inverse using Binary Exponentiation

Another method for finding modular inverse is to use Euler's theorem, which states that the following congruence is true if a and m are relatively prime:

$$a^{\phi(m)} \equiv 1 \pmod{m}$$

ϕ is [Euler's Totient function](#). Again, note that a and m being relative prime was also the condition for the modular inverse to exist.

If m is a prime number, this simplifies to [Fermat's little theorem](#):

$$a^{m-1} \equiv 1 \pmod{m}$$

Multiply both sides of the above equations by a^{-1} , and we get:

- For an arbitrary (but coprime) modulus m : $a^{\phi(m)-1} \equiv a^{-1} \pmod{m}$
- For a prime modulus m : $a^{m-2} \equiv a^{-1} \pmod{m}$

From these results, we can easily find the modular inverse using the [binary exponentiation algorithm](#), which works in $O(\log m)$ time.

Even though this method is easier to understand than the method described in previous paragraph, in the case when m is not a prime number, we need to calculate Euler phi function, which involves factorization of m , which might be very hard. If the prime factorization of m is known, then the complexity of this method is $O(\log m)$.

Finding the modular inverse for prime moduli using Euclidean Division

Given a prime modulus $m > a$ (or we can apply modulo to make it smaller in 1 step), according to [Euclidean Division](#)

$$m = k \cdot a + r$$

where $k = \lfloor \frac{m}{a} \rfloor$ and $r = m \bmod a$, then

$$\begin{aligned} \implies & 0 \equiv k \cdot a + r \pmod{m} \\ \iff & r \equiv -k \cdot a \pmod{m} \\ \iff & r \cdot a^{-1} \equiv -k \pmod{m} \\ \iff & a^{-1} \equiv -k \cdot r^{-1} \pmod{m} \end{aligned}$$

Note that this reasoning does not hold if m is not prime, since the existence of a^{-1} does not imply the existence of r^{-1} in the general case. To see this, let's try to calculate 5^{-1} modulo 12 with the above formula. We would like to arrive at 5, since $5 \cdot 5 \equiv 1 \pmod{12}$. However, $12 = 2 \cdot 5 + 2$, and we have $k = 2$ and $r = 2$, with 2 being not invertible modulo 12.

If the modulus is prime however, all a with $0 < a < m$ are invertible modulo m , and we can have the following recursive function (in C++) for computing the modular inverse for number a with respect to m

```
int inv(int a) {
    return a <= 1 ? a : m - (long long)(m/a) * inv(m % a) % m;
}
```

The exact time complexity of this recursion is not known. It's somewhere between $O(\frac{\log m}{\log \log m})$ and $O(m^{\frac{1}{3} - \frac{2}{177} + \epsilon})$. See [On the length of Pierce expansions](#). In practice this implementation is fast, e.g. for the modulus $10^9 + 7$ it will always finish in less than 50 iterations.

Applying this formula, we can also precompute the modular inverse for every number in the range $[1, m - 1]$ in $O(m)$.

```
inv[1] = 1;
for(int a = 2; a < m; ++a)
    inv[a] = m - (long long)(m/a) * inv[m%a] % m;
```

Finding the modular inverse for array of numbers modulo m

Suppose we are given an array and we want to find modular inverse for all numbers in it (all of them are invertible). Instead of computing the inverse for every number, we can expand the fraction by the prefix product (excluding itself) and suffix product (excluding itself), and end up only computing a single inverse instead.

$$x_i^{-1} = \frac{1}{x_i} = \frac{\overbrace{x_1 \cdot x_2 \cdots x_{i-1}}^{\text{prefix}_{i-1}} \cdot \overbrace{x_{i+1} \cdot x_{i+2} \cdots x_n}^{\text{suffix}_{i+1}}}{x_1 \cdot x_2 \cdots x_{i-1} \cdot x_i \cdot x_{i+1} \cdot x_{i+2} \cdots x_n} = \text{prefix}_{i-1} \cdot \text{suffix}_{i+1} \cdot (x_1 \cdot x_2 \cdots x_n)^{-1}$$

In the code we can just make a prefix product array (exclude itself, start from the identity element), compute the modular inverse for the product of all numbers and then multiply it by the prefix product and suffix product (exclude itself). The suffix product is computed by iterating from the back to the front.

```
std::vector<int> invs(const std::vector<int> &a, int m) {
    int n = a.size();
    if (n == 0) return {};
    std::vector<int> b(n);
    int v = 1;
    for (int i = 0; i != n; ++i) {
        b[i] = v;
        v = static_cast<long long>(v) * a[i] % m;
    }
    int x, y;
    extended_euclidean(v, m, x, y);
    x = (x % m + m) % m;
    for (int i = n - 1; i >= 0; --i) {
        b[i] = static_cast<long long>(x) * b[i] % m;
        x = static_cast<long long>(x) * a[i] % m;
    }
    return b;
}
```

Practice Problems

- [UVa 11904 - One Unit Machine](#)
- [Hackerrank - Longest Increasing Subsequence Arrays](#)
- [Codeforces 300C - Beautiful Numbers](#)
- [Codeforces 622F - The Sum of the k-th Powers](#)
- [Codeforces 717A - Festival Organization](#)
- [Codeforces 896D - Nephren Runs a Cinema](#)

Contributors:

[Jakob Kogler](#) (39.76%) [thanhtrnguyen](#) (27.11%) [hly1204](#) (13.25%) [Lam Gia Thuan](#) (5.42%) [Oleksandr Kulkov](#) (4.22%) [prpr](#) (3.61%)
[Mariia Mykhailova](#) (3.01%) [Matthias Langer](#) (1.81%) [Angkur Mondal](#) (0.6%) [Ayush Panwar](#) (0.6%) [jxu](#) (0.6%)

Translated From: e-maxx.ru

Linear Congruence Equation

This equation is of the form:

$$a \cdot x \equiv b \pmod{n},$$

where a , b and n are given integers and x is an unknown integer.

It is required to find the value x from the interval $[0, n - 1]$ (clearly, on the entire number line there can be infinitely many solutions that will differ from each other in $n \cdot k$, where k is any integer). If the solution is not unique, then we will consider how to get all the solutions.

Solution by finding the inverse element

Let us first consider a simpler case where a and n are **coprime** ($\gcd(a, n) = 1$). Then one can find the **inverse** of a , and multiplying both sides of the equation with the inverse, and we can get a **unique** solution.

$$x \equiv b \cdot a^{-1} \pmod{n}$$

Now consider the case where a and n are **not coprime** ($\gcd(a, n) \neq 1$). Then the solution will not always exist (for example $2 \cdot x \equiv 1 \pmod{4}$ has no solution).

Let $g = \gcd(a, n)$, i.e. the **greatest common divisor** of a and n (which in this case is greater than one).

Then, if b is not divisible by g , there is no solution. In fact, for any x the left side of the equation $a \cdot x \pmod{n}$ is always divisible by g , while the right-hand side is not divisible by it, hence it follows that there are no solutions.

If g divides b , then by dividing both sides of the equation by g (i.e. dividing a , b and n by g), we receive a new equation:

$$a' \cdot x \equiv b' \pmod{n'}$$

in which a' and n' are already relatively prime, and we have already learned how to handle such an equation. We get x' as solution for x .

It is clear that this x' will also be a solution of the original equation. However it will **not be the only solution**. It can be shown that the original equation has exactly g solutions, and they will look like this:

$$x_i \equiv (x' + i \cdot n') \pmod{n} \quad \text{for } i = 0 \dots g - 1$$

Summarizing, we can say that the **number of solutions** of the linear congruence equation is equal to either $g = \gcd(a, n)$ or to zero.

Solution with the Extended Euclidean Algorithm

We can rewrite the linear congruence to the following Diophantine equation:

$$a \cdot x + n \cdot k = b,$$

where x and k are unknown integers.

The method of solving this equation is described in the corresponding article [Linear Diophantine equations](#) and it consists of applying the [Extended Euclidean Algorithm](#).

It also describes the method of obtaining all solutions of this equation from one found solution, and incidentally this method, when carefully considered, is absolutely equivalent to the method described in the previous section.

Contributors:

[Jaideep Pyne](#) (64.29%) [Oleksandr Kulkov](#) (19.64%) [Jakob Kogler](#) (7.14%) [sundaram123krishnan](#) (7.14%) [Abhijeet Krishnan](#) (1.79%)

Translated

From: e-maxx.ru

Chinese Remainder Theorem

The Chinese Remainder Theorem (which will be referred to as CRT in the rest of this article) was discovered by Chinese mathematician Sun Zi.

Formulation

Let $m = m_1 \cdot m_2 \cdots m_k$, where m_i are pairwise coprime. In addition to m_i , we are also given a system of congruences

$$\begin{cases} a \equiv a_1 \pmod{m_1} \\ a \equiv a_2 \pmod{m_2} \\ \vdots \\ a \equiv a_k \pmod{m_k} \end{cases}$$

where a_i are some given constants. The original form of CRT then states that the given system of congruences always has *one and exactly one* solution modulo m .

E.g. the system of congruences

$$\begin{cases} a \equiv 2 \pmod{3} \\ a \equiv 3 \pmod{5} \\ a \equiv 2 \pmod{7} \end{cases}$$

has the solution 23 modulo 105, because $23 \bmod 3 = 2$, $23 \bmod 5 = 3$, and $23 \bmod 7 = 2$. We can write down every solution as $23 + 105 \cdot k$ for $k \in \mathbb{Z}$.

Corollary

A consequence of the CRT is that the equation

$$x \equiv a \pmod{m}$$

is equivalent to the system of equations

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

(As above, assume that $m = m_1 m_2 \cdots m_k$ and m_i are pairwise coprime).

Solution for Two Moduli

Consider a system of two equations for coprime m_1, m_2 :

$$\begin{cases} a \equiv a_1 \pmod{m_1} \\ a \equiv a_2 \pmod{m_2} \end{cases}$$

We want to find a solution for $a \pmod{m_1 m_2}$. Using the [Extended Euclidean Algorithm](#) we can find Bézout coefficients n_1, n_2 such that

$$n_1 m_1 + n_2 m_2 = 1.$$

In fact n_1 and n_2 are just the [modular inverses](#) of m_1 and m_2 modulo m_2 and m_1 . We have $n_1 m_1 \equiv 1 \pmod{m_2}$ so $n_1 \equiv m_1^{-1} \pmod{m_2}$, and vice versa $n_2 \equiv m_2^{-1} \pmod{m_1}$.

With those two coefficients we can define a solution:

$$a = a_1 n_2 m_2 + a_2 n_1 m_1 \pmod{m_1 m_2}$$

It's easy to verify that this is indeed a solution by computing $a \pmod{m_1}$ and $a \pmod{m_2}$.

$$\begin{aligned} a &\equiv a_1 n_2 m_2 + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1 (1 - n_1 m_1) + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1 - a_1 n_1 m_1 + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1 && \pmod{m_1} \end{aligned}$$

Notice, that the Chinese Remainder Theorem also guarantees, that only 1 solution exists modulo $m_1 m_2$. This is also easy to prove.

Lets assume that you have two different solutions x and y . Because $x \equiv a_i \pmod{m_i}$ and $y \equiv a_i \pmod{m_i}$, it follows that $x - y \equiv 0 \pmod{m_i}$ and therefore $x - y \equiv 0 \pmod{m_1 m_2}$ or equivalently $x \equiv y \pmod{m_1 m_2}$. So x and y are actually the same solution.

Solution for General Case

Inductive Solution

As $m_1 m_2$ is coprime to m_3 , we can inductively repeatedly apply the solution for two moduli for any number of moduli. First you compute $b_2 := a \pmod{m_1 m_2}$ using the first two congruences, then you can compute $b_3 := a \pmod{m_1 m_2 m_3}$ using the congruences $a \equiv b_2 \pmod{m_1 m_2}$ and $a \equiv a_3 \pmod{m_3}$, etc.

Direct Construction

A direct construction similar to Lagrange interpolation is possible.

Let $M_i := \prod_{j \neq i} m_j$, the product of all moduli but m_i , and N_i the modular inverses $N_i := M_i^{-1} \pmod{m_i}$. Then a solution to the system of congruences is:

$$a \equiv \sum_{i=1}^k a_i M_i N_i \pmod{m_1 m_2 \cdots m_k}$$

We can check this is indeed a solution, by computing $a \pmod{m_i}$ for all i . Because M_j is a multiple of m_i for $i \neq j$ we have

$$\begin{aligned} a &\equiv \sum_{j=1}^k a_j M_j N_j && \pmod{m_i} \\ &\equiv a_i M_i N_i && \pmod{m_i} \\ &\equiv a_i M_i M_i^{-1} && \pmod{m_i} \\ &\equiv a_i && \pmod{m_i} \end{aligned}$$

Implementation

```
struct Congruence {
    long long a, m;
```

```

};

long long chinese_remainder_theorem(vector<Congruence> const& congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i, congruence.m);
        solution = (solution + a_i * M_i % M * N_i) % M;
    }
    return solution;
}

```

Solution for not coprime moduli

As mentioned, the algorithm above only works for coprime moduli $m_1, m_2, \dots, m_k$.

In the not coprime case, a system of congruences has exactly one solution modulo $\text{lcm}(m_1, m_2, \dots, m_k)$, or has no solution at all.

E.g. in the following system, the first congruence implies that the solution is odd, and the second congruence implies that the solution is even. It's not possible that a number is both odd and even, therefore there is clearly no solution.

$$\begin{cases} a \equiv 1 \pmod{4} \\ a \equiv 2 \pmod{6} \end{cases}$$

It is pretty simple to determine if a system has a solution. And if it has one, we can use the original algorithm to solve a slightly modified system of congruences.

A single congruence $a \equiv a_i \pmod{m_i}$ is equivalent to the system of congruences $a \equiv a_i \pmod{p_j^{n_j}}$ where $p_1^{n_1} p_2^{n_2} \dots p_k^{n_k}$ is the prime factorization of m_i .

With this fact, we can modify the system of congruences into a system, that only has prime powers as moduli. E.g. the above system of congruences is equivalent to:

$$\begin{cases} a \equiv 1 \pmod{4} \\ a \equiv 2 \pmod{2} \\ a \equiv 2 \pmod{3} \end{cases}$$

Because originally some moduli had common factors, we will get some congruences moduli based on the same prime, however possibly with different prime powers.

You can observe, that the congruence with the highest prime power modulus will be the strongest congruence of all congruences based on the same prime number. Either it will give a contradiction with some other congruence, or it will imply already all other congruences.

In our case, the first congruence $a \equiv 1 \pmod{4}$ implies $a \equiv 1 \pmod{2}$, and therefore contradicts the second congruence $a \equiv 0 \pmod{2}$. Therefore this system of congruences has no solution.

If there are no contradictions, then the system of equation has a solution. We can ignore all congruences except the ones with the highest prime power moduli. These moduli are now coprime, and therefore we can solve this one with the algorithm discussed in the sections above.

E.g. the following system has a solution modulo $\text{lcm}(10, 12) = 60$.

$$\begin{cases} a \equiv 3 \pmod{10} \\ a \equiv 5 \pmod{12} \end{cases}$$

The system of congruence is equivalent to the system of congruences:

$$\begin{cases} a \equiv 3 \equiv 1 \pmod{2} \\ a \equiv 3 \equiv 3 \pmod{5} \\ a \equiv 5 \equiv 1 \pmod{4} \\ a \equiv 5 \equiv 2 \pmod{3} \end{cases}$$

The only congruence with same prime modulo are $a \equiv 1 \pmod{4}$ and $a \equiv 1 \pmod{2}$. The first one already implies the second one, so we can ignore the second one, and solve the following system with coprime moduli instead:

$$\begin{cases} a \equiv 3 \equiv 3 \pmod{5} \\ a \equiv 5 \equiv 1 \pmod{4} \\ a \equiv 5 \equiv 2 \pmod{3} \end{cases}$$

It has the solution $53 \pmod{60}$, and indeed $53 \bmod 10 = 3$ and $53 \bmod 12 = 5$.

Garner's Algorithm

Another consequence of the CRT is that we can represent big numbers using an array of small integers.

Instead of doing a lot of computations with very large numbers, which might be expensive (think of doing divisions with 1000-digit numbers), you can pick a couple of coprime moduli and represent the large number as a system of congruences, and perform all operations on the system of equations. Any number a less than $m_1 m_2 \cdots m_k$ can be represented as an array $a_1, \dots, a_k$, where $a \equiv a_i \pmod{m_i}$.

By using the above algorithm, you can again reconstruct the large number whenever you need it.

Alternatively you can represent the number in the **mixed radix** representation:

$$a = x_1 + x_2 m_1 + x_3 m_1 m_2 + \dots + x_k m_1 \cdots m_{k-1} \text{ with } x_i \in [0, m_i)$$

Garner's algorithm, which is discussed in the dedicated article [Garner's algorithm](#), computes the coefficients x_i . And with those coefficients you can restore the full number.

Practice Problems:

- [Google Code Jam - Golf Gophers](#)
- [Hackerrank - Number of sequences](#)
- [Codeforces - Remainders Game](#)

Contributors:

[Jakob Kogler](#) (74.35%) [jxu](#) (13.04%) [Anton Ålgmyr](#) (5.65%) [Oleksandr Kulkov](#) (3.04%) [thanhtnguyen](#) (1.74%) [Vatsal Sharma](#) (0.87%) [JATIN GAUR](#) (0.43%)
[Max](#) (0.43%) [Mayank Pratap Singh](#) (0.43%)

Garner's algorithm

A consequence of the [Chinese Remainder Theorem](#) is, that we can represent big numbers using an array of small integers. For example, let p be the product of the first 1000 primes. p has around 3000 digits.

Any number a less than p can be represented as an array $a_1, \dots, a_k$, where $a_i \equiv a \pmod{p_i}$. But to do this we obviously need to know how to get back the number a from its representation. One way is discussed in the article about the Chinese Remainder Theorem.

In this article we discuss an alternative, Garner's Algorithm, which can also be used for this purpose.

Mixed Radix Representation

We can represent the number a in the **mixed radix** representation:

$$a = x_1 + x_2 p_1 + x_3 p_1 p_2 + \dots + x_k p_1 \cdots p_{k-1} \text{ with } x_i \in [0, p_i)$$

A mixed radix representation is a positional numeral system, that's a generalization of the typical number systems, like the binary numeral system or the decimal numeral system. For instance the decimal numeral system is a positional numeral system with the radix (or base) 10. Every a number is represented as a string of digits $d_1 d_2 d_3 \dots d_n$ between 0 and 9, and E.g. the string 415 represents the number $4 \cdot 10^2 + 1 \cdot 10^1 + 5 \cdot 10^0$. In general the string of digits $d_1 d_2 d_3 \dots d_n$ represents the number $d_1 b^{n-1} + d_2 b^{n-2} + \dots + d_n b^0$ in the positional numeral system with radix b .

In a mixed radix system, we don't have one radix any more. The base varies from position to position.

Garner's algorithm

Garner's algorithm computes the digits $x_1, \dots, x_k$. Notice, that the digits are relatively small. The digit x_i is an integer between 0 and $p_i - 1$.

Let r_{ij} denote the inverse of p_i modulo p_j

$$r_{ij} = (p_i)^{-1} \pmod{p_j}$$

which can be found using the algorithm described in [Modular Inverse](#).

Substituting a from the mixed radix representation into the first congruence equation we obtain

$$a_1 \equiv x_1 \pmod{p_1}.$$

Substituting into the second equation yields

$$a_2 \equiv x_1 + x_2 p_1 \pmod{p_2},$$

which can be rewritten by subtracting x_1 and dividing by p_1 to get

$$\begin{aligned} a_2 - x_1 &\equiv x_2 p_1 \pmod{p_2} \\ (a_2 - x_1) r_{12} &\equiv x_2 \pmod{p_2} \\ x_2 &\equiv (a_2 - x_1) r_{12} \pmod{p_2} \end{aligned}$$

Similarly we get that

$$x_3 \equiv ((a_3 - x_1) r_{13} - x_2) r_{23} \pmod{p_3}.$$

Now, we can clearly see an emerging pattern, which can be expressed by the following code:

```
for (int i = 0; i < k; ++i) {
    x[i] = a[i];
    for (int j = 0; j < i; ++j) {
        x[i] = r[j][i] * (x[i] - x[j]);

        x[i] = x[i] % p[i];
        if (x[i] < 0)
            x[i] += p[i];
    }
}
```

So we learned how to calculate digits x_i in $O(k^2)$ time. The number a can now be calculated using the previously mentioned formula

$$a = x_1 + x_2 \cdot p_1 + x_3 \cdot p_1 \cdot p_2 + \dots + x_k \cdot p_1 \cdots p_{k-1}$$

It is worth noting that in practice, we almost probably need to compute the answer a using [Arbitrary-Precision Arithmetic](#), but the digits x_i (because they are small) can usually be calculated using built-in types, and therefore Garner's algorithm is very efficient.

Implementation of Garner's Algorithm

It is convenient to implement this algorithm using Java, because it has built-in support for large numbers through the `BigInteger` class.

Here we show an implementation that can store big numbers in the form of a set of congruence equations. It supports addition, subtraction and multiplication. And with Garner's algorithm we can convert the set of equations into the unique integer. In this code, we take 100 prime numbers greater than 10^9 , which allows representing numbers as large as 10^{900} .

```
final int SZ = 100;
int pr[] = new int[SZ];
int r[][] = new int[SZ][SZ];

void init() {
    for (int x = 1000 * 1000 * 1000, i = 0; i < SZ; ++x)
        if (BigInteger.valueOf(x).isProbablePrime(100))
            pr[i++] = x;

    for (int i = 0; i < SZ; ++i)
        for (int j = i + 1; j < SZ; ++j)
            r[i][j] =
                BigInteger.valueOf(pr[i]).modInverse(BigInteger.valueOf(pr[j])).intValue();
}

class Number {
    int a[] = new int[SZ];

    public Number() {}

    public Number(int n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n % pr[i];
    }

    public Number(BigInteger n) {
        for (int i = 0; i < SZ; ++i)
            a[i] = n.mod(BigInteger.valueOf(pr[i])).intValue();
    }

    public Number add(Number n) {
        Number result = new Number();
        for (int i = 0; i < SZ; ++i)
            result.a[i] = (a[i] + n.a[i]) % pr[i];
        return result;
    }

    public Number subtract(Number n) {
        Number result = new Number();
        for (int i = 0; i < SZ; ++i)
            result.a[i] = (a[i] - n.a[i] + pr[i]) % pr[i];
        return result;
    }
}
```

```

public Number multiply(Number n) {
    Number result = new Number();
    for (int i = 0; i < SZ; ++i)
        result.a[i] = (int)((a[i] * 11 * n.a[i]) % pr[i]);
    return result;
}

public BigInteger bigIntegerValue(boolean can_be_negative) {
    BigInteger result = BigInteger.ZERO, mult = BigInteger.ONE;
    int x[] = new int[SZ];
    for (int i = 0; i < SZ; ++i) {
        x[i] = a[i];
        for (int j = 0; j < i; ++j) {
            long cur = (x[i] - x[j]) * 11 * r[j][i];
            x[i] = (int)((cur % pr[i] + pr[i]) % pr[i]);
        }
        result = result.add(mult.multiply(BigInteger.valueOf(x[i])));
        mult = mult.multiply(BigInteger.valueOf(pr[i]));
    }

    if (can_be_negative)
        if (result.compareTo(mult.shiftRight(1)) >= 0)
            result = result.subtract(mult);

    return result;
}
}

```

Contributors:

[Jakob Kogler](#) (97.52%)
 [GILGAMESH](#) (1.86%)
 [Le Bao Hiep](#) (0.62%)

Translated From: e-maxx.ru

Factorial modulo p

In some cases it is necessary to consider complex formulas modulo some prime p , containing factorials in both numerator and denominator, like such that you encounter in the formula for Binomial coefficients. We consider the case when p is relatively small. This problem makes only sense when the factorials appear in both numerator and denominator of fractions. Otherwise $p!$ and subsequent terms will reduce to zero. But in fractions the factors of p can cancel, and the resulting expression will be non-zero modulo p .

Thus, formally the task is: You want to calculate $n! \bmod p$, without taking all the multiple factors of p into account that appear in the factorial. Imagine you write down the prime factorization of $n!$, remove all factors p , and compute the product modulo p . We will denote this *modified* factorial with $n!_{\%p}$. For instance $7!_{\%3} \equiv 1 \cdot 2 \cdot \frac{1}{3} \cdot 4 \cdot 5 \cdot \frac{2}{6} \cdot 7 \equiv 2 \pmod{3}$.

Learning how to effectively calculate this modified factorial allows us to quickly calculate the value of the various combinatorial formulas (for example, [Binomial coefficients](#)).

Algorithm

Let's write this modified factorial explicitly.

$$\begin{aligned} n!_{\%p} &= 1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-2) \cdot (p-1) \cdot \frac{1}{p} \cdot (p+1) \cdot (p+2) \cdot \dots \cdot (2p-1) \cdot \frac{2}{2p} \cdot \\ &\quad \cdot (2p+1) \cdot \dots \cdot (p^2-1) \cdot \frac{1}{p^2} \cdot (p^2+1) \cdot \dots \cdot n \pmod{p} \\ &= 1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-2) \cdot (p-1) \cdot \frac{1}{p} \cdot 1 \cdot 2 \cdot \dots \cdot (p-1) \cdot \frac{2}{2p} \cdot 1 \cdot 2 \cdot \dots \cdot (p-1) \cdot \frac{1}{p^2} \cdot 1 \cdot 2 \cdot \dots \cdot (n \bmod p) \pmod{p} \end{aligned}$$

It can be clearly seen that factorial is divided into several blocks of same length except for the last one.

$$\begin{aligned} n!_{\%p} &= \underbrace{1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-2) \cdot (p-1) \cdot 1}_{1st} \cdot \underbrace{1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-2) \cdot (p-1) \cdot 2}_{2nd} \cdot \dots \\ &\quad \cdot \underbrace{1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-2) \cdot (p-1) \cdot 1}_{pth} \cdot \dots \cdot \underbrace{1 \cdot 2 \cdot \dots \cdot (n \bmod p)}_{tail} \pmod{p}. \end{aligned}$$

The main part of the blocks it is easy to count – it's just $(p-1)! \bmod p$. We can compute that programmatically or just apply Wilson theorem which states that $(p-1)! \bmod p = -1$ for any prime p .

We have exactly $\lfloor \frac{n}{p} \rfloor$ such blocks, therefore we need to raise -1 to the power of $\lfloor \frac{n}{p} \rfloor$. This can be done in logarithmic time using [Binary Exponentiation](#); however you can also notice that the result will switch between -1 and 1 , so we only need to look at the parity of the exponent and multiply by -1 if the parity is odd. And instead of a multiplication, we can also just subtract the current result from p .

The value of the last partial block can be calculated separately in $O(p)$.

This leaves only the last element of each block. If we hide the already handled elements, we can see the following pattern:

$$n!_{\%p} = \frac{1}{1} \cdot \frac{2}{2} \cdot \dots \cdot \frac{(p-1)}{(p-1)} \cdot \frac{1}{1} \cdot \frac{1}{2} \cdot \frac{2}{2} \cdot \dots$$

This again is a *modified* factorial, only with a much smaller dimension. It's $\lfloor n/p \rfloor! \pmod p$.

Thus, during the calculation of the *modified* factorial $n! \pmod p$ we did $O(p)$ operations and are left with the calculation of $\lfloor n/p \rfloor! \pmod p$. We have a recursive formula. The recursion depth is $O(\log_p n)$, and therefore the complete asymptotic behavior of the algorithm is $O(p \log_p n)$.

Notice, if you precompute the factorials $0!, 1!, 2!, \dots, (p-1)!$ modulo p , then the complexity will just be $O(\log_p n)$.

Implementation

We don't need recursion because this is a case of tail recursion and thus can be easily implemented using iteration. In the following implementation we precompute the factorials $0!, 1!, \dots, (p-1)!$, and thus have the runtime $O(p + \log_p n)$. If you need to call the function multiple times, then you can do the precomputation outside of the function and do the computation of $n! \pmod p$ in $O(\log_p n)$ time.

```
int factmod(int n, int p) {
    vector<int> f(p);
    f[0] = 1;
    for (int i = 1; i < p; i++)
        f[i] = f[i-1] * i % p;

    int res = 1;
    while (n > 1) {
        if ((n/p) % 2)
            res = p - res;
        res = res * f[n%p] % p;
        n /= p;
    }
    return res;
}
```

Alternative, if you only have limit memory and can't afford storing all factorials, you can also just remember the factorials that you need, sort them, and then compute them in one sweep by computing the factorials $0!, 1!, 2!, \dots, (p-1)!$ in a loop without storing them explicitly.

Multiplicity of p

If we want to compute a Binomial coefficient modulo p , then we additionally need the multiplicity of the p in n , i.e. the number of times p occurs in the prime factorization of n , or number of times we erased p during the computation of the *modified* factorial.

[Legendre's formula](#) gives us a way to compute this in $O(\log_p n)$ time. The formula gives the multiplicity v_p as:

$$v_p(n!) = \sum_{i=1}^{\infty} \left\lfloor \frac{n}{p^i} \right\rfloor$$

Thus we get the implementation:

```
int multiplicity_factorial(int n, int p) {
    int count = 0;
    do {
        n /= p;
        count += n;
    } while (n);
    return count;
}
```

This formula can be proven very easily using the same ideas that we did in the previous sections. Remove all elements that don't contain the factor p . This leaves $\lfloor n/p \rfloor$ element remaining. If we remove the factor p from each of them, we get the product $1 \cdot 2 \cdots \lfloor n/p \rfloor = \lfloor n/p \rfloor!$, and again we have a recursion.

Contributors:

[Jakob Kogler](#) (65.52%) [Madhur Chauhan](#) (26.72%) [Oleksandr Kulkov](#) (6.03%) [Amit Kumar](#) (0.86%) [Pranshu Srivastava](#) (0.86%)

Translated

From: e-maxx.ru

Discrete Logarithm

The discrete logarithm is an integer x satisfying the equation

$$a^x \equiv b \pmod{m}$$

for given integers a , b and m .

The discrete logarithm does not always exist, for instance there is no solution to $2^x \equiv 3 \pmod{7}$. There is no simple condition to determine if the discrete logarithm exists.

In this article, we describe the **Baby-step giant-step** algorithm, an algorithm to compute the discrete logarithm proposed by Shanks in 1971, which has the time complexity $O(\sqrt{m})$. This is a **meet-in-the-middle** algorithm because it uses the technique of separating tasks in half.

Algorithm

Consider the equation:

$$a^x \equiv b \pmod{m},$$

where a and m are relatively prime.

Let $x = np - q$, where n is some pre-selected constant (we will describe how to select n later). p is known as **giant step**, since increasing it by one increases x by n . Similarly, q is known as **baby step**.

Obviously, any number x in the interval $[0; m)$ can be represented in this form, where $p \in [1; \lceil \frac{m}{n} \rceil]$ and $q \in [0; n]$.

Then, the equation becomes:

$$a^{np-q} \equiv b \pmod{m}.$$

Using the fact that a and m are relatively prime, we obtain:

$$a^{np} \equiv ba^q \pmod{m}$$

This new equation can be rewritten in a simplified form:

$$f_1(p) = f_2(q).$$

This problem can be solved using the meet-in-the-middle method as follows:

- Calculate f_1 for all possible arguments p . Sort the array of value-argument pairs.
- For all possible arguments q , calculate f_2 and look for the corresponding p in the sorted array using binary search.

Complexity

We can calculate $f_1(p)$ in $O(\log m)$ using the [binary exponentiation algorithm](#). Similarly for $f_2(q)$.

In the first step of the algorithm, we need to calculate f_1 for every possible argument p and then sort the values. Thus, this step has complexity:

$$O\left(\left\lceil \frac{m}{n} \right\rceil (\log m + \log \left\lceil \frac{m}{n} \right\rceil)\right) = O\left(\left\lceil \frac{m}{n} \right\rceil \log m\right)$$

In the second step of the algorithm, we need to calculate $f_2(q)$ for every possible argument q and then do a binary search on the array of values of f_1 , thus this step has complexity:

$$O\left(n \left(\log m + \log \frac{m}{n}\right)\right) = O(n \log m).$$

Now, when we add these two complexities, we get $\log m$ multiplied by the sum of n and m/n , which is minimal when $n = \sqrt{m}$, which means, to achieve optimal performance, n should be chosen such that:

$$n = \sqrt{m}.$$

Then, the complexity of the algorithm becomes:

$$O(\sqrt{m} \log m).$$

Implementation

The simplest implementation

In the following code, the function `powmod` calculates $a^b \pmod{m}$ and the function `solve` produces a proper solution to the problem. It returns -1 if there is no solution and returns one of the possible solutions otherwise.

```
int powmod(int a, int b, int m) {
    int res = 1;
    while (b > 0) {
        if (b & 1) {
            res = (res * 1ll * a) % m;
        }
        a = (a * 1ll * a) % m;
        b >>= 1;
    }
    return res;
}

int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;
    map<int, int> vals;
    for (int p = 1; p <= n; ++p)
        vals[powmod(a, p * n, m)] = p;
    for (int q = 0; q <= n; ++q) {
        int cur = (powmod(a, q, m) * 1ll * b) % m;
        if (vals.count(cur)) {
            int ans = vals[cur] * n - q;
            return ans;
        }
    }
    return -1;
}
```

In this code, we used `map` from the C++ standard library to store the values of f_1 . Internally, `map` uses a red-black tree to store values. Thus this code is a little bit slower than if we had used an array and binary searched, but is much easier to write.

Notice that our code assumes $0^0 = 1$, i.e. the code will compute 0 as solution for the equation $0^x \equiv 1 \pmod{m}$ and also as solution for $0^x \equiv 0 \pmod{1}$. This is an often used convention in algebra, but it's also not universally accepted in all areas. Sometimes 0^0 is simply undefined. If you don't like our convention, then you need to handle the case $a = 0$ separately:

```
if (a == 0)
    return b == 0 ? 1 : -1;
```

Another thing to note is that, if there are multiple arguments p that map to the same value of f_1 , we only store one such argument. This works in this case because we only want to return one possible solution. If we need to return all possible solutions, we need to change `map<int, int>` to, say, `map<int, vector<int>>`. We also need to change the second step accordingly.

Improved implementation

A possible improvement is to get rid of binary exponentiation. This can be done by keeping a variable that is multiplied by a each time we increase q and a variable that is multiplied by a^n each time we increase p . With this change, the complexity of the algorithm is still the same, but now the $\log$ factor is only for the `map`. Instead of a `map`, we can also use a hash table (`unordered_map` in C++) which has the average time complexity $O(1)$ for inserting and searching.

Problems often ask for the minimum x which satisfies the solution.

It is possible to get all answers and take the minimum, or reduce the first found answer using [Euler's theorem](#), but we can be smart about the order in which we calculate values and ensure the first answer we find is the minimum.

```
// Returns minimum x for which a ^ x % m = b % m, a and m are coprime.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int n = sqrt(m) + 1;

    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 111 * a) % m;

    unordered_map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 111 * a) % m;
    }

    for (int p = 1, cur = 1; p <= n; ++p) {
        cur = (cur * 111 * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur];
            return ans;
        }
    }
    return -1;
}
```

The complexity is $O(\sqrt{m})$ using `unordered_map`.

When a and m are not coprime

Let $g = \gcd(a, m)$, and $g > 1$. Clearly $a^x \bmod m$ for every $x \geq 1$ will be divisible by g .

If $g \nmid b$, there is no solution for x .

If $g \mid b$, let $a = g\alpha$, $b = g\beta$, $m = gv$.

$$\begin{aligned} a^x &\equiv b \pmod{m} \\ (g\alpha)a^{x-1} &\equiv g\beta \pmod{gv} \\ \alpha a^{x-1} &\equiv \beta \pmod{v} \end{aligned}$$

The baby-step giant-step algorithm can be easily extended to solve $ka^x \equiv b \pmod{m}$ for x .

```
// Returns minimum x for which a ^ x % m = b % m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
    }
```

```

    k = (k * 111 * a / g) % m;
}

int n = sqrt(m) + 1;
int an = 1;
for (int i = 0; i < n; ++i)
    an = (an * 111 * a) % m;

unordered_map<int, int> vals;
for (int q = 0, cur = b; q <= n; ++q) {
    vals[cur] = q;
    cur = (cur * 111 * a) % m;
}

for (int p = 1, cur = k; p <= n; ++p) {
    cur = (cur * 111 * an) % m;
    if (vals.count(cur)) {
        int ans = n * p - vals[cur] + add;
        return ans;
    }
}
return -1;
}

```

The time complexity remains $O(\sqrt{m})$ as before since the initial reduction to coprime a and m is done in $O(\log^2 m)$.

Practice Problems

- [Spoj - Power Modulo Inverted](#)
- [Topcoder - SplittingFoxes3](#)
- [CodeChef - Inverse of a Function](#)
- [Hard Equation](#) (assume that 0^0 is undefined)
- [CodeChef - Chef and Modular Sequence](#)

References

- [Wikipedia - Baby-step giant-step](#)
- [Answer by Zander on Mathematics StackExchange](#)

Contributors:

[thanhtnguyen](#) (35.24%)
 [Soumik Sarkar](#) (34.36%)
 [wiktorkuchta](#) (15.86%)
 [Akash Bhalotia](#) (4.41%)
 [Oleksandr Kulkov](#) (3.08%)
 [Jakob Kogler](#) (2.2%)
[Tarun Khullar](#) (2.2%)
 [Bhuvnesh Jain](#) (1.32%)
 [Le Bao Hiep](#) (0.88%)
 [Mariia Mykhailova](#) (0.44%)

Translated From: e-maxx.ru

Primitive Root

Definition

In modular arithmetic, a number g is called a **primitive root modulo n** if every number coprime to n is congruent to a power of g modulo n . Mathematically, g is a **primitive root modulo n** if and only if for any integer a such that $\gcd(a, n) = 1$, there exists an integer k such that:

$$g^k \equiv a \pmod{n}.$$

k is then called the **index** or **discrete logarithm** of a to the base g modulo n . g is also called the **generator** of the multiplicative group of integers modulo n .

In particular, for the case where n is a prime, the powers of primitive root runs through all numbers from 1 to $n - 1$.

Existence

Primitive root modulo n exists if and only if:

- n is $1, 2, 4$, or
- n is power of an odd prime number ($n = p^k$), or
- n is twice power of an odd prime number ($n = 2 \cdot p^k$).

This theorem was proved by Gauss in 1801.

Relation with the Euler function

Let g be a primitive root modulo n . Then we can show that the smallest number k for which $g^k \equiv 1 \pmod{n}$ is equal $\phi(n)$. Moreover, the reverse is also true, and this fact will be used in this article to find a primitive root.

Furthermore, the number of primitive roots modulo n , if there are any, is equal to $\phi(\phi(n))$.

Algorithm for finding a primitive root

A naive algorithm is to consider all numbers in range $[1, n - 1]$. And then check if each one is a primitive root, by calculating all its power to see if they are all different. This algorithm has complexity $O(g \cdot n)$, which would be too slow. In this section, we propose a faster algorithm using several well-known theorems.

From previous section, we know that if the smallest number k for which $g^k \equiv 1 \pmod{n}$ is $\phi(n)$, then g is a primitive root. Since for any number a relative prime to n , we know from Euler's theorem that $a^{\phi(n)} \equiv 1 \pmod{n}$, then to check if g is primitive root, it is enough to check that for all d less than $\phi(n)$, $g^d \not\equiv 1 \pmod{n}$. However, this algorithm is still too slow.

From Lagrange's theorem, we know that the index of 1 of any number modulo n must be a divisor of $\phi(n)$. Thus, it is sufficient to verify for all proper divisor $d \mid \phi(n)$ that $g^d \not\equiv 1 \pmod{n}$. This is already a much faster algorithm, but we can still do better.

Factorize $\phi(n) = p_1^{a_1} \cdots p_s^{a_s}$. We prove that in the previous algorithm, it is sufficient to consider only the values of d which have the form $\frac{\phi(n)}{p_j}$. Indeed, let d be any proper divisor of $\phi(n)$. Then, obviously, there exists such j that $d \mid \frac{\phi(n)}{p_j}$, i.e. $d \cdot k = \frac{\phi(n)}{p_j}$. However, if $g^d \equiv 1 \pmod{n}$, we would get:

$$g^{\frac{\phi(n)}{p_j}} \equiv g^{d \cdot k} \equiv (g^d)^k \equiv 1^k \equiv 1 \pmod{n}.$$

i.e. among the numbers of the form $\frac{\phi(n)}{p_i}$, there would be at least one such that the conditions were not met.

Now we have a complete algorithm for finding the primitive root:

- First, find $\phi(n)$ and factorize it.
- Then iterate through all numbers $g \in [1, n]$, and for each number, to check if it is primitive root, we do the following:
 - Calculate all $g^{\frac{\phi(n)}{p_i}} \pmod n$.
 - If all the calculated values are different from 1, then g is a primitive root.

Running time of this algorithm is $O(n \cdot \log \phi(n) \cdot \log n)$ (assume that $\phi(n)$ has $\log \phi(n)$ divisors).

Shoup (1990, 1992) proved, assuming the [generalized Riemann hypothesis](#), that g is $O(\log^6 p)$.

Implementation

The following code assumes that the modulo p is a prime number. To make it works for any value of p , we must add calculation of $\phi(p)$.

```
int powmod (int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int (res * 1ll * a % p), --b;
        else
            a = int (a * 1ll * a % p), b >>= 1;
    return res;
}

int generator (int p) {
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=p; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= powmod (res, phi / fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}
```

Contributors:

[thanhtrnguyen](#) (83.33%) [Tanner Hoke](#) (9.38%) [Oleksandr Kulkov](#) (6.25%) [Jakob Kogler](#) (1.04%)

Translated

From: e-maxx.ru

Discrete Root

The problem of finding a discrete root is defined as follows. Given a prime n and two integers a and k , find all x for which:

$$x^k \equiv a \pmod{n}$$

The algorithm

We will solve this problem by reducing it to the [discrete logarithm problem](#).

Let's apply the concept of a [primitive root](#) modulo n . Let g be a primitive root modulo n . Note that since n is prime, it must exist, and it can be found in $O(\text{Ans} \cdot \log \phi(n) \cdot \log n) = O(\text{Ans} \cdot \log^2 n)$ plus time of factoring $\phi(n)$.

We can easily discard the case where $a = 0$. In this case, obviously there is only one answer: $x = 0$.

Since we know that n is a prime and any number between 1 and $n - 1$ can be represented as a power of the primitive root, we can represent the discrete root problem as follows:

$$(g^y)^k \equiv a \pmod{n}$$

where

$$x \equiv g^y \pmod{n}$$

This, in turn, can be rewritten as

$$(g^k)^y \equiv a \pmod{n}$$

Now we have one unknown y , which is a discrete logarithm problem. The solution can be found using Shanks' baby-step giant-step algorithm in $O(\sqrt{n} \log n)$ (or we can verify that there are no solutions).

Having found one solution y_0 , one of solutions of discrete root problem will be $x_0 = g^{y_0} \pmod{n}$.

Finding all solutions from one known solution

To solve the given problem in full, we need to find all solutions knowing one of them: $x_0 = g^{y_0} \pmod{n}$.

Let's recall the fact that a primitive root always has order of $\phi(n)$, i.e. the smallest power of g which gives 1 is $\phi(n)$. Therefore, if we add the term $\phi(n)$ to the exponential, we still get the same value:

$$x^k \equiv g^{y_0 \cdot k + l \cdot \phi(n)} \equiv a \pmod{n} \quad \forall l \in \mathbb{Z}$$

Hence, all the solutions are of the form:

$$x = g^{y_0 + l \cdot \frac{\phi(n)}{k}} \pmod{n} \quad \forall l \in \mathbb{Z}.$$

where l is chosen such that the fraction must be an integer. For this to be true, the numerator has to be divisible by the least common multiple of $\phi(n)$ and k . Remember that least common multiple of two numbers $\text{lcm}(a, b) = \frac{a \cdot b}{\text{gcd}(a, b)}$; we'll get

$$x = g^{y_0 + l \cdot \frac{\phi(n)}{\text{gcd}(k, \phi(n))}} \pmod{n} \quad \forall l \in \mathbb{Z}.$$

This is the final formula for all solutions of the discrete root problem.

Implementation

Here is a full implementation, including procedures for finding the primitive root, discrete log and finding and printing all solutions.

```

int gcd(int a, int b) {
    return a ? gcd(b % a, a) : b;
}

int powmod(int a, int b, int p) {
    int res = 1;
    while (b > 0) {
        if (b & 1) {
            res = res * a % p;
        }
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

// Finds the primitive root modulo p
int generator(int p) {
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    }
    if (n > 1)
        fact.push_back(n);

    for (int res = 2; res <= p; ++res) {
        bool ok = true;
        for (int factor : fact) {
            if (powmod(res, phi / factor, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return res;
    }
    return -1;
}

// This program finds all numbers x such that x^k = a (mod n)
int main() {
    int n, k, a;
    scanf("%d %d %d", &n, &k, &a);
    if (a == 0) {
        puts("1\n0");
        return 0;
    }

    int g = generator(n);

    // Baby-step giant-step discrete logarithm algorithm
    int sq = (int) sqrt(n + .0) + 1;
    vector<pair<int, int>> dec(sq);
    for (int i = 1; i <= sq; ++i)
        dec[i-1] = {powmod(g, i * sq * k % (n - 1), n), i};
    sort(dec.begin(), dec.end());
    int any_ans = -1;
    for (int i = 0; i < sq; ++i) {
        int my = powmod(g, i * k % (n - 1), n) * a % n;
        auto it = lower_bound(dec.begin(), dec.end(), make_pair(my, 0));
        if (it != dec.end() && it->first == my) {
            any_ans = it->second * sq - i;
            break;
        }
    }
    if (any_ans == -1) {
        puts("0");
        return 0;
    }

    // Print all possible answers
    int delta = (n-1) / gcd(k, n-1);
    vector<int> ans;
    for (int cur = any_ans % delta; cur < n-1; cur += delta)
        ans.push_back(powmod(g, cur, n));
}

```

```
sort(ans.begin(), ans.end());  
printf("%d\n", ans.size());  
for (int answer : ans)  
    printf("%d ", answer);  
}
```

Practice problems

- [Codeforces - Lunar New Year and a Recursive Sequence](#)

Contributors:

[thanhtnguyen](#) (60.14%) [wiktorkuchta](#) (21.62%) [Mariia Mykhailova](#) (12.16%) [Oleksandr Kulkov](#) (4.05%) [Jakob Kogler](#) (1.35%) [WalidGhallab](#) (0.68%)

Original

Montgomery Multiplication

Many algorithms in number theory, like [prime testing](#) or [integer factorization](#), and in cryptography, like RSA, require lots of operations modulo a large number. A multiplications like $xy \bmod n$ is quite slow to compute with the typical algorithms, since it requires a division to know how many times n has to be subtracted from the product. And division is a really expensive operation, especially with big numbers.

The **Montgomery (modular) multiplication** is a method that allows computing such multiplications faster. Instead of dividing the product and subtracting n multiple times, it adds multiples of n to cancel out the lower bits and then just discards the lower bits.

Montgomery representation

However the Montgomery multiplication doesn't come for free. The algorithm works only in the **Montgomery space**. And we need to transform our numbers into that space, before we can start multiplying.

For the space we need a positive integer $r \geq n$ coprime to n , i.e. $\gcd(n, r) = 1$. In practice we always choose r to be 2^m for a positive integer m , since multiplications, divisions and modulo r operations can then be efficiently implemented using shifts and other bit operations. n will be an odd number in pretty much all applications, since it is not hard to factorize an even number. So every power of 2 will be coprime to n .

The representative $\bar{x}$ of a number x in the Montgomery space is defined as:

$$\bar{x} := x \cdot r \bmod n$$

Notice, the transformation is actually such a multiplication that we want to optimize. So this is still an expensive operation. However you only need to transform a number once into the space. As soon as you are in the Montgomery space, you can perform as many operations as you want efficiently. And at the end you transform the final result back. So as long as you are doing lots of operations modulo n , this will be no problem.

Inside the Montgomery space you can still perform most operations as usual. You can add two elements ($x \cdot r + y \cdot r \equiv (x + y) \cdot r \bmod n$), subtract, check for equality, and even compute the greatest common divisor of a number with n (since $\gcd(n, r) = 1$). All with the usual algorithms.

However this is not the case for multiplication.

We expect the result to be:

$$\bar{x} * \bar{y} = \overline{x \cdot y} = (x \cdot y) \cdot r \bmod n.$$

But the normal multiplication will give us:

$$\bar{x} \cdot \bar{y} = (x \cdot y) \cdot r \cdot r \bmod n.$$

Therefore the multiplication in the Montgomery space is defined as:

$$\bar{x} * \bar{y} := \bar{x} \cdot \bar{y} \cdot r^{-1} \bmod n.$$

Montgomery reduction

The multiplication of two numbers in the Montgomery space requires an efficient computation of $x \cdot r^{-1} \bmod n$. This operation is called the **Montgomery reduction**, and is also known as the algorithm **REDC**.

Because $\gcd(n, r) = 1$, we know that there are two numbers r^{-1} and n' with $0 < r^{-1}, n' < n$ with

$$r \cdot r^{-1} + n \cdot n' = 1.$$

Both r^{-1} and n' can be computed using the [Extended Euclidean algorithm](#).

Using this identity we can write $x \cdot r^{-1}$ as:

$$\begin{aligned} x \cdot r^{-1} &= x \cdot r \cdot r^{-1} / r = x \cdot (-n \cdot n' + 1) / r \\ &= (-x \cdot n \cdot n' + x) / r \equiv (-x \cdot n \cdot n' + 1 \cdot r \cdot n + x) / r \pmod n \\ &\equiv ((-x \cdot n' + 1 \cdot r) \cdot n + x) / r \pmod n \end{aligned}$$

The equivalences hold for any arbitrary integer l . This means, that we can add or subtract an arbitrary multiple of r to $x \cdot n'$, or in other words, we can compute $q := x \cdot n' \pmod r$.

This gives us the following algorithm to compute $x \cdot r^{-1} \pmod n$:

```
function reduce(x):
    q = (x mod r) * n' mod r
    a = (x - q * n) / r
    if a < 0:
        a += n
    return a
```

Since $x < n \cdot n < r \cdot n$ (even if x is the product of a multiplication) and $q \cdot n < r \cdot n$ we know that $-n < (x - q \cdot n) / r < n$. Therefore the final modulo operation is implemented using a single check and one addition.

As we see, we can perform the Montgomery reduction without any heavy modulo operations. If we choose r as a power of 2, the modulo operations and divisions in the algorithm can be computed using bitmasking and shifting.

A second application of the Montgomery reduction is to transfer a number back from the Montgomery space into the normal space.

Fast inverse trick

For computing the inverse $n' := n^{-1} \pmod r$ efficiently, we can use the following trick (which is inspired from the Newton's method):

$$a \cdot x \equiv 1 \pmod{2^k} \implies a \cdot x \cdot (2 - a \cdot x) \equiv 1 \pmod{2^{2k}}$$

This can easily be proven. If we have $a \cdot x = 1 + m \cdot 2^k$, then we have:

$$\begin{aligned} a \cdot x \cdot (2 - a \cdot x) &= 2 \cdot a \cdot x - (a \cdot x)^2 \\ &= 2 \cdot (1 + m \cdot 2^k) - (1 + m \cdot 2^k)^2 \\ &= 2 + 2 \cdot m \cdot 2^k - 1 - 2 \cdot m \cdot 2^k - m^2 \cdot 2^{2k} \\ &= 1 - m^2 \cdot 2^{2k} \\ &\equiv 1 \pmod{2^{2k}}. \end{aligned}$$

This means we can start with $x = 1$ as the inverse of a modulo 2^1 , apply the trick a few times and in each iteration we double the number of correct bits of x .

Implementation

Using the GCC compiler we can compute $x \cdot y \pmod n$ still efficiently, when all three numbers are 64 bit integer, since the compiler supports 128 bit integer with the types `__int128` and `__uint128`.

```
long long result = (__int128)x * y % n;
```

However there is no type for 256 bit integer. Therefore we will here show an implementation for a 128 bit multiplication.

```
using u64 = uint64_t;
using u128 = __uint128_t;
using i128 = __int128_t;

struct u256 {
    u128 high, low;

    static u256 mult(u128 x, u128 y) {
        u64 a = x >> 64, b = x;
        u64 c = y >> 64, d = y;
        // (a*2^64 + b) * (c*2^64 + d) =
        // (a*c) * 2^128 + (a*d + b*c)*2^64 + (b*d)
        u128 ac = (u128)a * c;
        u128 ad = (u128)a * d;
        u128 bc = (u128)b * c;
        u128 bd = (u128)b * d;
        u128 carry = (u128)(u64)ad + (u128)(u64)bc + (bd >> 64u);
        u128 high = ac + (ad >> 64u) + (bc >> 64u) + (carry >> 64u);
        u128 low = (ad << 64u) + (bc << 64u) + bd;
        return {high, low};
    }
};

struct Montgomery {
    Montgomery(u128 n) : mod(n), inv(1) {
        for (int i = 0; i < 7; i++)
            inv *= 2 - n * inv;
    }

    u128 init(u128 x) {
        x %= mod;
        for (int i = 0; i < 128; i++) {
            x <<= 1;
            if (x >= mod)
                x -= mod;
        }
        return x;
    }

    u128 reduce(u256 x) {
        u128 q = x.low * inv;
        i128 a = x.high - u256::mult(q, mod).high;
        if (a < 0)
            a += mod;
        return a;
    }

    u128 mult(u128 a, u128 b) {
        return reduce(u256::mult(a, b));
    }

    u128 mod, inv;
};
```

Fast transformation

The current method of transforming a number into Montgomery space is pretty slow. There are faster ways.

You can notice the following relation:

$$\bar{x} := x \cdot r \bmod n = x \cdot r^2 / r = x * r^2$$

Transforming a number into the space is just a multiplication inside the space of the number with r^2 . Therefore we can precompute $r^2 \bmod n$ and just perform a multiplication instead of shifting the number 128 times.

In the following code we initialize `r2` with `-n % n`, which is equivalent to $r - n \equiv r \pmod n$, shift it 4 times to get $r \cdot 2^4 \pmod n$. This number can be interpreted as 2^4 in Montgomery space. If we square it 5 times, we get $(2^4)^{2^5} = (2^4)^{32} = 2^{128} = r$ in Montgomery space, which is exactly $r^2 \pmod n$.

```
struct Montgomery {
    Montgomery(u128 n) : mod(n), inv(1), r2(-n % n) {
        for (int i = 0; i < 7; i++)
            inv *= 2 - n * inv;

        for (int i = 0; i < 4; i++) {
            r2 <<= 1;
            if (r2 >= mod)
                r2 -= mod;
        }
        for (int i = 0; i < 5; i++)
            r2 = mul(r2, r2);
    }

    u128 init(u128 x) {
        return mult(x, r2);
    }

    u128 mod, inv, r2;
};
```

Contributors:

[Jakob Kogler](#) (96.81%) [Oleksandr Kulkov](#) (2.28%) [Prakash Jha](#) (0.46%) [kyomukyomupurin](#) (0.46%)

Translated

From: e-maxx.ru

Balanced Ternary



This is a non-standard but still positional **numeral system**. Its feature is that digits can have one of the values -1 , 0 and 1 . Nevertheless, its base is still 3 (because there are three possible values). Since it is not convenient to write -1 as a digit, we'll use letter Z further for this purpose. If you think it is quite a strange system - look at the picture - here is one of the computers utilizing it.

So here are few first numbers written in balanced ternary:

0	0
1	1
2	1Z
3	10
4	11
5	1ZZ
6	1Z0
7	1Z1
8	10Z
9	100

This system allows you to write negative values without leading minus sign: you can simply invert digits in any positive number.

-1	Z
-2	Z1
-3	Z0
-4	ZZ
-5	Z11

Note that a negative number starts with Z and positive with 1 .

Conversion algorithm

It is easy to represent a given number in **balanced ternary** via temporary representing it in normal ternary number system. When value is in standard ternary, its digits are either 0 or 1 or 2 . Iterating from the lowest digit we can safely skip any 0 s and 1 s, however 2 should be turned into Z with adding 1 to the next digit. Digits 3 should be turned into 0 on the same terms - such digits are not present in the number initially but they can be encountered after increasing some 2 s.

Example 1: Let us convert 64 to balanced ternary. At first we use normal ternary to rewrite the number:

$$64_{10} = 02101_3$$

Let us process it from the least significant (rightmost) digit:

- $1, 0$ and 1 are skipped as it is. (Because 0 and 1 are allowed in balanced ternary)
- 2 is turned into Z increasing the digit to its left, so we get $1Z101$.

The final result is $1Z101$.

Let us convert it back to the decimal system by adding the weighted positional values:

$$1Z101 = 81 \cdot 1 + 27 \cdot (-1) + 9 \cdot 1 + 3 \cdot 0 + 1 \cdot 1 = 64_{10}$$

Example 2: Let us convert 237 to balanced ternary. At first we use normal ternary to rewrite the number:

$$237_{10} = 22210_3$$

Let us process it from the least significant (rightmost) digit:

- 0 and 1 are skipped as it is. (Because 0 and 1 are allowed in balanced ternary)
- 2 is turned into Z increasing the digit to its left, so we get $23Z10$.
- 3 is turned into 0 increasing the digit to its left, so we get $30Z10$.
- 3 is turned into 0 increasing the digit to its left (which is by default 0), and so we get $100Z10$.

The final result is $100Z10$.

Let us convert it back to the decimal system by adding the weighted positional values:

$$100Z10 = 243 \cdot 1 + 81 \cdot 0 + 27 \cdot 0 + 9 \cdot (-1) + 3 \cdot 1 + 1 \cdot 0 = 237_{10}$$

Practice Problems

- [Topcoder SRM 604, Div1-250](#)

Contributors:

[RodionGork](#) (45.24%) [gampu](#) (34.52%) [Oleksandr Kulkov](#) (7.14%) [Jakob Kogler](#) (4.76%) [thanhtnguyen](#) (4.76%) [Bhuvnesh Jain](#) (2.38%)
[wiktorkuchta](#) (1.19%)

Translated From: e-maxx.ru

Gray code

Gray code is a binary numeral system where two successive values differ in only one bit.

For example, the sequence of Gray codes for 3-bit numbers is: 000, 001, 011, 010, 110, 111, 101, 100, so $G(4) = 6$.

This code was invented by Frank Gray in 1953.

Finding Gray code

Let's look at the bits of number n and the bits of number $G(n)$. Notice that i -th bit of $G(n)$ equals 1 only when i -th bit of n equals 1 and $i + 1$ -th bit equals 0 or the other way around (i -th bit equals 0 and $i + 1$ -th bit equals 1). Thus, $G(n) = n \oplus (n \gg 1)$:

```
int g (int n) {
    return n ^ (n >> 1);
}
```

Finding inverse Gray code

Given Gray code g , restore the original number n .

We will move from the most significant bits to the least significant ones (the least significant bit has index 1 and the most significant bit has index k). The relation between the bits n_i of number n and the bits g_i of number g :

$$\begin{aligned} n_k &= g_k, \\ n_{k-1} &= g_{k-1} \oplus n_k = g_k \oplus g_{k-1}, \\ n_{k-2} &= g_{k-2} \oplus n_{k-1} = g_k \oplus g_{k-1} \oplus g_{k-2}, \\ n_{k-3} &= g_{k-3} \oplus n_{k-2} = g_k \oplus g_{k-1} \oplus g_{k-2} \oplus g_{k-3}, \quad \vdots \end{aligned}$$

The easiest way to write it in code is:

```
int rev_g (int g) {
    int n = 0;
    for (; g; g >>= 1)
        n ^= g;
    return n;
}
```

Practical applications

Gray codes have some useful applications, sometimes quite unexpected:

- Gray code of n bits forms a Hamiltonian cycle on a hypercube, where each bit corresponds to one dimension.
- Gray codes are used to minimize the errors in digital-to-analog signals conversion (for example, in sensors).
- Gray code can be used to solve the Towers of Hanoi problem. Let n denote number of disks. Start with Gray code of length n which consists of all zeroes ($G(0)$) and move between consecutive Gray codes (from $G(i)$ to $G(i + 1)$). Let i -th bit of current Gray code represent n -th disk (the least significant bit corresponds to the smallest disk and the most significant bit to the biggest disk). Since exactly one bit changes on each step, we can treat changing i -th bit as moving i -th disk. Notice that there is exactly one move option for each disk (except the smallest one) on each step (except start and finish positions). There are always two move options for the smallest disk but there is a strategy which will always lead to answer: if n is odd then sequence of the smallest disk moves looks like $f \rightarrow t \rightarrow r \rightarrow f \rightarrow t \rightarrow r \rightarrow \dots$ where f is the initial rod, t is the terminal rod and r is the remaining rod), and if n is even: $f \rightarrow r \rightarrow t \rightarrow f \rightarrow r \rightarrow t \rightarrow \dots$.

- Gray codes are also used in genetic algorithms theory.

Practice Problems

- [Gray Code](#) [Difficulty: easy]
- [SGU #249 "Matrix"](#) [Difficulty: medium]

Contributors:

[Taras Panychok](#) (35.14%) [Mariia Mykhailova](#) (28.38%) [Anton Älgmyr](#) (20.27%) [Oleksandr Kulkov](#) (8.11%) [Jakob Kogler](#) (5.41%) [Zarif Muhtasim](#) (1.35%)
[traxex2](#) (1.35%)

Original

Bit manipulation

Binary number

A **binary number** is a number expressed in the base-2 numeral system or binary numeral system, it is a method of mathematical expression which uses only two symbols: typically "0" (zero) and "1" (one).

We say that a certain bit is **set**, if it is one, and **cleared** if it is zero.

The binary number $(a_k a_{k-1} \dots a_1 a_0)_2$ represents the number:

$$(a_k a_{k-1} \dots a_1 a_0)_2 = a_k \cdot 2^k + a_{k-1} \cdot 2^{k-1} + \dots + a_1 \cdot 2^1 + a_0 \cdot 2^0.$$

For instance the binary number 1101_2 represents the number 13:

$$\begin{aligned} 1101_2 &= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13 \end{aligned}$$

Computers represent integers as binary numbers. Positive integers (both signed and unsigned) are just represented with their binary digits, and negative signed numbers (which can be positive and negative) are usually represented with the [Two's complement](#).

```
unsigned int unsigned_number = 13;
assert(unsigned_number == 0b1101);

int positive_signed_number = 13;
assert(positive_signed_number == 0b1101);

int negative_signed_number = -13;
assert(negative_signed_number == 0b1111'1111'1111'1111'1111'1111'1111'0011);
```

CPUs are very fast manipulating those bits with specific operations. For some problems we can take these binary number representations to our advantage, and speed up the execution time. And for some problems (typically in combinatorics or dynamic programming) where we want to track which objects we already picked from a given set of objects, we can just use an large enough integer where each digit represents an object and depending on if we pick or drop the object we set or clear the digit.

Bit operators

All those introduced operators are instant (same speed as an addition) on a CPU for fixed-length integers.

Bitwise operators

- **&** : The bitwise AND operator compares each bit of its first operand with the corresponding bit of its second operand. If both bits are 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.
- **|** : The bitwise inclusive OR operator compares each bit of its first operand with the corresponding bit of its second operand. If one of the two bits is 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.
- **^** : The bitwise exclusive OR (XOR) operator compares each bit of its first operand with the corresponding bit of its second operand. If one bit is 0 and the other bit is 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.
- **~** : The bitwise complement (NOT) operator flips each bit of a number, if a bit is set the operator will clear it, if it is cleared the operator sets it.

Examples:

```
n      = 01011000
n-1    = 01010111
-----
n & (n-1) = 01010000
```

```
n      = 01011000
n-1    = 01010111
-----
n | (n-1) = 01011111
```

```
n      = 01011000
n-1    = 01010111
-----
n ^ (n-1) = 00001111
```

```
n      = 01011000
-----
~n     = 10100111
```

Shift operators

There are two operators for shifting bits.

- $\gg$ Shifts a number to the right by removing the last few binary digits of the number. Each shift by one represents an integer division by 2, so a right shift by k represents an integer division by 2^k .

E.g. $5 \gg 2 = 101_2 \gg 2 = 1_2 = 1$ which is the same as $\frac{5}{2^2} = \frac{5}{4} = 1$. For a computer though shifting some bits is a lot faster than doing divisions.

- $\ll$ Shifts a number to left by appending zero digits. In similar fashion to a right shift by k , a left shift by k represents a multiplication by 2^k .

E.g. $5 \ll 3 = 101_2 \ll 3 = 101000_2 = 40$ which is the same as $5 \cdot 2^3 = 5 \cdot 8 = 40$.

Notice however that for a fixed-length integer that means dropping the most left digits, and if you shift too much you end up with the number 0.

Useful tricks

Set/flip/clear a bit

Using bitwise shifts and some basic bitwise operations we can easily set, flip or clear a bit. $1 \ll x$ is a number with only the x -th bit set, while $\sim(1 \ll x)$ is a number with all bits set except the x -th bit.

- $n | (1 \ll x)$ sets the x -th bit in the number n
- $n \wedge (1 \ll x)$ flips the x -th bit in the number n
- $n \& \sim(1 \ll x)$ clears the x -th bit in the number n

Check if a bit is set

The value of the x -th bit can be checked by shifting the number x positions to the right, so that the x -th bit is at the unit place, after which we can extract it by performing a bitwise $\&$ with 1.

```
bool is_set(unsigned int number, int x) {
    return (number >> x) & 1;
}
```

Check if the number is divisible by a power of 2

Using the and operation, we can check if a number n is even because $n \& 1 = 0$ if n is even, and $n \& 1 = 1$ if n is odd. More generally, n is divisible by 2^k exactly when $n \& (2^k - 1) = 0$.

```
bool isDivisibleByPowerOf2(int n, int k) {
    int powerOf2 = 1 << k;
    return (n & (powerOf2 - 1)) == 0;
}
```

We can calculate 2^k by left shifting 1 by k positions. The trick works, because $2^k - 1$ is a number that consists of exactly k ones. And a number that is divisible by 2^k must have zero digits in those places.

Check if an integer is a power of 2

A power of two is a number that has only a single bit in it (e.g. $32 = 0010\ 0000_2$), while the predecessor of that number has that digit not set and all the digits after it set ($31 = 0001\ 1111_2$). So the bitwise AND of a number with its predecessor will always be 0, as they don't have any common digits set. You can easily check that this only happens for the power of twos and for the number 0 which already has no digit set.

```
bool isPowerOfTwo(unsigned int n) {
    return n && !(n & (n - 1));
}
```

Clear the right-most set bit

The expression $n \& (n - 1)$ can be used to turn off the rightmost set bit of a number n . This works because the expression $n - 1$ flips all bits after the rightmost set bit of n , including the rightmost set bit. So all those digits are different from the original number, and by doing a bitwise AND they are all set to 0, giving you the original number n with the rightmost set bit flipped.

For example, consider the number $52 = 0011\ 0100_2$:

```
n      = 00110100
n-1    = 00110011
-----
n & (n-1) = 00110000
```

Brian Kernighan's algorithm

We can count the number of bits set with the above expression.

The idea is to consider only the set bits of an integer by turning off its rightmost set bit (after counting it), so the next iteration of the loop considers the Next Rightmost bit.

```
int countSetBits(int n)
{
    int count = 0;
    while (n)
    {
        n = n & (n - 1);
        count++;
    }
    return count;
}
```

Count set bits upto n

To count the number of set bits of all numbers upto the number n (inclusive), we can run the Brian Kernighan's algorithm on all numbers upto n . But this will result in a "Time Limit Exceeded" in contest submissions.

We can use the fact that for numbers upto 2^x (i.e. from 1 to $2^x - 1$) there are $x \cdot 2^{x-1}$ set bits. This can be visualised as follows.

```
0 -> 0 0 0 0
1 -> 0 0 0 1
2 -> 0 0 1 0
3 -> 0 0 1 1
4 -> 0 1 0 0
5 -> 0 1 0 1
6 -> 0 1 1 0
7 -> 0 1 1 1
8 -> 1 0 0 0
```

We can see that the all the columns except the leftmost have 4 (i.e. 2^2) set bits each, i.e. upto the number $2^3 - 1$, the number of set bits is $3 \cdot 2^{3-1}$.

With the new knowledge in hand we can come up with the following algorithm:

- Find the highest power of 2 that is lesser than or equal to the given number. Let this number be x .
- Calculate the number of set bits from 1 to $2^x - 1$ by using the formula $x \cdot 2^{x-1}$.
- Count the no. of set bits in the most significant bit from 2^x to n and add it.
- Subtract 2^x from n and repeat the above steps using the new n .

```
int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        int x = std::bit_width(n) - 1;
        count += x << (x - 1);
        n -= 1 << x;
        count += n + 1;
    }
    return count;
}
```

Additional tricks

- $n \& (n + 1)$ clears all trailing ones: $0011\ 0111_2 \rightarrow 0011\ 0000_2$.
- $n | (n + 1)$ sets the last cleared bit: $0011\ 0101_2 \rightarrow 0011\ 0111_2$.
- $n \& -n$ extracts the last set bit: $0011\ 0100_2 \rightarrow 0000\ 0100_2$.

Many more can be found in the book [Hacker's Delight](#).

Language and compiler support

C++ supports some of those operations since C++20 via the [bit](#) standard library:

- `has_single_bit`: checks if the number is a power of two
- `bit_ceil` / `bit_floor`: round up/down to the next power of two
- `rotl` / `rotr`: rotate the bits in the number
- `countl_zero` / `countr_zero` / `countl_one` / `countr_one`: count the leading/trailing zeros/ones
- `popcount`: count the number of set bits

Additionally, there are also predefined functions in some compilers that help working with bits. E.g. GCC defines a list at [Built-in Functions Provided by GCC](#) that also work in older versions of C++:

- `__builtin_popcount(unsigned int)` returns the number of set bits (`__builtin_popcount(0b0001'0010'1100) == 4`)
- `__builtin_ffs(int)` finds the index of the first (most right) set bit (`__builtin_ffs(0b0001'0010'1100) == 3`)
- `__builtin_clz(unsigned int)` the count of leading zeros (`__builtin_clz(0b0001'0010'1100) == 23`)
- `__builtin_ctz(unsigned int)` the count of trailing zeros (`__builtin_ctz(0b0001'0010'1100) == 2`)
- `__builtin_parity(x)` the parity (even or odd) of the number of ones in the bit representation

Note that some of the operations (both the C++20 functions and the Compiler Built-in ones) might be quite slow in GCC if you don't enable a specific compiler target with `#pragma GCC target("popcnt")`.

Practice Problems

- [Codeforces - Raising Bacteria](#)
- [Codeforces - Fedor and New Game](#)
- [Codeforces - And Then There Were K](#)

Contributors:

Jakob Kogler (44.62%) Carlos Javier Blanco (35.38%) Ganeshvar Balaji (14.23%) Harsh Dev Pathak (3.85%) Akash Nayar (0.38%)
Konstantinos Alatzas (0.38%) Le Bao Hiep (0.38%) Oleksandr Kulkov (0.38%) Yan Huang (0.38%)

Translated

From: e-maxx.ru

Submask Enumeration

Enumerating all submasks of a given mask

Given a bitmask m , you want to efficiently iterate through all of its submasks, that is, masks s in which only bits that were included in mask m are set.

Consider the implementation of this algorithm, based on tricks with bit operations:

```
int s = m;
while (s > 0) {
    ... you can use s ...
    s = (s-1) & m;
}
```

or, using a more compact `for` statement:

```
for (int s=m; s; s=(s-1)&m)
    ... you can use s ...
```

In both variants of the code, the submask equal to zero will not be processed. We can either process it outside the loop, or use a less elegant design, for example:

```
for (int s=m; ; s=(s-1)&m) {
    ... you can use s ...
    if (s==0) break;
}
```

Let us examine why the above code visits all submasks of m , without repetition, and in descending order.

Suppose we have a current bitmask s , and we want to move on to the next bitmask. By subtracting from the mask s one unit, we will remove the rightmost set bit and all bits to the right of it will become 1. Then we remove all the "extra" one bits that are not included in the mask m and therefore can't be a part of a submask. We do this removal by using the bitwise operation $(s-1) \& m$. As a result, we "cut" mask $s-1$ to determine the highest value that it can take, that is, the next submask after s in descending order.

Thus, this algorithm generates all submasks of this mask in descending order, performing only two operations per iteration.

A special case is when $s = 0$. After executing $s-1$ we get a mask where all bits are set (bit representation of -1), and after $(s-1) \& m$ we will have that s will be equal to m . Therefore, with the mask $s = 0$ be careful — if the loop does not end at zero, the algorithm may enter an infinite loop.

Iterating through all masks with their submasks. Complexity $O(3^n)$

In many problems, especially those that use bitmask dynamic programming, you want to iterate through all bitmasks and for each mask, iterate through all of its submasks:

```
for (int m=0; m<(1<<n); ++m)
    for (int s=m; s; s=(s-1)&m)
        ... s and m ...
```

Let's prove that the inner loop will execute a total of $O(3^n)$ iterations.

First proof: Consider the i -th bit. There are exactly three options for it:

1. it is not included in the mask m (and therefore not included in submask s),

2. it is included in m , but not included in s , or
3. it is included in both m and s .

As there are a total of n bits, there will be 3^n different combinations.

Second proof: Note that if mask m has k enabled bits, then it will have 2^k submasks. As we have a total of $\binom{n}{k}$ masks with k enabled bits (see [binomial coefficients](#)), then the total number of combinations for all masks will be:

$$\sum_{k=0}^n \binom{n}{k} \cdot 2^k$$

To calculate this number, note that the sum above is equal to the expansion of $(1 + 2)^n$ using the binomial theorem. Therefore, we have 3^n combinations, as we wanted to prove.

Practice Problems

- [Atcoder - Close Group](#)
- [Codeforces - Nuclear Fusion](#)
- [Codeforces - Sandy and Nuts](#)
- [Uva 1439 - Exclusive Access 2](#)
- [Uva 11825 - Hackers' Crackdown](#)

Contributors:

[ffao](#) (67.09%) [Abhijeet Krishnan](#) (12.66%) [prpr](#) (8.86%) [Oleksandr Kulkov](#) (7.59%) [Abhishek kumar](#) (1.27%) [Gabriel Silva Simões](#) (1.27%)
[Jakob Kogler](#) (1.27%)

Translated From: e-maxx.ru

Arbitrary-Precision Arithmetic

Arbitrary-Precision arithmetic, also known as "bignum" or simply "long arithmetic" is a set of data structures and algorithms which allows to process much greater numbers than can be fit in standard data types. Here are several types of arbitrary-precision arithmetic.

Classical Integer Long Arithmetic

The main idea is that the number is stored as an array of its "digits" in some base. Several most frequently used bases are decimal, powers of decimal (10^4 or 10^9) and binary.

Operations on numbers in this form are performed using "school" algorithms of column addition, subtraction, multiplication and division. It's also possible to use fast multiplication algorithms: fast Fourier transform and Karatsuba algorithm.

Here we describe long arithmetic for only non-negative integers. To extend the algorithms to handle negative integers one has to introduce and maintain additional "negative number" flag or use two's complement integer representation.

Data Structure

We'll store numbers as a `vector<int>`, in which each element is a single "digit" of the number.

```
typedef vector<int> lnum;
```

To improve performance we'll use 10^9 as the base, so that each "digit" of the long number contains 9 decimal digits at once.

```
const int base = 1000*1000*1000;
```

Digits will be stored in order from least to most significant. All operations will be implemented so that after each of them the result doesn't have any leading zeros, as long as operands didn't have any leading zeros either. All operations which might result in a number with leading zeros should be followed by code which removes them. Note that in this representation there are two valid notations for number zero: an empty vector, and a vector with a single zero digit.

Output

Printing the long integer is the easiest operation. First we print the last element of the vector (or 0 if the vector is empty), followed by the rest of the elements padded with leading zeros if necessary so that they are exactly 9 digits long.

```
printf ("%d", a.empty() ? 0 : a.back());
for (int i=(int)a.size()-2; i>=0; --i)
    printf ("%09d", a[i]);
```

Note that we cast `a.size()` to integer to avoid unsigned integer underflow if vector contains less than 2 elements.

Input

To read a long integer, read its notation into a `string` and then convert it to "digits":

```
for (int i=(int)s.length(); i>0; i-=9)
    if (i < 9)
        a.push_back (atoi (s.substr (0, i).c_str()));
    else
        a.push_back (atoi (s.substr (i-9, 9).c_str()));
```

If we use an array of `char` instead of a `string`, the code will be even shorter:

```
for (int i=(int)strlen(s); i>0; i-=9) {
    s[i] = 0;
    a.push_back (atoi (i>=9 ? s+i-9 : s));
}
```

If the input can contain leading zeros, they can be removed as follows:

```
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Addition

Increment long integer a by b and store result in a:

```
int carry = 0;
for (size_t i=0; i<max(a.size(),b.size()) || carry; ++i) {
    if (i == a.size())
        a.push_back (0);
    a[i] += carry + (i < b.size() ? b[i] : 0);
    carry = a[i] >= base;
    if (carry) a[i] -= base;
}
```

Subtraction

Decrement long integer a by b ($a \geq b$) and store result in a:

```
int carry = 0;
for (size_t i=0; i<b.size() || carry; ++i) {
    a[i] -= carry + (i < b.size() ? b[i] : 0);
    carry = a[i] < 0;
    if (carry) a[i] += base;
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Note that after performing subtraction we remove leading zeros to keep up with the premise that our long integers don't have leading zeros.

Multiplication by short integer

Multiply long integer a by short integer b ($b < \text{base}$) and store result in a:

```
int carry = 0;
for (size_t i=0; i<a.size() || carry; ++i) {
    if (i == a.size())
        a.push_back (0);
    long long cur = carry + a[i] * 111 * b;
    a[i] = int (cur % base);
    carry = int (cur / base);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Additional optimization: If runtime is extremely important, you can try to replace two divisions with one by finding only integer result of division (variable `carry`) and then use it to find modulo using multiplication. This usually makes the code faster, though not dramatically.

Multiplication by long integer

Multiply long integers a and b and store result in c:

```
lnum c (a.size()+b.size());
for (size_t i=0; i<a.size(); ++i)
    for (int j=0, carry=0; j<(int)b.size() || carry; ++j) {
        long long cur = c[i+j] + a[i] * 111 * (j < (int)b.size() ? b[j] : 0) + carry;
```

```

        c[i+j] = int (cur % base);
        carry = int (cur / base);
    }
    while (c.size() > 1 && c.back() == 0)
        c.pop_back();

```

Division by short integer

Divide long integer a by short integer b ($b < \text{base}$), store integer result in a and remainder in `carry`:

```

int carry = 0;
for (int i=(int)a.size()-1; i>=0; --i) {
    long long cur = a[i] + carry * 111 * base;
    a[i] = int (cur / b);
    carry = int (cur % b);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();

```

Long Integer Arithmetic for Factorization Representation

The idea is to store the integer as its factorization, i.e. the powers of primes which divide it.

This approach is very easy to implement, and allows to do multiplication and division easily (asymptotically faster than the classical method), but not addition or subtraction. It is also very memory-efficient compared to the classical approach.

This method is often used for calculations modulo non-prime number M ; in this case a number is stored as powers of divisors of M which divide the number, plus the remainder modulo M .

Long Integer Arithmetic in prime modulus (Garner Algorithm)

The idea is to choose a set of prime numbers (typically they are small enough to fit into standard integer data type) and to store an integer as a vector of remainders from division of the integer by each of those primes.

Chinese remainder theorem states that this representation is sufficient to uniquely restore any number from 0 to product of these primes minus one. [Garner algorithm](#) allows to restore the number from such representation to normal integer.

This method allows to save memory compared to the classical approach (though the savings are not as dramatic as in factorization representation). Besides, it allows to perform fast addition, subtraction and multiplication in time proportional to the number of prime numbers used as modulus (see [Chinese remainder theorem](#) article for implementation).

The tradeoff is that converting the integer back to normal form is rather laborious and requires implementing classical arbitrary-precision arithmetic with multiplication. Besides, this method doesn't support division.

Fractional Arbitrary-Precision Arithmetic

Fractions occur in programming competitions less frequently than integers, and long arithmetic is much trickier to implement for fractions, so programming competitions feature only a small subset of fractional long arithmetic.

Arithmetic in Irreducible Fractions

A number is represented as an irreducible fraction $\frac{a}{b}$, where a and b are integers. All operations on fractions can be represented as operations on integer numerators and denominators of these fractions. Usually this requires using classical arbitrary-precision arithmetic for storing numerator and denominator, but sometimes a built-in 64-bit integer data type suffices.

Storing Floating Point Position as Separate Type

Sometimes a problem requires handling very small or very large numbers without allowing overflow or underflow. Built-in double data type uses 8-10 bytes and allows values of the exponent in $[-308; 308]$ range, which sometimes might be insufficient.

The approach is very simple: a separate integer variable is used to store the value of the exponent, and after each operation the floating-point number is normalized, i.e. returned to $[0.1; 1)$ interval by adjusting the exponent accordingly.

When two such numbers are multiplied or divided, their exponents should be added or subtracted, respectively. When numbers are added or subtracted, they have to be brought to common exponent first by multiplying one of them by 10 raised to the power equal to the difference of exponent values.

As a final note, the exponent base doesn't have to equal 10. Based on the internal representation of floating-point numbers, it makes most sense to use 2 as the exponent base.

Practice Problems

- [UVA - How Many Fibs?](#)
- [UVA - Product](#)
- [UVA - Maximum Sub-sequence Product](#)
- [SPOJ - Fast Multiplication](#)
- [SPOJ - GCD2](#)
- [UVA - Division](#)
- [UVA - Fibonacci Freeze](#)
- [UVA - Krakovia](#)
- [UVA - Simplifying Fractions](#)
- [UVA - 500!](#)
- [Hackerrank - Factorial digit sum](#)
- [UVA - Immortal Rabbits](#)
- [SPOJ - 0110SS](#)
- [Codeforces - Notepad](#)

Contributors:

[Mariia Mykhailova](#) (76.08%) [Morass](#) (8.61%) [Anton Ālgmyr](#) (5.26%) [Rodion Gorkovenko](#) (5.26%) [Oleksandr Kulkov](#) (2.87%) [AYUSH KUMAR TIWARI](#) (0.48%)
[Jakob Kogler](#) (0.48%) [Manan Grover](#) (0.48%) [wiktorkuchta](#) (0.48%)

Translated From: e-maxx.ru

Fast Fourier transform

In this article we will discuss an algorithm that allows us to multiply two polynomials of length n in $O(n \log n)$ time, which is better than the trivial multiplication which takes $O(n^2)$ time. Obviously also multiplying two long numbers can be reduced to multiplying polynomials, so also two long numbers can be multiplied in $O(n \log n)$ time (where n is the number of digits in the numbers).

The discovery of the **Fast Fourier transformation (FFT)** is attributed to Cooley and Tukey, who published an algorithm in 1965. But in fact the FFT has been discovered repeatedly before, but the importance of it was not understood before the inventions of modern computers. Some researchers attribute the discovery of the FFT to Runge and König in 1924. But actually Gauss developed such a method already in 1805, but never published it.

Notice, that the FFT algorithm presented here runs in $O(n \log n)$ time, but it doesn't work for multiplying arbitrary big polynomials with arbitrary large coefficients or for multiplying arbitrary big integers. It can easily handle polynomials of size 10^5 with small coefficients, or multiplying two numbers of size 10^6 , which is usually enough for solving competitive programming problems. Beyond the scale of multiplying numbers with 10^6 bits, the range and precision of the floating point numbers used during the computation will not be enough to give accurate final results, though there are more complex variations that can perform arbitrary large polynomial/integer multiplications. E.g. in 1971 Schönhage and Strasser developed a variation for multiplying arbitrary large numbers that applies the FFT recursively in rings structures running in $O(n \log n \log \log n)$. And recently (in 2019) Harvey and van der Hoeven published an algorithm that runs in true $O(n \log n)$.

Discrete Fourier transform

Let there be a polynomial of degree $n - 1$:

$$A(x) = a_0x^0 + a_1x^1 + \dots + a_{n-1}x^{n-1}$$

Without loss of generality we assume that n - the number of coefficients - is a power of 2. If n is not a power of 2, then we simply add the missing terms a_ix^i and set the coefficients a_i to 0.

The theory of complex numbers tells us that the equation $x^n = 1$ has n complex solutions (called the n -th roots of unity), and the solutions are of the form $w_{n,k} = e^{\frac{2k\pi i}{n}}$ with $k = 0 \dots n - 1$. Additionally these complex numbers have some very interesting properties: e.g. the principal n -th root $w_n = w_{n,1} = e^{\frac{2\pi i}{n}}$ can be used to describe all other n -th roots: $w_{n,k} = (w_n)^k$.

The **discrete Fourier transform (DFT)** of the polynomial $A(x)$ (or equivalently the vector of coefficients $(a_0, a_1, \dots, a_{n-1})$) is defined as the values of the polynomial at the points $x = w_{n,k}$, i.e. it is the vector:

$$\begin{aligned} \text{DFT}(a_0, a_1, \dots, a_{n-1}) &= (y_0, y_1, \dots, y_{n-1}) \\ &= (A(w_{n,0}), A(w_{n,1}), \dots, A(w_{n,n-1})) \\ &= (A(w_n^0), A(w_n^1), \dots, A(w_n^{n-1})) \end{aligned}$$

Similarly the **inverse discrete Fourier transform** is defined: The inverse DFT of values of the polynomial $(y_0, y_1, \dots, y_{n-1})$ are the coefficients of the polynomial $(a_0, a_1, \dots, a_{n-1})$.

$$\text{InverseDFT}(y_0, y_1, \dots, y_{n-1}) = (a_0, a_1, \dots, a_{n-1})$$

Thus, if a direct DFT computes the values of the polynomial at the points at the n -th roots, the inverse DFT can restore the coefficients of the polynomial using those values.

Application of the DFT: fast multiplication of polynomials

Let there be two polynomials A and B . We compute the DFT for each of them: $\text{DFT}(A)$ and $\text{DFT}(B)$.

What happens if we multiply these polynomials? Obviously at each point the values are simply multiplied, i.e.

$$(A \cdot B)(x) = A(x) \cdot B(x).$$

This means that if we multiply the vectors $\text{DFT}(A)$ and $\text{DFT}(B)$ - by multiplying each element of one vector by the corresponding element of the other vector - then we get nothing other than the DFT of the polynomial $\text{DFT}(A \cdot B)$:

$$\text{DFT}(A \cdot B) = \text{DFT}(A) \cdot \text{DFT}(B)$$

Finally, applying the inverse DFT, we obtain:

$$A \cdot B = \text{InverseDFT}(\text{DFT}(A) \cdot \text{DFT}(B))$$

On the right the product of the two DFTs we mean the pairwise product of the vector elements. This can be computed in $O(n)$ time. If we can compute the DFT and the inverse DFT in $O(n \log n)$, then we can compute the product of the two polynomials (and consequently also two long numbers) with the same time complexity.

It should be noted, that the two polynomials should have the same degree. Otherwise the two result vectors of the DFT have different length. We can accomplish this by adding coefficients with the value 0.

And also, since the result of the product of two polynomials is a polynomial of degree $2(n - 1)$, we have to double the degrees of each polynomial (again by padding 0s). From a vector with n values we cannot reconstruct the desired polynomial with $2n - 1$ coefficients.

Fast Fourier Transform

The **fast Fourier transform** is a method that allows computing the DFT in $O(n \log n)$ time. The basic idea of the FFT is to apply divide and conquer. We divide the coefficient vector of the polynomial into two vectors, recursively compute the DFT for each of them, and combine the results to compute the DFT of the complete polynomial.

So let there be a polynomial $A(x)$ with degree $n - 1$, where n is a power of 2, and $n > 1$:

$$A(x) = a_0x^0 + a_1x^1 + \dots + a_{n-1}x^{n-1}$$

We divide it into two smaller polynomials, the one containing only the coefficients of the even positions, and the one containing the coefficients of the odd positions:

$$\begin{aligned} A_0(x) &= a_0x^0 + a_2x^1 + \dots + a_{n-2}x^{\frac{n}{2}-1} \\ A_1(x) &= a_1x^0 + a_3x^1 + \dots + a_{n-1}x^{\frac{n}{2}-1} \end{aligned}$$

It is easy to see that

$$A(x) = A_0(x^2) + xA_1(x^2).$$

The polynomials A_0 and A_1 have only half as many coefficients as the polynomial A . If we can compute the $\text{DFT}(A)$ in linear time using $\text{DFT}(A_0)$ and $\text{DFT}(A_1)$, then we get the recurrence $T_{\text{DFT}}(n) = 2T_{\text{DFT}}(\frac{n}{2}) + O(n)$ for the time complexity, which results in $T_{\text{DFT}}(n) = O(n \log n)$ by the **master theorem**.

Let's learn how we can accomplish that.

Suppose we have computed the vectors $(y_k^0)_{k=0}^{n/2-1} = \text{DFT}(A_0)$ and $(y_k^1)_{k=0}^{n/2-1} = \text{DFT}(A_1)$. Let us find an expression for $(y_k)_{k=0}^{n-1} = \text{DFT}(A)$.

For the first $\frac{n}{2}$ values we can just use the previously noted equation $A(x) = A_0(x^2) + xA_1(x^2)$:

$$y_k = y_k^0 + w_n^k y_k^1, \quad k = 0 \dots \frac{n}{2} - 1.$$

However for the second $\frac{n}{2}$ values we need to find a slightly different expression:

$$\begin{aligned} y_{k+n/2} &= A(w_n^{k+n/2}) \\ &= A_0(w_n^{2k+n}) + w_n^{k+n/2} A_1(w_n^{2k+n}) \\ &= A_0(w_n^{2k} w_n^n) + w_n^k w_n^{n/2} A_1(w_n^{2k} w_n^n) \\ &= A_0(w_n^{2k}) - w_n^k A_1(w_n^{2k}) \\ &= y_k^0 - w_n^k y_k^1 \end{aligned}$$

Here we used again $A(x) = A_0(x^2) + x A_1(x^2)$ and the two identities $w_n^n = 1$ and $w_n^{n/2} = -1$.

Therefore we get the desired formulas for computing the whole vector (y_k) :

$$\begin{aligned} y_k &= y_k^0 + w_n^k y_k^1, & k &= 0 \dots \frac{n}{2} - 1, \\ y_{k+n/2} &= y_k^0 - w_n^k y_k^1, & k &= 0 \dots \frac{n}{2} - 1. \end{aligned}$$

(This pattern $a + b$ and $a - b$ is sometimes called a **butterfly**.)

Thus we learned how to compute the DFT in $O(n \log n)$ time.

Inverse FFT

Let the vector $(y_0, y_1, \dots, y_{n-1})$ - the values of polynomial A of degree $n - 1$ in the points $x = w_n^k$ - be given. We want to restore the coefficients $(a_0, a_1, \dots, a_{n-1})$ of the polynomial. This known problem is called **interpolation**, and there are general algorithms for solving it. But in this special case (since we know the values of the points at the roots of unity), we can obtain a much simpler algorithm (that is practically the same as the direct FFT).

We can write the DFT, according to its definition, in the matrix form:

$$\begin{pmatrix} w_n^0 & w_n^0 & w_n^0 & w_n^0 & \dots & w_n^0 \\ w_n^0 & w_n^1 & w_n^2 & w_n^3 & \dots & w_n^{n-1} \\ w_n^0 & w_n^2 & w_n^4 & w_n^6 & \dots & w_n^{2(n-1)} \\ w_n^0 & w_n^3 & w_n^6 & w_n^9 & \dots & w_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_n^0 & w_n^{n-1} & w_n^{2(n-1)} & w_n^{3(n-1)} & \dots & w_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

This matrix is called the **Vandermonde matrix**.

Thus we can compute the vector $(a_0, a_1, \dots, a_{n-1})$ by multiplying the vector $(y_0, y_1, \dots, y_{n-1})$ from the left with the inverse of the matrix:

$$\begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} w_n^0 & w_n^0 & w_n^0 & w_n^0 & \cdots & w_n^0 \\ w_n^0 & w_n^1 & w_n^2 & w_n^3 & \cdots & w_n^{n-1} \\ w_n^0 & w_n^2 & w_n^4 & w_n^6 & \cdots & w_n^{2(n-1)} \\ w_n^0 & w_n^3 & w_n^6 & w_n^9 & \cdots & w_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_n^0 & w_n^{n-1} & w_n^{2(n-1)} & w_n^{3(n-1)} & \cdots & w_n^{(n-1)(n-1)} \end{pmatrix}^{-1} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

A quick check can verify that the inverse of the matrix has the following form:

$$\frac{1}{n} \begin{pmatrix} w_n^0 & w_n^0 & w_n^0 & w_n^0 & \cdots & w_n^0 \\ w_n^0 & w_n^{-1} & w_n^{-2} & w_n^{-3} & \cdots & w_n^{-(n-1)} \\ w_n^0 & w_n^{-2} & w_n^{-4} & w_n^{-6} & \cdots & w_n^{-2(n-1)} \\ w_n^0 & w_n^{-3} & w_n^{-6} & w_n^{-9} & \cdots & w_n^{-3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_n^0 & w_n^{-(n-1)} & w_n^{-2(n-1)} & w_n^{-3(n-1)} & \cdots & w_n^{-(n-1)(n-1)} \end{pmatrix}$$

Thus we obtain the formula:

$$a_k = \frac{1}{n} \sum_{j=0}^{n-1} y_j w_n^{-kj}$$

Comparing this to the formula for y_k

$$y_k = \sum_{j=0}^{n-1} a_j w_n^{kj},$$

we notice that these problems are almost the same, so the coefficients a_k can be found by the same divide and conquer algorithm, as well as the direct FFT, only instead of w_n^k we have to use w_n^{-k} , and at the end we need to divide the resulting coefficients by n .

Thus the computation of the inverse DFT is almost the same as the calculation of the direct DFT, and it also can be performed in $O(n \log n)$ time.

Implementation

Here we present a simple recursive **implementation of the FFT** and the inverse FFT, both in one function, since the difference between the forward and the inverse FFT are so minimal. To store the complex numbers we use the complex type in the C++ STL.

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    if (n == 1)
        return;

    vector<cd> a0(n / 2), a1(n / 2);
    for (int i = 0; 2 * i < n; i++) {
        a0[i] = a[2*i];
        a1[i] = a[2*i+1];
    }
    fft(a0, invert);
    fft(a1, invert);

    double ang = 2 * PI / n * (invert ? -1 : 1);
    cd w(1), wn(cos(ang), sin(ang));

```

```

for (int i = 0; 2 * i < n; i++) {
    a[i] = a0[i] + w * a1[i];
    a[i + n/2] = a0[i] - w * a1[i];
    if (invert) {
        a[i] /= 2;
        a[i + n/2] /= 2;
    }
    w *= wn;
}
}

```

The function gets passed a vector of coefficients, and the function will compute the DFT or inverse DFT and store the result again in this vector. The argument `invert` shows whether the direct or the inverse DFT should be computed. Inside the function we first check if the length of the vector is equal to one, if this is the case then we don't have to do anything. Otherwise we divide the vector `a` into two vectors `a0` and `a1` and compute the DFT for both recursively. Then we initialize the value `wn` and a variable `w`, which will contain the current power of w_n . Then the values of the resulting DFT are computed using the above formulas.

If the flag `invert` is set, then we replace w_n with w_n^{-1} , and each of the values of the result is divided by 2 (since this will be done in each level of the recursion, this will end up dividing the final values by n).

Using this function we can create a function for **multiplying two polynomials**:

```

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}

```

This function works with polynomials with integer coefficients, however you can also adjust it to work with other types. Since there is some error when working with complex numbers, we need round the resulting coefficients at the end.

Finally the function for **multiplying** two long numbers practically doesn't differ from the function for multiplying polynomials. The only thing we have to do afterwards, is to normalize the number:

```

int carry = 0;
for (int i = 0; i < n; i++)
    result[i] += carry;
    carry = result[i] / 10;
    result[i] %= 10;
}

```

Since the length of the product of two numbers never exceed the total length of both numbers, the size of the vector is enough to perform all carry operations.

Improved implementation: in-place computation

To increase the efficiency we will switch from the recursive implementation to an iterative one. In the above recursive implementation we explicitly separated the vector `a` into two vectors - the element on the even positions got assigned to one temporary vector, and the elements on odd positions to another. However if we reorder the elements in a certain way, we don't need to create these temporary vectors (i.e. all the calculations can be done "in-place", right in the vector `A` itself).

Note that at the first recursion level, the elements whose lowest bit of the position was zero got assigned to the vector `a0`, and the ones with a one as the lowest bit of the position got assigned to `a1`. In the second recursion level the same thing happens, but with the second lowest bit instead, etc. Therefore if we reverse the bits of the position of each coefficient, and sort them by these reversed values, we get the desired order (it is called the bit-reversal permutation).

For example the desired order for $n = 8$ has the form:

$$a = \{ [(a_0, a_4), (a_2, a_6)], [(a_1, a_5), (a_3, a_7)] \}$$

Indeed in the first recursion level (surrounded by curly braces), the vector gets divided into two parts $[a_0, a_2, a_4, a_6]$ and $[a_1, a_3, a_5, a_7]$. As we see, in the bit-reversal permutation this corresponds to simply dividing the vector into two halves: the first $\frac{n}{2}$ elements and the last $\frac{n}{2}$ elements. Then there is a recursive call for each half. Let the resulting DFT for each of them be returned in place of the elements themselves (i.e. the first half and the second half of the vector a respectively).

$$a = \{ [y_0^0, y_1^0, y_2^0, y_3^0], [y_1^1, y_1^1, y_2^1, y_3^1] \}$$

Now we want to combine the two DFTs into one for the complete vector. The order of the elements is ideal, and we can also perform the union directly in this vector. We can take the elements y_0^0 and y_1^1 and perform the butterfly transform. The place of the resulting two values is the same as the place of the two initial values, so we get:

$$a = \{ [y_0^0 + w_n^0 y_1^1, y_2^0 + w_n^0 y_3^1], [y_0^0 - w_n^0 y_1^1, y_2^0 - w_n^0 y_3^1] \}$$

Similarly we can compute the butterfly transform of y_1^0 and y_1^1 and put the results in their place, and so on. As a result we get:

$$a = \{ [y_0^0 + w_n^0 y_1^1 + w_n^1 y_2^0 + w_n^3 y_3^1], [y_0^0 - w_n^0 y_1^1 - w_n^1 y_2^0 - w_n^3 y_3^1], [y_2^0 - w_n^2 y_1^1, y_2^0 - w_n^2 y_3^1] \}$$

Thus we computed the required DFT from the vector a .

Here we described the process of computing the DFT only at the first recursion level, but the same works obviously also for all other levels. Thus, after applying the bit-reversal permutation, we can compute the DFT in-place, without any additional memory.

This additionally allows us to get rid of the recursion. We just start at the lowest level, i.e. we divide the vector into pairs and apply the butterfly transform to them. This results with the vector a with the work of the last level applied. In the next step we divide the vector into vectors of size 4, and again apply the butterfly transform, which gives us the DFT for each block of size 4. And so on. Finally in the last step we obtained the result of the DFTs of both halves of a , and by applying the butterfly transform we obtain the DFT for the complete vector a .

```
using cd = complex<double>;
const double PI = acos(-1);

int reverse(int num, int lg_n) {
    int res = 0;
    for (int i = 0; i < lg_n; i++) {
        if (num & (1 << i))
            res |= (1 << (lg_n - 1 - i));
    }
    return res;
}

void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    int lg_n = 0;
    while ((1 << lg_n) < n)
        lg_n++;

    for (int i = 0; i < n; i++) {
        if (i < reverse(i, lg_n))
            swap(a[i], a[reverse(i, lg_n)]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
            }
            w = w * wlen;
        }
    }
}
```

```

        a[i+j] = u + v;
        a[i+j+len/2] = u - v;
        w *= wlen;
    }
}

if (invert) {
    for (cd & x : a)
        x /= n;
}
}

```

At first we apply the bit-reversal permutation by swapping the each element with the element of the reversed position. Then the $\log n - 1$ states of the algorithm we compute the DFT for each block of the corresponding size len . For all those blocks we have the same root of unity w_{len} . We iterate all blocks and perform the butterfly transform on each of them.

We can further optimize the reversal of the bits. In the previous implementation we iterated all bits of the index and created the bitwise reversed index. However we can reverse the bits in a different way.

Suppose that j already contains the reverse of i . Then by to go to $i + 1$, we have to increment i , and we also have to increment j , but in a "reversed" number system. Adding one in the conventional binary system is equivalent to flip all trailing ones into zeros and flipping the zero right before them into a one. Equivalently in the "reversed" number system, we flip all leading ones, and the also the next zero.

Thus we get the following implementation:

```

using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

```

Additionally we can precompute the bit-reversal permutation beforehand. This is especially useful when the size n is the same for all calls. But even when we only have three calls (which are necessary for multiplying two polynomials), the effect is noticeable. Also we can precompute all roots of unity and their powers.

Number theoretic transform

Now we switch the objective a little bit. We still want to multiply two polynomials in $O(n \log n)$ time, but this time we want to compute the coefficients modulo some prime number p . Of course for this task we can use the normal DFT and apply the modulo operator to the result.

However, doing so might lead to rounding errors, especially when dealing with large numbers. The **number theoretic transform (NTT)** has the advantage, that it only works with integer, and therefore the result are guaranteed to be correct.

The discrete Fourier transform is based on complex numbers, and the n -th roots of unity. To efficiently compute it, we extensively use properties of the roots (e.g. that there is one root that generates all other roots by exponentiation).

But the same properties hold for the n -th roots of unity in modular arithmetic. A n -th root of unity under a primitive field is such a number w_n that satisfies:

$$\begin{aligned}(w_n)^n &= 1 \pmod{p}, \\ (w_n)^k &\neq 1 \pmod{p}, \quad 1 \leq k < n.\end{aligned}$$

The other $n - 1$ roots can be obtained as powers of the root w_n .

To apply it in the fast Fourier transform algorithm, we need a root to exist for some n , which is a power of 2, and also for all smaller powers. We can notice the following interesting property:

$$\begin{aligned}(w_n^2)^m &= w_n^n = 1 \pmod{p}, \quad \text{with } m = \frac{n}{2} \\ (w_n^2)^k &= w_n^{2k} \neq 1 \pmod{p}, \quad 1 \leq k < m.\end{aligned}$$

Thus if w_n is a n -th root of unity, then w_n^2 is a $\frac{n}{2}$ -th root of unity. And consequently for all smaller powers of two there exist roots of the required degree, and they can be computed using w_n .

For computing the inverse DFT, we need the inverse w_n^{-1} of w_n . But for a prime modulus the inverse always exists.

Thus all the properties that we need from the complex roots are also available in modular arithmetic, provided that we have a large enough module p for which a n -th root of unity exists.

For example we can take the following values: module $p = 7340033$, $w_{2^{20}} = 5$. If this module is not enough, we need to find a different pair. We can use that fact that for modules of the form $p = c2^k + 1$ (and p is prime), there always exists the 2^k -th root of unity. It can be shown that g^c is such a 2^k -th root of unity, where g is a [primitive root](#) of p .

```
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1 << 20;

void fft(vector<int> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
                w = (int)(1LL * w * wlen % mod);
            }
        }
    }

    if (invert) {
```

```

    int n_1 = inverse(n, mod);
    for (int & x : a)
        x = (int)(1LL * x * n_1 % mod);
}

```

Here the function `inverse` computes the modular inverse (see [Modular Multiplicative Inverse](#)). The constants `mod`, `root`, `root_pw` determine the module and the root, and `root_1` is the inverse of `root` modulo `mod`.

In practice this implementation is slower than the implementation using complex numbers (due to the huge number of modulo operations), but it has some advantages such as less memory usage and no rounding errors.

Multiplication with arbitrary modulus

Here we want to achieve the same goal as in previous section. Multiplying two polynomial $A(x)$ and $B(x)$, and computing the coefficients modulo some number M . The number theoretic transform only works for certain prime numbers. What about the case when the modulus is not of the desired form?

One option would be to perform multiple number theoretic transforms with different prime numbers of the form $c2^k + 1$, then apply the [Chinese Remainder Theorem](#) to compute the final coefficients.

Another options is to distribute the polynomials $A(x)$ and $B(x)$ into two smaller polynomials each

$$A(x) = A_1(x) + A_2(x) \cdot C$$

$$B(x) = B_1(x) + B_2(x) \cdot C$$

with $C \approx \sqrt{M}$.

Then the product of $A(x)$ and $B(x)$ can then be represented as:

$$A(x) \cdot B(x) = A_1(x) \cdot B_1(x) + (A_1(x) \cdot B_2(x) + A_2(x) \cdot B_1(x)) \cdot C + (A_2(x) \cdot B_2(x)) \cdot C^2$$

The polynomials $A_1(x)$, $A_2(x)$, $B_1(x)$ and $B_2(x)$ contain only coefficients smaller than $\sqrt{M}$, therefore the coefficients of all the appearing products are smaller than $M \cdot n$, which is usually small enough to handle with typical floating point types.

This approach therefore requires computing the products of polynomials with smaller coefficients (by using the normal FFT and inverse FFT), and then the original product can be restored using modular addition and multiplication in $O(n)$ time.

Applications

DFT can be used in a huge variety of other problems, which at the first glance have nothing to do with multiplying polynomials.

All possible sums

We are given two arrays $a[]$ and $b[]$. We have to find all possible sums $a[i] + b[j]$, and for each sum count how often it appears.

For example for $a = [1, 2, 3]$ and $b = [2, 4]$ we get: then sum 3 can be obtained in 1 way, the sum 4 also in 1 way, 5 in 2, 6 in 1, 7 in 1.

We construct for the arrays a and b two polynomials A and B . The numbers of the array will act as the exponents in the polynomial ($a[i] \Rightarrow x^{a[i]}$); and the coefficients of this term will be how often the number appears in the array.

Then, by multiplying these two polynomials in $O(n \log n)$ time, we get a polynomial C , where the exponents will tell us which sums can be obtained, and the coefficients tell us how often. To demonstrate this on the example:

$$(1x^1 + 1x^2 + 1x^3)(1x^2 + 1x^4) = 1x^3 + 1x^4 + 2x^5 + 1x^6 + 1x^7$$

All possible scalar products

We are given two arrays $a[]$ and $b[]$ of length n . We have to compute the products of a with every cyclic shift of b .

We generate two new arrays of size $2n$: We reverse a and append n zeros to it. And we just append b to itself. When we multiply these two arrays as polynomials, and look at the coefficients $c[n-1]$, $c[n]$, $\dots$, $c[2n-2]$ of the product c , we get:

$$c[k] = \sum_{i+j=k} a[i]b[j]$$

And since all the elements $a[i] = 0$ for $i \geq n$:

$$c[k] = \sum_{i=0}^{n-1} a[i]b[k-i]$$

It is easy to see that this sum is just the scalar product of the vector a with the $(k - (n - 1))$ -th cyclic left shift of b . Thus these coefficients are the answer to the problem, and we were still able to obtain it in $O(n \log n)$ time. Note here that $c[2n-1]$ also gives us the n -th cyclic shift but that is the same as the 0-th cyclic shift so we don't need to consider that separately into our answer.

Two stripes

We are given two Boolean stripes (cyclic arrays of values 0 and 1) a and b . We want to find all ways to attach the first stripe to the second one, such that at no position we have a 1 of the first stripe next to a 1 of the second stripe.

The problem doesn't actually differ much from the previous problem. Attaching two stripes just means that we perform a cyclic shift on the second array, and we can attach the two stripes, if scalar product of the two arrays is 0.

String matching

We are given two strings, a text T and a pattern P , consisting of lowercase letters. We have to compute all the occurrences of the pattern in the text.

We create a polynomial for each string ($T[i]$ and $P[i]$ are numbers between 0 and 25 corresponding to the 26 letters of the alphabet):

$$A(x) = a_0x^0 + a_1x^1 + \dots + a_{n-1}x^{n-1}, \quad n = |T|$$

with

$$a_i = \cos(\alpha_i) + i \sin(\alpha_i), \quad \alpha_i = \frac{2\pi T[i]}{26}.$$

And

$$B(x) = b_0x^0 + b_1x^1 + \dots + b_{m-1}x^{m-1}, \quad m = |P|$$

with

$$b_i = \cos(\beta_i) - i \sin(\beta_i), \quad \beta_i = \frac{2\pi P[m-i-1]}{26}.$$

Notice that with the expression $P[m-i-1]$ explicitly reverses the pattern.

The $(m-1+i)$ th coefficients of the product of the two polynomials $C(x) = A(x) \cdot B(x)$ will tell us, if the pattern appears in the text at position i .

$$c_{m-1+i} = \sum_{j=0}^{m-1} a_{i+j} \cdot b_{m-1-j} = \sum_{j=0}^{m-1} (\cos(\alpha_{i+j}) + i \sin(\alpha_{i+j})) \cdot (\cos(\beta_j) - i \sin(\beta_j))$$

$$\text{with } \alpha_{i+j} = \frac{2\pi T[i+j]}{26} \text{ and } \beta_j = \frac{2\pi P[j]}{26}$$

If there is a match, then $T[i+j] = P[j]$, and therefore $\alpha_{i+j} = \beta_j$. This gives (using the Pythagorean trigonometric identity):

$$\begin{aligned} c_{m-1+i} &= \sum_{j=0}^{m-1} (\cos(\alpha_{i+j}) + i \sin(\alpha_{i+j})) \cdot (\cos(\alpha_{i+j}) - i \sin(\alpha_{i+j})) \\ &= \sum_{j=0}^{m-1} \cos^2(\alpha_{i+j}) + \sin^2(\alpha_{i+j}) = \sum_{j=0}^{m-1} 1 = m \end{aligned}$$

If there isn't a match, then at least a character is different, which leads that one of the products $a_{i+1} \cdot b_{m-1-j}$ is not equal to 1, which leads to the coefficient $c_{m-1+i} \neq m$.

String matching with wildcards

This is an extension of the previous problem. This time we allow that the pattern contains the wildcard character `*`, which can match every possible letter. E.g. the pattern `a*c` appears in the text `abccaaacc` at exactly three positions, at index 0, index 4 and index 5.

We create the exact same polynomials, except that we set $b_i = 0$ if $P[m-i-1] = *$. If x is the number of wildcards in P , then we will have a match of P in T at index i if $c_{m-1+i} = m - x$.

Practice problems

- [SPOJ - POLYMULT](#)
- [SPOJ - MAXMATCH](#)
- [SPOJ - ADAMATCH](#)
- [Codeforces - Yet Another String Matching Problem](#)
- [Codeforces - Lightsabers \(hard\)](#)
- [Codeforces - Running Competition](#)
- [Kattis - A+B Problem](#)
- [Kattis - K-Inversions](#)
- [Codeforces - Dasha and cyclic table](#)
- [CodeChef - Expected Number of Customers](#)
- [CodeChef - Power Sum](#)
- [Codeforces - Centroid Probabilities](#)

Contributors:

[Jakob Kogler](#) (95.48%)
 [Oleksandr Kulkov](#) (0.9%)
 [wiktorkuchta](#) (0.75%)
 [Deji Oyerinde](#) (0.6%)
 [Dalgerok](#) (0.3%)
 [Marco Rubin](#) (0.3%)
 [Tarun Khullar](#) (0.3%)
[Adithya Dsilva](#) (0.15%)
[Darshik N. Hirapara](#) (0.15%)
[Peter Zeman](#) (0.15%)
[Rathin Bhargava](#) (0.15%)
[Shorya](#) (0.15%)
[Sohieb Abdelrahman](#) (0.15%)
[Thomas Zeger](#) (0.15%)
[Tomasz Nowak](#) (0.15%)
[Yan Huang](#) (0.15%)

Original

Operations on polynomials and series

Problems in competitive programming, especially the ones involving enumeration some kind, are often solved by reducing the problem to computing something on polynomials and formal power series.

This includes concepts such as polynomial multiplication, interpolation, and more complicated ones, such as polynomial logarithms and exponents. In this article, a brief overview of such operations and common approaches to them is presented.

Basic Notion and facts

In this section, we focus more on the definitions and "intuitive" properties of various polynomial operations. The technical details of their implementation and complexities will be covered in later sections.

Polynomial multiplication

Definition

Univariate polynomial is an expression of form $A(x) = a_0 + a_1x + \dots + a_nx^n$.

The values $a_0, \dots, a_n$ are polynomial coefficients, typically taken from some set of numbers or number-like structures. In this article, we assume that the coefficients are taken from some [field](#), meaning that operations of addition, subtraction, multiplication and division are well-defined for them (except for division by 0) and they generally behave in a similar way to real numbers.

Typical example of such field is the field of remainders modulo prime number p .

For simplicity we will drop the term *univariate*, as this is the only kind of polynomials we consider in this article. We will also write A instead of $A(x)$ wherever possible, which will be understandable from the context. It is assumed that either $a_n \neq 0$ or $A(x) = 0$.

Definition

The **product** of two polynomials is defined by expanding it as an arithmetic expression:

$$A(x)B(x) = \left(\sum_{i=0}^n a_i x^i \right) \left(\sum_{j=0}^m b_j x^j \right) = \sum_{i,j} a_i b_j x^{i+j} = \sum_{k=0}^{n+m} c_k x^k = C(x).$$

The sequence $c_0, c_1, \dots, c_{n+m}$ of the coefficients of $C(x)$ is called the **convolution** of $a_0, \dots, a_n$ and $b_0, \dots, b_m$.

Definition

The **degree** of a polynomial A with $a_n \neq 0$ is defined as $\deg A = n$.

For consistency, degree of $A(x) = 0$ is defined as $\deg A = -\infty$.

In this notion, $\deg AB = \deg A + \deg B$ for any polynomials A and B .

Convolutions are the basis of solving many enumerative problems.

Example

You have n objects of the first kind and m objects of the second kind.

Objects of first kind are valued $a_1, \dots, a_n$, and objects of the second kind are valued $b_1, \dots, b_m$.

You pick a single object of the first kind and a single object of the second kind. How many ways are there to get the total value k ?

Solution

Consider the product $(x^{a_1} + \dots + x^{a_n})(x^{b_1} + \dots + x^{b_m})$. If you expand it, each monomial will correspond to the pair (a_i, b_j) and contribute to the coefficient near $x^{a_i+b_j}$. In other words, the answer is the coefficient near x^k in the product.

Example

You throw a 6-sided die n times and sum up the results from all throws. What is the probability of getting sum of k ?

Solution

The answer is the number of outcomes having the sum k , divided by the total number of outcomes, which is 6^n .

What is the number of outcomes having the sum k ? For $n = 1$, it may be represented by a polynomial $A(x) = x^1 + x^2 + \dots + x^6$.

For $n = 2$, using the same approach as in the example above, we conclude that it is represented by the polynomial $(x^1 + x^2 + \dots + x^6)^2$.

That being said, the answer to the problem is the k -th coefficient of $(x^1 + x^2 + \dots + x^6)^n$, divided by 6^n .

The coefficient near x^k in the polynomial $A(x)$ is denoted shortly as $[x^k]A$.

Formal power series

Definition

A **formal power series** is an infinite sum $A(x) = a_0 + a_1x + a_2x^2 + \dots$, considered regardless of its convergence properties.

In other words, when we consider e.g. a sum $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 2$, we imply that it *converges* to 2 when the number of summands approach infinity. However, formal series are only considered in terms of sequences that make them.

Definition

The **product** of formal power series $A(x)$ and $B(x)$, is also defined by expanding it as an arithmetic expression:

$$A(x)B(x) = \left(\sum_{i=0}^{\infty} a_i x^i \right) \left(\sum_{j=0}^{\infty} b_j x^j \right) = \sum_{i,j} a_i b_j x^{i+j} = \sum_{k=0}^{\infty} c_k x^k = C(x),$$

where the coefficients $c_0, c_1, \dots$ are defined as finite sums

$$c_k = \sum_{i=0}^k a_i b_{k-i}.$$

The sequence $c_0, c_1, \dots$ is also called a **convolution** of $a_0, a_1, \dots$ and $b_0, b_1, \dots$, generalizing the concept to infinite sequences.

Thus, polynomials may be considered formal power series, but with finite number of coefficients.

Formal power series play a crucial role in enumerative combinatorics, where they're studied as [generating functions](#) for various sequences. Detailed explanation of generating functions and the intuition behind them will, unfortunately, be out of scope for this article, therefore the curious reader is referenced e.g. [here](#) for details about their combinatorial meaning.

However, we will very briefly mention that if $A(x)$ and $B(x)$ are generating functions for sequences that enumerate some objects by number of "atoms" in them (e.g. trees by the number of vertices), then the product $A(x)B(x)$ enumerates objects that can be described as pairs of objects of kinds A and B , enumerated by the total number of "atoms" in the pair.

Example

Let $A(x) = \sum_{i=0}^{\infty} 2^i x^i$ enumerate packs of stones, each stone colored in one of 2 colors (so, there are 2^i such packs of size i) and $B(x) = \sum_{j=0}^{\infty} 3^j x^j$ enumerate packs of stones, each stone colored in one of 3 colors. Then $C(x) = A(x)B(x) = \sum_{k=0}^{\infty} c_k x^k$ would enumerate objects that may be described as "two packs of stones, first pack only of stones of type A , second pack only of stones of type B , with total number of stones being k " for c_k .

In a similar way, there is an intuitive meaning to some other functions over formal power series.

Long polynomial division

Similar to integers, it is possible to define long division on polynomials.

Definition

For any polynomials A and $B \neq 0$, one may represent A as

$$A = D \cdot B + R, \quad \deg R < \deg B,$$

where R is called the **remainder** of A modulo B and D is called the **quotient**.

Denoting $\deg A = n$ and $\deg B = m$, naive way to do it is to use long division, during which you multiply B by the monomial $\frac{a_n}{b_m} x^{n-m}$ and subtract it from A , until the degree of A is smaller than that of B . What remains of A in the end will be the remainder (hence the name), and the polynomials with which you multiplied B in the process, summed together, form the quotient.

Definition

If A and B have the same remainder modulo C , they're said to be **equivalent** modulo C , which is denoted as

$$A \equiv B \pmod{C}.$$

Polynomial long division is useful because of its many important properties:

- A is a multiple of B if and only if $A \equiv 0 \pmod{B}$.
- It implies that $A \equiv B \pmod{C}$ if and only if $A - B$ is a multiple of C .
- In particular, $A \equiv B \pmod{C \cdot D}$ implies $A \equiv B \pmod{C}$.
- For any linear polynomial $x - r$ it holds that $A(x) \equiv A(r) \pmod{x - r}$.
- It implies that A is a multiple of $x - r$ if and only if $A(r) = 0$.
- For modulo being x^k , it holds that $A \equiv a_0 + a_1 x + \dots + a_{k-1} x^{k-1} \pmod{x^k}$.

Note that long division can't be properly defined for formal power series. Instead, for any $A(x)$ such that $a_0 \neq 0$, it is possible to define an inverse formal power series $A^{-1}(x)$, such that $A(x)A^{-1}(x) = 1$. This fact, in turn, can be used to compute the result of long division for polynomials.

Basic implementation

[Here](#) you can find the basic implementation of polynomial algebra.

It supports all trivial operations and some other useful methods. The main class is `poly<T>` for polynomials with coefficients of type `T`.

All arithmetic operation `+`, `-`, `*`, `%` and `/` are supported, `%` and `/` standing for remainder and quotient in Euclidean division.

There is also the class `modular<m>` for performing arithmetic operations on remainders modulo a prime number `m`.

Other useful functions:

- `deriv()`: computes the derivative $P'(x)$ of $P(x)$.
- `integr()`: computes the indefinite integral $Q(x) = \int P(x)$ of $P(x)$ such that $Q(0) = 0$.
- `inv(size_t n)`: calculate the first n coefficients of $P^{-1}(x)$ in $O(n \log n)$.
- `log(size_t n)`: calculate the first n coefficients of $\ln P(x)$ in $O(n \log n)$.
- `exp(size_t n)`: calculate the first n coefficients of $\exp P(x)$ in $O(n \log n)$.
- `pow(size_t k, size_t n)`: calculate the first n coefficients for $P^k(x)$ in $O(n \log nk)$.
- `deg()`: returns the degree of $P(x)$.
- `lead()`: returns the coefficient of $x^{\deg P(x)}$.
- `resultant(poly<T> a, poly<T> b)`: computes the resultant of a and b in $O(|a| \cdot |b|)$.
- `bpow(T x, size_t n)`: computes x^n .
- `bpow(T x, size_t n, T m)`: computes $x^n \pmod{m}$.
- `chirpz(T z, size_t n)`: computes $P(1), P(z), P(z^2), \dots, P(z^{n-1})$ in $O(n \log n)$.
- `vector<T> eval(vector<T> x)`: evaluates $P(x_1), \dots, P(x_n)$ in $O(n \log^2 n)$.
- `poly<T> inter(vector<T> x, vector<T> y)`: interpolates a polynomial by a set of pairs $P(x_i) = y_i$ in $O(n \log^2 n)$.
- And some more, feel free to explore the code!

Arithmetic

Multiplication

The very core operation is the multiplication of two polynomials. That is, given the polynomials A and B :

$$A = a_0 + a_1x + \dots + a_nx^n$$

$$B = b_0 + b_1x + \dots + b_mx^m$$

You have to compute polynomial $C = A \cdot B$, which is defined as

$$C = \sum_{i=0}^n \sum_{j=0}^m a_i b_j x^{i+j} = c_0 + c_1x + \dots + c_{n+m}x^{n+m}.$$

It can be computed in $O(n \log n)$ via the [Fast Fourier transform](#) and almost all methods here will use it as subroutine.

Inverse series

If $A(0) \neq 0$ there always exists an infinite formal power series $A^{-1}(x) = q_0 + q_1x + q_2x^2 + \dots$ such that $A^{-1}A = 1$. It often proves useful to compute first k coefficients of A^{-1} (that is, to compute it modulo x^k). There are two major ways to calculate it.

Divide and conquer

This algorithm was mentioned in [Schönhage's article](#) and is inspired by [Graeffe's method](#). It is known that for $B(x) = A(x)A(-x)$ it holds that $B(x) = B(-x)$, that is, $B(x)$ is an even polynomial. It means that it only has non-zero coefficients with even numbers and can be represented as $B(x) = T(x^2)$. Thus, we can do the following transition:

$$A^{-1}(x) \equiv \frac{1}{A(x)} \equiv \frac{A(-x)}{A(x)A(-x)} \equiv \frac{A(-x)}{T(x^2)} \pmod{x^k}$$

Note that $T(x)$ can be computed with a single multiplication, after which we're only interested in the first half of coefficients of its inverse series. This effectively reduces the initial problem of computing $A^{-1} \pmod{x^k}$ to computing $T^{-1} \pmod{x^{\lceil k/2 \rceil}}$.

The complexity of this method can be estimated as

$$T(n) = T(n/2) + O(n \log n) = O(n \log n).$$

Sieveking-Kung algorithm

The generic process described here is known as Hensel lifting, as it follows from Hensel's lemma. We'll cover it in more detail further below, but for now let's focus on ad hoc solution. "Lifting" part here means that we start with the approximation $B_0 = q_0 = a_0^{-1}$, which is $A^{-1} \pmod{x}$ and then iteratively lift from $\pmod{x^a}$ to $\pmod{x^{2a}}$.

Let $B_k \equiv A^{-1} \pmod{x^a}$. The next approximation needs to follow the equation $AB_{k+1} \equiv 1 \pmod{x^{2a}}$ and may be represented as $B_{k+1} = B_k + x^a C$. From this follows the equation

$$A(B_k + x^a C) \equiv 1 \pmod{x^{2a}}.$$

Let $AB_k \equiv 1 + x^a D \pmod{x^{2a}}$, then the equation above implies

$$x^a(D + AC) \equiv 0 \pmod{x^{2a}} \implies D \equiv -AC \pmod{x^a} \implies C \equiv -B_k D \pmod{x^a}.$$

From this, one can obtain the final formula, which is

$$x^a C \equiv -B_k x^a D \equiv B_k(1 - AB_k) \pmod{x^{2a}} \implies \boxed{B_{k+1} \equiv B_k(2 - AB_k) \pmod{x^{2a}}}$$

Thus starting with $B_0 \equiv a_0^{-1} \pmod{x}$ we will compute the sequence B_k such that $AB_k \equiv 1 \pmod{x^{2^k}}$ with the complexity

$$T(n) = T(n/2) + O(n \log n) = O(n \log n).$$

The algorithm here might seem a bit more complicated than the first one, but it has a very solid and practical reasoning behind it, as well as a great generalization potential if looked from a different perspective, which would be explained further below.

Euclidean division

Consider two polynomials $A(x)$ and $B(x)$ of degrees n and m . As it was said earlier you can rewrite $A(x)$ as

$$A(x) = B(x)D(x) + R(x), \deg R < \deg B.$$

Let $n \geq m$, it would imply that $\deg D = n - m$ and the leading $n - m + 1$ coefficients of A don't influence R . It means that you can recover $D(x)$ from the largest $n - m + 1$ coefficients of $A(x)$ and $B(x)$ if you consider it as a system of equations.

The system of linear equations we're talking about can be written in the following form:

$$\begin{bmatrix} a_n \\ \vdots \\ a_{m+1} \\ a_m \end{bmatrix} = \begin{bmatrix} b_m & \cdots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots \\ \cdots & \cdots & b_m & 0 \\ \cdots & \cdots & b_{m-1} & b_m \end{bmatrix} \begin{bmatrix} d_{n-m} \\ \vdots \\ d_1 \\ d_0 \end{bmatrix}$$

From the looks of it, we can conclude that with the introduction of reversed polynomials

$$A^R(x) = x^n A(x^{-1}) = a_n + a_{n-1}x + \cdots + a_0x^n$$

$$B^R(x) = x^m B(x^{-1}) = b_m + b_{m-1}x + \cdots + b_0x^m$$

$$D^R(x) = x^{n-m} D(x^{-1}) = d_{n-m} + d_{n-m-1}x + \cdots + d_0x^{n-m}$$

the system may be rewritten as

$$A^R(x) \equiv B^R(x)D^R(x) \pmod{x^{n-m+1}}.$$

From this you can unambiguously recover all coefficients of $D(x)$:

$$D^R(x) \equiv A^R(x)(B^R(x))^{-1} \pmod{x^{n-m+1}}$$

And from this, in turn, you can recover $R(x)$ as $R(x) = A(x) - B(x)D(x)$.

Note that the matrix above is a so-called triangular [Toeplitz matrix](#) and, as we see here, solving system of linear equations with arbitrary Toeplitz matrix is, in fact, equivalent to polynomial inversion. Moreover, inverse matrix of it would also be triangular Toeplitz matrix and its entries, in terms used above, are the coefficients of $(B^R(x))^{-1} \pmod{x^{n-m+1}}$.

Calculating functions of polynomial

Newton's method

Let's generalize the Sieveking-Kung algorithm. Consider equation $F(P) = 0$ where $P(x)$ should be a polynomial and $F(x)$ is some polynomial-valued function defined as

$$F(x) = \sum_{i=0}^{\infty} \alpha_i (x - \beta)^i,$$

where β is some constant. It can be proven that if we introduce a new formal variable y , we can express $F(x)$ as

$$F(x) = F(y) + (x - y)F'(y) + (x - y)^2 G(x, y),$$

where $F'(x)$ is the derivative formal power series defined as

$$F'(x) = \sum_{i=0}^{\infty} (i+1)\alpha_{i+1}(x - \beta)^i,$$

and $G(x, y)$ is some formal power series of x and y . With this result we can find the solution iteratively.

Let $F(Q_k) \equiv 0 \pmod{x^a}$. We need to find $Q_{k+1} \equiv Q_k + x^a C \pmod{x^{2a}}$ such that $F(Q_{k+1}) \equiv 0 \pmod{x^{2a}}$.

Substituting $x = Q_{k+1}$ and $y = Q_k$ in the formula above, we get

$$F(Q_{k+1}) \equiv F(Q_k) + (Q_{k+1} - Q_k)F'(Q_k) + (Q_{k+1} - Q_k)^2 G(x, y) \pmod{x^{2a}}.$$

Since $Q_{k+1} - Q_k \equiv 0 \pmod{x^a}$, it also holds that $(Q_{k+1} - Q_k)^2 \equiv 0 \pmod{x^{2a}}$, thus

$$0 \equiv F(Q_{k+1}) \equiv F(Q_k) + (Q_{k+1} - Q_k)F'(Q_k) \pmod{x^{2a}}.$$

The last formula gives us the value of Q_{k+1} :

$$Q_{k+1} = Q_k - \frac{F(Q_k)}{F'(Q_k)} \pmod{x^{2a}}$$

Thus, knowing how to invert polynomials and how to compute $F(Q_k)$, we can find n coefficients of P with the complexity

$$T(n) = T(n/2) + f(n),$$

where $f(n)$ is the time needed to compute $F(Q_k)$ and $F'(Q_k)^{-1}$ which is usually $O(n \log n)$.

The iterative rule above is known in numerical analysis as [Newton's method](#).

Hensel's lemma

As was mentioned earlier, formally and generically this result is known as [Hensel's lemma](#) and it may in fact be used in even broader sense when we work with a series of nested rings. In this particular case we worked with a sequence of polynomial remainders modulo x , x^2 , x^3 and so on.

Another example where Hensel's lifting might be helpful are so-called [p-adic numbers](#) where we, in fact, work with the sequence of integer remainders modulo p , p^2 , p^3 and so on. For example, Newton's method can be used to find all possible [automorphic numbers](#) (numbers that end on itself when squared) with a given number base. The problem is left as an exercise to the reader. You might consider [this problem](#) to check if your solution works for 10-based numbers.

Logarithm

For the function $\ln P(x)$ it's known that:

$$(\ln P(x))' = \frac{P'(x)}{P(x)}$$

Thus we can calculate n coefficients of $\ln P(x)$ in $O(n \log n)$.

Inverse series

Turns out, we can get the formula for A^{-1} using Newton's method. For this we take the equation $A = Q^{-1}$, thus:

$$F(Q) = Q^{-1} - A$$

$$F'(Q) = -Q^{-2}$$

$$Q_{k+1} \equiv Q_k(2 - AQ_k) \pmod{x^{2^{k+1}}}$$

Exponent

Let's learn to calculate $e^{P(x)} = Q(x)$. It should hold that $\ln Q = P$, thus:

$$F(Q) = \ln Q - P$$

$$F'(Q) = Q^{-1}$$

$$Q_{k+1} \equiv Q_k(1 + P - \ln Q_k) \pmod{x^{2^{k+1}}}$$

k-th power

Now we need to calculate $P^k(x) = Q$. This may be done via the following formula:

$$Q = \exp[k \ln P(x)]$$

Note though, that you can calculate the logarithms and the exponents correctly only if you can find some initial Q_0 .

To find it, you should calculate the logarithm or the exponent of the constant coefficient of the polynomial.

But the only reasonable way to do it is if $P(0) = 1$ for $Q = \ln P$ so $Q(0) = 0$ and if $P(0) = 0$ for $Q = e^P$ so $Q(0) = 1$.

Thus you can use formula above only if $P(0) = 1$. Otherwise if $P(x) = \alpha x^t T(x)$ where $T(0) = 1$ you can write that:

$$P^k(x) = \alpha^k x^{kt} \exp[k \ln T(x)]$$

Note that you also can calculate some k-th root of a polynomial if you can calculate $\sqrt[k]{\alpha}$, for example for $\alpha = 1$.

Evaluation and Interpolation

Chirp-z Transform

For the particular case when you need to evaluate a polynomial in the points $x_r = z^{2r}$ you can do the following:

$$A(z^{2r}) = \sum_{k=0}^n a_k z^{2kr}$$

Let's substitute $2kr = r^2 + k^2 - (r - k)^2$. Then this sum rewrites as:

$$A(z^{2r}) = z^{r^2} \sum_{k=0}^n (a_k z^{k^2}) z^{-(r-k)^2}$$

Which is up to the factor z^{r^2} equal to the convolution of the sequences $u_k = a_k z^{k^2}$ and $v_k = z^{-k^2}$.

Note that u_k has indexes from 0 to n here and v_k has indexes from $-n$ to m where m is the maximum power of z which you need.

Now if you need to evaluate a polynomial in the points $x_r = z^{2r+1}$ you can reduce it to the previous task by the transformation $a_k \rightarrow a_k z^k$.

It gives us an $O(n \log n)$ algorithm when you need to compute values in powers of z, thus you may compute the DFT for non-powers of two.

Another observation is that $kr = \binom{k+r}{2} - \binom{k}{2} - \binom{r}{2}$. Then we have

$$A(z^r) = z^{-\binom{r}{2}} \sum_{k=0}^n (a_k z^{-\binom{k}{2}}) z^{\binom{k+r}{2}}$$

The coefficient of x^{n+r} of the product of the polynomials $A_0(x) = \sum_{k=0}^n a_{n-k} z^{-\binom{n-k}{2}} x^k$ and $A_1(x) = \sum_{k \geq 0} z^{\binom{k}{2}} x^k$ equals $z^{\binom{r}{2}} A(z^r)$. You can use the formula $z^{\binom{k+1}{2}} = z^{\binom{k}{2}+k}$ to calculate the coefficients of $A_0(x)$ and $A_1(x)$.

Multi-point Evaluation

Assume you need to calculate $A(x_1), \dots, A(x_n)$. As mentioned earlier, $A(x) \equiv A(x_i) \pmod{x - x_i}$. Thus you may do the following:

1. Compute a segment tree such that in the segment $[l, r]$ stands the product $P_{l,r}(x) = (x - x_l)(x - x_{l+1}) \dots (x - x_{r-1})$.
2. Starting with $l = 1$ and $r = n + 1$ at the root node. Let $m = \lfloor (l + r) / 2 \rfloor$. Let's move down to $[l, m]$ with the polynomial $A(x) \pmod{P_{l,m}(x)}$.
3. This will recursively compute $A(x_1), \dots, A(x_{m-1})$, now do the same for $[m, r]$ with $A(x) \pmod{P_{m,r}(x)}$.
4. Concatenate the results from the first and second recursive call and return them.

The whole procedure will run in $O(n \log^2 n)$.

Interpolation

There's a direct formula by Lagrange to interpolate a polynomial, given set of pairs (x_i, y_i) :

$$A(x) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Computing it directly is a hard thing but turns out, we may compute it in $O(n \log^2 n)$ with a divide and conquer approach:

Consider $P(x) = (x - x_1) \dots (x - x_n)$. To know the coefficients of the denominators in $A(x)$ we should compute products like:

$$P_i = \prod_{j \neq i} (x_i - x_j)$$

But if you consider the derivative $P'(x)$ you'll find out that $P'(x_i) = P_i$. Thus you can compute P_i 's via evaluation in $O(n \log^2 n)$.

Now consider the recursive algorithm done on same segment tree as in the multi-point evaluation. It starts in the leaves with the value $\frac{y_i}{P_i}$ in each leaf.

When we return from the recursion we should merge the results from the left and the right vertices as $A_{l,r} = A_{l,m} P_{m,r} + P_{l,m} A_{m,r}$.

In this way when you return back to the root you'll have exactly $A(x)$ in it. The total procedure also works in $O(n \log^2 n)$.

GCD and Resultants

Assume you're given polynomials $A(x) = a_0 + a_1 x + \dots + a_n x^n$ and $B(x) = b_0 + b_1 x + \dots + b_m x^m$.

Let $\lambda_0, \dots, \lambda_n$ be the roots of $A(x)$ and let $\mu_0, \dots, \mu_m$ be the roots of $B(x)$ counted with their multiplicities.

You want to know if $A(x)$ and $B(x)$ have any roots in common. There are two interconnected ways to do that.

Euclidean algorithm

Well, we already have an [article](#) about it. For an arbitrary domain you can write the Euclidean algorithm as easy as:

```
template<typename T>
T gcd(const T &a, const T &b) {
    return b == T(0) ? a : gcd(b, a % b);
}
```

It can be proven that for polynomials $A(x)$ and $B(x)$ it will work in $O(nm)$.

Resultant

Let's calculate the product $A(\mu_0) \cdots A(\mu_m)$. It will be equal to zero if and only if some μ_i is the root of $A(x)$.

For symmetry we can also multiply it with b_m^n and rewrite the whole product in the following form:

$$R(A, B) = b_m^n \prod_{j=0}^m A(\mu_j) = b_m^n a_m^n \prod_{i=0}^n \prod_{j=0}^m (\mu_j - \lambda_i) = (-1)^{mn} a_m^n \prod_{i=0}^n B(\lambda_i)$$

The value defined above is called the resultant of the polynomials $A(x)$ and $B(x)$. From the definition you may find the following properties:

1. $R(A, B) = (-1)^{nm} R(B, A)$.
2. $R(A, B) = a_m^n b_m^n$ when $n = 0$ or $m = 0$.
3. If $b_m = 1$ then $R(A - CB, B) = R(A, B)$ for an arbitrary polynomial $C(x)$ and $n, m \geq 1$.
4. From this follows $R(A, B) = b_m^{\deg(A) - \deg(A - CB)} R(A - CB, B)$ for arbitrary $A(x), B(x), C(x)$.

Miraculously it means that resultant of two polynomials is actually always from the same ring as their coefficients!

Also these properties allow us to calculate the resultant alongside the Euclidean algorithm, which works in $O(nm)$.

```
template<typename T>
T resultant(poly<T> a, poly<T> b) {
    if(b.is_zero()) {
        return 0;
    } else if(b.deg() == 0) {
        return bpow(b.lead(), a.deg());
    } else {
        int pw = a.deg();
        a %= b;
        pw -= a.deg();
        base mul = bpow(b.lead(), pw) * base((b.deg() & a.deg() & 1) ? -1 : 1);
        base ans = resultant(b, a);
        return ans * mul;
    }
}
```

Half-GCD algorithm

There is a way to calculate the GCD and resultants in $O(n \log^2 n)$.

The procedure to do so implements a 2×2 linear transform which maps a pair of polynomials $a(x), b(x)$ into another pair $c(x), d(x)$ such that $\deg d(x) \leq \frac{\deg a(x)}{2}$. If you're careful enough, you can compute the half-GCD of any pair of polynomials with at most 2 recursive calls to the polynomials which are at least 2 times smaller.

The specific details of the algorithm are somewhat tedious to explain, however you can find its implementation in the library, as `half_gcd` function.

After half-GCD is implemented, you can repeatedly apply it to polynomials until you're reduced to the pair of `gcd(a, b)` and 0.

Problems

- [CodeChef - RNG](#)
- [CodeForces - Basis Change](#)
- [CodeForces - Permutant](#)
- [CodeForces - Medium Hadron Collider](#)

Contributors:

[Jakob Kogler](#) (60.9%) [Oleksandr Kulkov](#) (36.05%) [Le Bao Hiep](#) (0.81%) [Egor Gorbachev](#) (0.61%) [hly1204](#) (0.61%) [Anton Älgmyr](#) (0.41%)
[Tanmay Sinha](#) (0.41%) [Егор Фефилов](#) (0.2%)

Original

Continued fractions

Continued fraction is a representation of a real number as a specific convergent sequence of rational numbers. They are useful in competitive programming because they are easy to compute and can be efficiently used to find the best possible rational approximation of the underlying real number (among all numbers whose denominator doesn't exceed a given value).

Besides that, continued fractions are closely related to Euclidean algorithm which makes them useful in a bunch of number-theoretical problems.

Continued fraction representation

Definition

Let $a_0, a_1, \dots, a_k \in \mathbb{Z}$ and $a_1, a_2, \dots, a_k \geq 1$. Then the expression

$$r = a_0 + \frac{1}{a_1 + \frac{1}{\dots + \frac{1}{a_k}}},$$

is called the **continued fraction representation** of the rational number r and is denoted shortly as $r = [a_0; a_1, a_2, \dots, a_k]$.

Example

Let $r = \frac{5}{3}$. There are two ways to represent it as a continued fraction:

$$r = [1; 1, 1, 1] = 1 + \frac{1}{1 + \frac{1}{1 + \frac{1}{1}}},$$

$$r = [1; 1, 2] = 1 + \frac{1}{1 + \frac{1}{2}}.$$

It can be proven that any rational number can be represented as a continued fraction in exactly 2 ways:

$$r = [a_0; a_1, \dots, a_k, 1] = [a_0; a_1, \dots, a_k + 1].$$

Moreover, the length k of such continued fraction is estimated as $k = O(\log \min(p, q))$ for $r = \frac{p}{q}$.

The reasoning behind this will be clear once we delve into the details of the continued fraction construction.

Definition

Let $a_0, a_1, a_2, \dots$ be an integer sequence such that $a_1, a_2, \dots \geq 1$. Let $r_k = [a_0; a_1, \dots, a_k]$. Then the expression

$$r = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \dots}} = \lim_{k \rightarrow \infty} r_k.$$

is called the **continued fraction representation** of the irrational number r and is denoted shortly as $r = [a_0; a_1, a_2, \dots]$.

Note that for $r = [a_0; a_1, \dots]$ and integer k , it holds that $r + k = [a_0 + k; a_1, \dots]$.

Another important observation is that $\frac{1}{r} = [0; a_0, a_1, \dots]$ when $a_0 > 0$ and $\frac{1}{r} = [a_1; a_2, \dots]$ when $a_0 = 0$.

Definition

In the definition above, rational numbers $r_0, r_1, r_2, \dots$ are called the **convergents** of r .

Correspondingly, individual $r_k = [a_0; a_1, \dots, a_k] = \frac{p_k}{q_k}$ is called the k -th **convergent** of r .

Example

Consider $r = [1; 1, 1, 1, \dots]$. It can be proven by induction that $r_k = \frac{F_{k+2}}{F_{k+1}}$, where F_k is the Fibonacci sequence defined as $F_0 = 0, F_1 = 1$ and $F_k = F_{k-1} + F_{k-2}$. From the Binet's formula, it is known that

$$r_k = \frac{\phi^{k+2} - \psi^{k+2}}{\phi^{k+1} - \psi^{k+1}},$$

where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ is the golden ratio and $\psi = \frac{1-\sqrt{5}}{2} = -\frac{1}{\phi} \approx -0.618$. Thus,

$$r = 1 + \frac{1}{1 + \frac{1}{1 + \dots}} = \lim_{k \rightarrow \infty} r_k = \phi = \frac{1 + \sqrt{5}}{2}.$$

Note that in this specific case, an alternative way to find r would be to solve the equation

$$r = 1 + \frac{1}{r} \implies r^2 = r + 1.$$

Definition

Let $r_k = [a_0; a_1, \dots, a_{k-1}, a_k]$. The numbers $[a_0; a_1, \dots, a_{k-1}, t]$ for $1 \leq t \leq a_k$ are called **semiconvergents**.

We will typically refer to (semi)convergents that are greater than r as **upper** (semi)convergents and to those that are less than r as **lower** (semi)convergents.

Definition

Complementary to convergents, we define the **complete quotients** as $s_k = [a_k; a_{k+1}, a_{k+2}, \dots]$.

Correspondingly, we will call an individual s_k the k -th complete quotient of r .

From the definitions above, one can conclude that $s_k \geq 1$ for $k \geq 1$.

Treating $[a_0; a_1, \dots, a_k]$ as a formal algebraic expression and allowing arbitrary real numbers instead of a_i , we obtain

$$r = [a_0; a_1, \dots, a_{k-1}, s_k].$$

In particular, $r = [s_0] = s_0$. On the other hand, we can express s_k as

$$s_k = [a_k; s_{k+1}] = a_k + \frac{1}{s_{k+1}},$$

meaning that we can compute $a_k = \lfloor s_k \rfloor$ and $s_{k+1} = (s_k - a_k)^{-1}$ from s_k .

The sequence $a_0, a_1, \dots$ is well-defined unless $s_k = a_k$ which only happens when r is a rational number.

Thus the continued fraction representation is uniquely defined for any irrational number r .

Implementation

In the code snippets we will mostly assume finite continued fractions.

From s_k , the transition to s_{k+1} looks like

$$s_k = \lfloor s_k \rfloor + \frac{1}{s_{k+1}}.$$

From this expression, the next complete quotient s_{k+1} is obtained as

$$s_{k+1} = (s_k - \lfloor s_k \rfloor)^{-1}.$$

For $s_k = \frac{p}{q}$ it means that

$$s_{k+1} = \left(\frac{p}{q} - \left\lfloor \frac{p}{q} \right\rfloor \right)^{-1} = \frac{q}{p - q \cdot \left\lfloor \frac{p}{q} \right\rfloor} = \frac{q}{p \bmod q}.$$

Thus, the computation of a continued fraction representation for $r = \frac{p}{q}$ follows the steps of the Euclidean algorithm for p and q .

From this also follows that $\gcd(p_k, q_k) = 1$ for $\frac{p_k}{q_k} = [a_0; a_1, \dots, a_k]$. Hence, convergents are always irreducible.

C++

```
auto fraction(int p, int q) {
    vector<int> a;
    while(q) {
        a.push_back(p / q);
        tie(p, q) = make_pair(q, p % q);
    }
    return a;
}
```

Python

```
def fraction(p, q):
    a = []
    while q:
        a.append(p // q)
        p, q = q, p % q
    return a
```

Key results

To provide some motivation for further study of continued fraction, we give some key facts now.

Recurrence

For the convergents $r_k = \frac{p_k}{q_k}$, the following recurrence stands, allowing their fast computation:

$$\frac{p_k}{q_k} = \frac{a_k p_{k-1} + p_{k-2}}{a_k q_{k-1} + q_{k-2}},$$

where $\frac{p_{-1}}{q_{-1}} = \frac{1}{0}$ and $\frac{p_{-2}}{q_{-2}} = \frac{0}{1}$.

Deviations

The deviation of $r_k = \frac{p_k}{q_k}$ from r can be generally estimated as

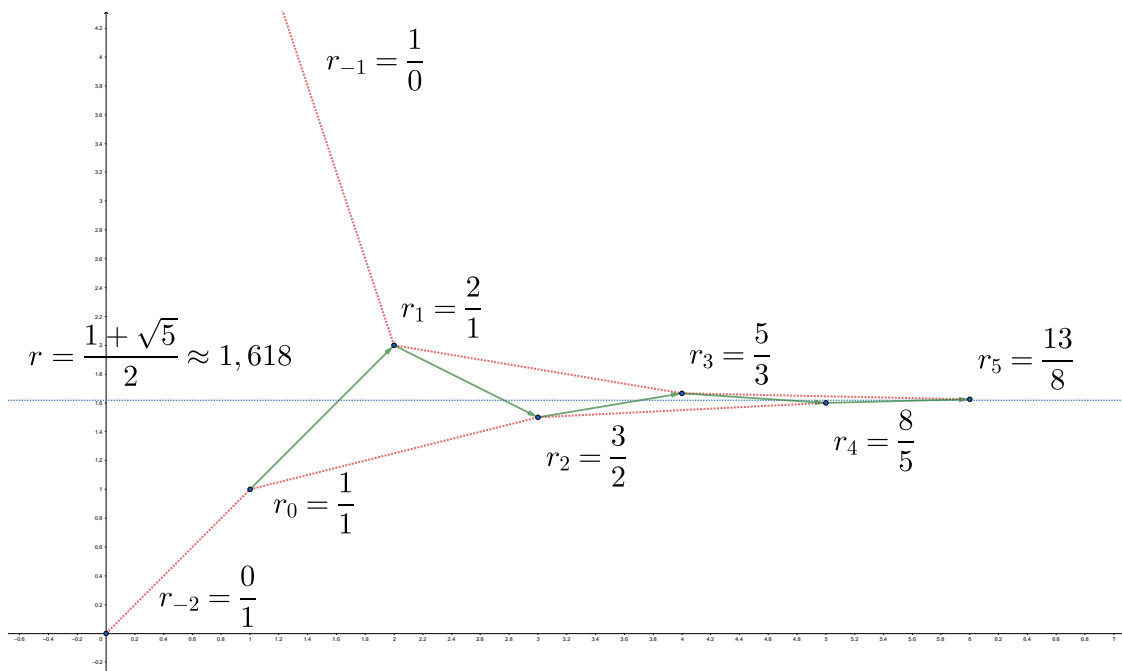
$$\frac{p_k}{q_k} - r \leq \frac{1}{q_k q_{k+1}} \leq \frac{1}{q_k^2}.$$

Multiplying both sides with q_k , we obtain alternate estimation:

$$|p_k - q_k r| \leq \frac{1}{q_{k+1}}.$$

From the recurrence above it follows that q_k grows at least as fast as Fibonacci numbers.

On the picture below you may see the visualization of how convergents r_k approach $r = \frac{1+\sqrt{5}}{2}$:



$r = \frac{1+\sqrt{5}}{2}$ is depicted by blue dotted line. Odd convergents approach it from above and even convergents approach it from below.

Lattice hulls

Consider convex hulls of points above and below the line $y = rx$.

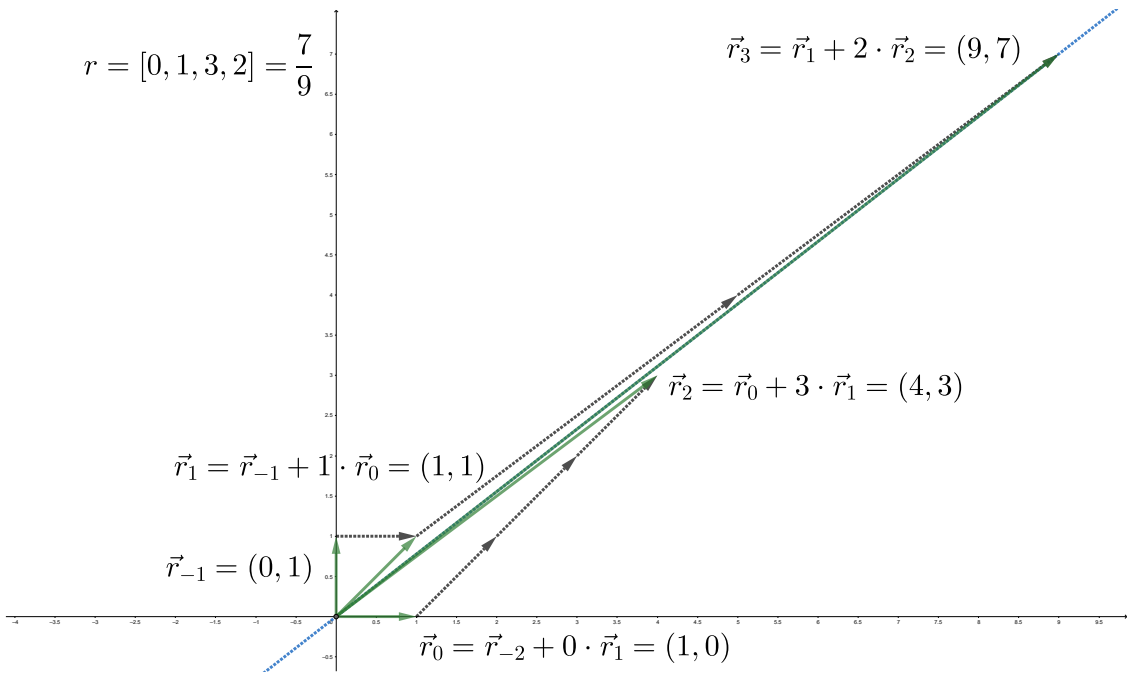
Odd convergents $(q_k; p_k)$ are the vertices of the upper hull, while the even convergents $(q_k; p_k)$ are the vertices of the bottom hull.

All integers vertices on the hulls are obtained as $(q; p)$ such that

$$\frac{p}{q} = \frac{tp_{k-1} + p_{k-2}}{tq_{k-1} + q_{k-2}}$$

for integer $0 \leq t \leq a_k$. In other words, the set of lattice points on the hulls corresponds to the set of semiconvergents.

On the picture below, you may see the convergents and semiconvergents (intermediate gray points) of $r = \frac{9}{7}$.



Best approximations

Let $\frac{p}{q}$ be the fraction to minimize $|r - \frac{p}{q}|$ subject to $q \leq x$ for some x .

Then $\frac{p}{q}$ is a semiconvergent of r .

The last fact allows to find the best rational approximations of r by checking its semiconvergents.

Below you will find the further explanation and a bit of intuition and interpretation for these facts.

Convergents

Let's take a closer look at the convergents that were defined earlier. For $r = [a_0, a_1, a_2, \dots]$, its convergents are

$$\begin{aligned} r_0 &= [a_0], \\ r_1 &= [a_0, a_1], \\ &\dots, \\ r_k &= [a_0, a_1, \dots, a_k]. \end{aligned}$$

Convergents are the core concept of continued fractions, so it is important to study their properties.

For the number r , its k -th convergent $r_k = \frac{p_k}{q_k}$ can be computed as

$$r_k = \frac{P_k(a_0, a_1, \dots, a_k)}{P_{k-1}(a_1, \dots, a_k)} = \frac{a_k p_{k-1} + p_{k-2}}{a_k q_{k-1} + q_{k-2}},$$

where $P_k(a_0, \dots, a_k)$ is the [continuant](#), a multivariate polynomial defined as

$$P_k(x_0, x_1, \dots, x_k) = \det \begin{bmatrix} x_k & 1 & 0 & \dots & 0 \\ -1 & x_{k-1} & 1 & \dots & 0 \\ & & & \ddots & \\ 0 & -1 & x_2 & \dots & \vdots \\ & \vdots & \vdots & \ddots & \ddots & 1 \\ 0 & 0 & \dots & -1 & x_0 \end{bmatrix}.$$

Thus, r_k is a weighted [mediant](#) of r_{k-1} and r_{k-2} .

For consistency, two additional convergents $r_{-1} = \frac{1}{0}$ and $r_{-2} = \frac{0}{1}$ are defined.

 Detailed explanation



The numerator and the denominator of r_k can be seen as multivariate polynomials of $a_0, a_1, \dots, a_k$:

$$r_k = \frac{P_k(a_0, a_1, \dots, a_k)}{Q_k(a_0, a_1, \dots, a_k)}.$$

From the definition of convergents,

$$r_k = a_0 + \frac{1}{[a_1; a_2, \dots, a_k]} = a_0 + \frac{Q_{k-1}(a_1, \dots, a_k)}{P_{k-1}(a_1, \dots, a_k)} = \frac{a_0 P_{k-1}(a_1, \dots, a_k) + Q_{k-1}(a_1, \dots, a_k)}{P_{k-1}(a_1, \dots, a_k)}.$$

From this follows $Q_k(a_0, \dots, a_k) = P_{k-1}(a_1, \dots, a_k)$. This yields the relation

$$P_k(a_0, \dots, a_k) = a_0 P_{k-1}(a_1, \dots, a_k) + P_{k-2}(a_2, \dots, a_k).$$

Initially, $r_0 = \frac{a_0}{1}$ and $r_1 = \frac{a_0 a_1 + 1}{a_1}$, thus

$$\begin{aligned} P_0(a_0) &= a_0, \\ P_1(a_0, a_1) &= a_0 a_1 + 1. \end{aligned}$$

For consistency, it is convenient to define $P_{-1} = 1$ and $P_{-2} = 0$ and formally say that $r_{-1} = \frac{1}{0}$ and $r_{-2} = \frac{0}{1}$.

From numerical analysis, it is known that the determinant of an arbitrary tridiagonal matrix

$$T_k = \det \begin{bmatrix} a_0 & b_0 & 0 & \dots & 0 \\ c_0 & a_1 & b_1 & \dots & 0 \\ 0 & c_1 & a_2 & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & c_{k-1} \\ 0 & 0 & \dots & b_{k-1} & a_k \end{bmatrix}$$

can be computed recursively as $T_k = a_k T_{k-1} - b_{k-1} c_{k-1} T_{k-2}$. Comparing it to P_k , we get a direct expression

$$P_k = \det \begin{bmatrix} x_k & 1 & 0 & \dots & 0 \\ -1 & x_{k-1} & 1 & \dots & 0 \\ 0 & -1 & x_2 & \dots & \vdots \\ \vdots & \vdots & \vdots & \ddots & 1 \\ 0 & 0 & \dots & -1 & x_0 \end{bmatrix}.$$

This polynomial is also known as [the continuant](#) due to its close relation with continued fraction. The continuant won't change if the sequence on the main diagonal is reversed. This yields an alternative formula to compute it:

$$P_k(a_0, \dots, a_k) = a_k P_{k-1}(a_0, \dots, a_{k-1}) + P_{k-2}(a_0, \dots, a_{k-2}).$$

Implementation

We will compute the convergents as a pair of sequences $p_{-2}, p_{-1}, p_0, p_1, \dots, p_k$ and $q_{-2}, q_{-1}, q_0, q_1, \dots, q_k$:

C++

```
auto convergents(vector<int> a) {
    vector<int> p = {0, 1};
```

```

vector<int> q = {1, 0};
for(auto it: a) {
    p.push_back(p[p.size() - 1] * it + p[p.size() - 2]);
    q.push_back(q[q.size() - 1] * it + q[q.size() - 2]);
}
return make_pair(p, q);
}

```

Python

```

def convergents(a):
    p = [0, 1]
    q = [1, 0]
    for it in a:
        p.append(p[-1]*it + p[-2])
        q.append(q[-1]*it + q[-2])
    return p, q

```

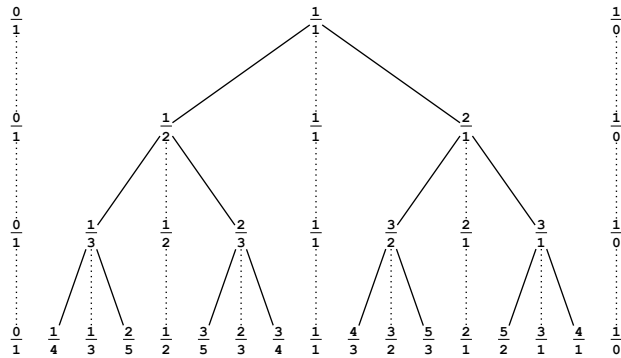
Trees of continued fractions

There are two major ways to unite all possible continued fractions into useful tree structures.

Stern-Brocot tree

The [Stern-Brocot tree](#) is a binary search tree that contains all distinct positive rational numbers.

The tree generally looks as follows:



The image by Aaron Rotenberg is licensed under CC BY-SA 3.0

Fractions $\frac{0}{1}$ and $\frac{1}{0}$ are "virtually" kept on the left and right sides of the tree correspondingly.

Then the fraction in a node is a mediant $\frac{a+c}{b+d}$ of two fractions $\frac{a}{b}$ and $\frac{c}{d}$ above it.

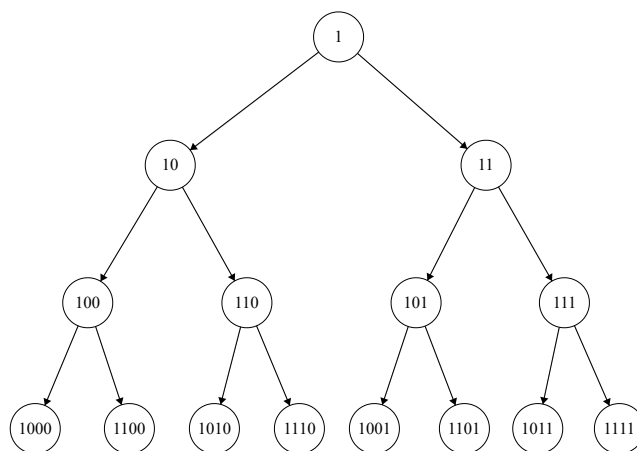
The recurrence $\frac{p_k}{q_k} = \frac{a_k p_{k-1} + p_{k-2}}{a_k q_{k-1} + q_{k-2}}$ means that the continued fraction representation encodes the path to $\frac{p_k}{q_k}$ in the tree. To find $[a_0; a_1, \dots, a_k, 1]$, one has to make a_0 moves to the right, a_1 moves to the left, a_2 moves to the right and so on up to a_k .

The parent of $[a_0; a_1, \dots, a_k, 1]$ then is the fraction obtained by taking one step back in the last used direction.

In other words, it is $[a_0; a_1, \dots, a_k - 1, 1]$ when $a_k > 1$ and $[a_0; a_1, \dots, a_{k-1}, 1]$ when $a_k = 1$.

Thus the children of $[a_0; a_1, \dots, a_k, 1]$ are $[a_0; a_1, \dots, a_k + 1, 1]$ and $[a_0; a_1, \dots, a_k, 1, 1]$.

Let's index the Stern-Brocot tree. The root vertex is assigned an index 1. Then for a vertex v , the index of its left child is assigned by changing the leading bit of v from 1 to 10 and for the right child, it's assigned by changing the leading bit from 1 to 11:



In this indexing, the continued fraction representation of a rational number specifies the [run-length encoding](#) of its binary index.

For $\frac{5}{2} = [2; 2] = [2; 1, 1]$, its index is 1011_2 and its run-length encoding, considering bits in the ascending order, is $[2; 1, 1]$.

Another example is $\frac{2}{5} = [0; 2, 2] = [0; 2, 1, 1]$, which has index 1100_2 and its run-length encoding is, indeed, $[0; 2, 2]$.

It is worth noting that the Stern-Brocot tree is, in fact, a [treap](#). That is, it is a binary search tree by $\frac{p}{q}$, but it is a heap by both p and q .

Comparing continued fractions

You're given $A = [a_0; a_1, \dots, a_n]$ and $B = [b_0; b_1, \dots, b_m]$. Which fraction is smaller?

 Solution


Assume for now that A and B are irrational and their continued fraction representations denote an infinite descent in the Stern-Brocot tree.

As we already mentioned, in this representation a_0 denotes the number of right turns in the descent, a_1 denotes the number of consequent left turns and so on. Therefore, when we compare a_k and b_k , if $a_k = b_k$ we should just move on to comparing a_{k+1} and b_{k+1} . Otherwise, if we're at right descents, we should check if $a_k < b_k$ and if we're at left descents, we should check if $a_k > b_k$ to tell whether $A < B$.

In other words, for irrational A and B it would be $A < B$ if and only if $(a_0, -a_1, a_2, -a_3, \dots) < (b_0, -b_1, b_2, -b_3, \dots)$ with lexicographical comparison.

Now, formally using ∞ as an element of continued fraction representation it is possible to emulate irrational numbers $A - \varepsilon$ and $A + \varepsilon$, that is, elements that are smaller (greater) than A , but greater (smaller) than any other real number. Specifically, for $A = [a_0; a_1, \dots, a_n]$, one of these two elements can be emulated as $[a_0; a_1, \dots, a_n, \infty]$ and the other can be emulated as $[a_0; a_1, \dots, a_n - 1, 1, \infty]$.

Which one corresponds to $A - \varepsilon$ and which one to $A + \varepsilon$ can be determined by the parity of n or by comparing them as irrational numbers.

Python

```
# check if a < b assuming that a[-1] = b[-1] = inf and a != b
def less(a, b):
    a = [(-1)**i*a[i] for i in range(len(a))]
    b = [(-1)**i*b[i] for i in range(len(b))]
    return a < b

# [a0; a1, ..., ak] -> [a0, a1, ..., ak-1, 1]
def expand(a):
    if a: # empty a = inf
        a[-1] -= 1
        a.append(1)
    return a

# return a-eps, a+eps
def pm_eps(a):
    b = expand(a.copy())
    a.append(float('inf'))
    b.append(float('inf'))
    return (a, b) if less(a, b) else (b, a)
```

 Best inner point

You're given $\frac{0}{1} \leq \frac{p_0}{q_0} < \frac{p_1}{q_1} \leq \frac{1}{0}$. Find the rational number $\frac{p}{q}$ such that $(q; p)$ is lexicographically smallest and $\frac{p_0}{q_0} < \frac{p}{q} < \frac{p_1}{q_1}$.

 Solution

In terms of the Stern-Brocot tree it means that we need to find the LCA of $\frac{p_0}{q_0}$ and $\frac{p_1}{q_1}$. Due to the connection between Stern-Brocot tree and continued fraction, this LCA would roughly correspond to the largest common prefix of continued fraction representations for $\frac{p_0}{q_0}$ and $\frac{p_1}{q_1}$.

So, if $\frac{p_0}{q_0} = [a_0; a_1, \dots, a_{k-1}, a_k, \dots]$ and $\frac{p_1}{q_1} = [a_0; a_1, \dots, a_{k-1}, b_k, \dots]$ are irrational numbers, the LCA is $[a_0; a_1, \dots, \min(a_k, b_k) + 1]$.

For rational r_0 and r_1 , one of them could be the LCA itself which would require us to casework it. To simplify the solution for rational r_0 and r_1 , it is possible to use continued fraction representation of $r_0 + \varepsilon$ and $r_1 - \varepsilon$ which was derived in the previous problem.

Python

```
# finds lexicographically smallest (q, p)
# such that p0/q0 < p/q < p1/q1
def middle(p0, q0, p1, q1):
    a0 = pm_eps(fraction(p0, q0))[1]
    a1 = pm_eps(fraction(p1, q1))[0]
    a = []
    for i in range(min(len(a0), len(a1))):
        a.append(min(a0[i], a1[i]))
        if a0[i] != a1[i]:
            break
    a[-1] += 1
    p, q = convergents(a)
    return p[-1], q[-1]
```

 GCJ 2019, Round 2 - New Elements: Part 2

You're given N positive integer pairs (C_i, J_i) . You need to find a positive integer pair (x, y) such that $C_i x + J_i y$ is a strictly increasing sequence.

Among such pairs, find the lexicographically minimum one.

Solution

Rephrasing the statement, $A_i x + B_i y$ must be positive for all i , where $A_i = C_i - C_{i-1}$ and $B_i = J_i - J_{i-1}$.

Among such equations we have four significant groups for $A_i x + B_i y > 0$:

1. $A_i, B_i > 0$ can be ignored since we're looking for $x, y > 0$.
2. $A_i, B_i \leq 0$ would provide "IMPOSSIBLE" as an answer.
3. $A_i > 0, B_i \leq 0$. Such constraints are equivalent to $\frac{y}{x} < \frac{-A_i}{-B_i}$.
4. $A_i \leq 0, B_i > 0$. Such constraints are equivalent to $\frac{y}{x} > \frac{-A_i}{B_i}$.

Let $\frac{p_0}{q_0}$ be the largest $\frac{-A_i}{B_i}$ from the fourth group and $\frac{p_1}{q_1}$ be the smallest $\frac{-A_i}{-B_i}$ from the third group.

The problem is now, given $\frac{p_0}{q_0} < \frac{p_1}{q_1}$, find a fraction $\frac{p}{q}$ such that $(q; p)$ is lexicographically smallest and $\frac{p_0}{q_0} < \frac{p}{q} < \frac{p_1}{q_1}$.

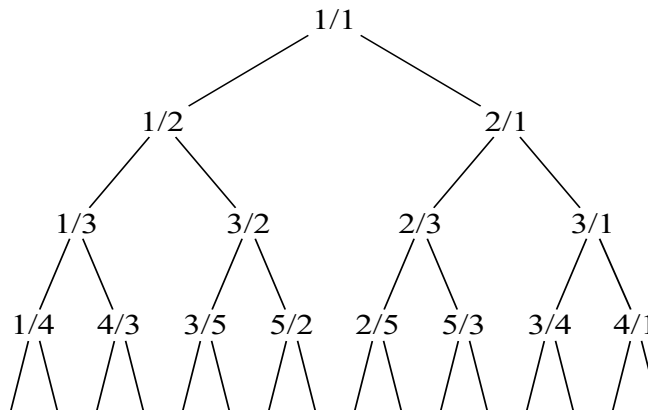
Python

```
def solve():
    n = int(input())
    C = [0] * n
    J = [0] * n
    # p0/q0 < y/x < p1/q1
    p0, q0 = 0, 1
    p1, q1 = 1, 0
    fail = False
    for i in range(n):
        C[i], J[i] = map(int, input().split())
        if i > 0:
            A = C[i] - C[i-1]
            B = J[i] - J[i-1]
            if A <= 0 and B <= 0:
                fail = True
            elif B > 0 and A < 0: # y/x > (-A)/B if B > 0
                if (-A)*q0 > p0*B:
                    p0, q0 = -A, B
            elif B < 0 and A > 0: # y/x < A/(-B) if B < 0
                if A*q1 < p1*(-B):
                    p1, q1 = A, -B
    if p0*q1 >= p1*q0 or fail:
        return 'IMPOSSIBLE'
    p, q = middle(p0, q0, p1, q1)
    return str(q) + ' ' + str(p)
```

Calkin-Wilf tree

A somewhat simpler way to organize continued fractions in a binary tree is the [Calkin-Wilf tree](#).

The tree generally looks like this:



The image by Olli Niemitalo, Proz is licensed under CC0 1.0

In the root of the tree, the number $\frac{1}{1}$ is located. Then, for the vertex with a number $\frac{p}{q}$, its children are $\frac{p}{p+q}$ and $\frac{p+q}{q}$.

Unlike the Stern-Brocot tree, the Calkin-Wilf tree is not a *binary search* tree, so it can't be used to perform rational binary search.

In the Calkin-Wilf tree, the direct parent of a fraction $\frac{p}{q}$ is $\frac{p-q}{q-p}$ when $p > q$ and $\frac{p}{q-p}$ otherwise.

For the Stern-Brocot tree, we used the recurrence for convergents. To draw the connection between the continued fraction and the Calkin-Wilf tree, we should recall the recurrence for complete quotients. If $s_k = \frac{p}{q}$, then $s_{k+1} = \frac{q}{p \bmod q} = \frac{q}{p - \lfloor p/q \rfloor q}$.

On the other hand, if we repeatedly go from $s_k = \frac{p}{q}$ to its parent in the Calkin-Wilf tree when $p > q$, we will end up in $\frac{p \bmod q}{q} = \frac{1}{s_{k+1}}$. If we continue doing so, we will end up in s_{k+2} , then $\frac{1}{s_{k+3}}$ and so on. From this we can deduce that:

1. When $a_0 > 0$, the direct parent of $[a_0; a_1, \dots, a_k]$ in the Calkin-Wilf tree is $\frac{p-q}{q} = [a_0 - 1; a_1, \dots, a_k]$.
2. When $a_0 = 0$ and $a_1 > 1$, its direct parent is $\frac{p}{q-p} = [0; a_1 - 1, a_2, \dots, a_k]$.
3. And when $a_0 = 0$ and $a_1 = 1$, its direct parent is $\frac{p}{q-p} = [a_2; a_3, \dots, a_k]$.

Correspondingly, children of $\frac{p}{q} = [a_0; a_1, \dots, a_k]$ are

1. $\frac{p+q}{q} = 1 + \frac{p}{q}$, which is $[a_0 + 1; a_1, \dots, a_k]$,
2. $\frac{p}{p+q} = \frac{1}{1 + \frac{q}{p}}$, which is $[0, 1, a_0, a_1, \dots, a_k]$ for $a_0 > 0$ and $[0, a_1 + 1, a_2, \dots, a_k]$ for $a_0 = 0$.

Noteworthy, if we enumerate vertices of the Calkin-Wilf tree in the breadth-first search order (that is, the root has a number 1, and the children of the vertex v have indices $2v$ and $2v + 1$ correspondingly), the index of the rational number in the Calkin-Wilf tree would be the same as in the Stern-Brocot tree.

Thus, numbers on the same levels of the Stern-Brocot tree and the Calkin-Wilf tree are the same, but their ordering differs through the [bit-reversal permutation](#).

Convergence

For the number r and its k -th convergent $r_k = \frac{p_k}{q_k}$ the following formula stands:

$$r_k = a_0 + \sum_{i=1}^k \frac{(-1)^{i-1}}{q_i q_{i-1}}.$$

In particular, it means that

$$r_k - r_{k-1} = \frac{(-1)^{k-1}}{q_k q_{k-1}}$$

and

$$p_k q_{k-1} - p_{k-1} q_k = (-1)^{k-1}.$$

From this we can conclude that

$$r - \frac{p_k}{q_k} \leq \frac{1}{q_{k+1} q_k} \leq \frac{1}{q_k^2}.$$

The latter inequality is due to the fact that r_k and r_{k+1} are generally located on different sides of r , thus

$$|r - r_k| = |r_k - r_{k+1}| - |r - r_{k+1}| \leq |r_k - r_{k+1}|.$$

Detailed explanation

To estimate $|r - r_k|$, we start by estimating the difference between adjacent convergents. By definition,

$$\frac{p_k}{q_k} - \frac{p_{k-1}}{q_{k-1}} = \frac{p_k q_{k-1} - p_{k-1} q_k}{q_k q_{k-1}}.$$

Replacing p_k and q_k in the numerator with their recurrences, we get

$$\begin{aligned} p_k q_{k-1} - p_{k-1} q_k &= (a_k p_{k-1} + p_{k-2}) q_{k-1} - p_{k-1} (a_k q_{k-1} + q_{k-2}) \\ &= p_{k-2} q_{k-1} - p_{k-1} q_{k-2}, \end{aligned}$$

thus the numerator of $r_k - r_{k-1}$ is always the negated numerator of $r_{k-1} - r_{k-2}$. It, in turn, equals to 1 for

$$r_1 - r_0 = \left(a_0 + \frac{1}{a_1}\right) - a_0 = \frac{1}{a_1},$$

thus

$$r_k - r_{k-1} = \frac{(-1)^{k-1}}{q_k q_{k-1}}.$$

This yields an alternative representation of r_k as a partial sum of infinite series:

$$r_k = (r_k - r_{k-1}) + \dots + (r_1 - r_0) + r_0 = a_0 + \sum_{i=1}^k \frac{(-1)^{i-1}}{q_i q_{i-1}}.$$

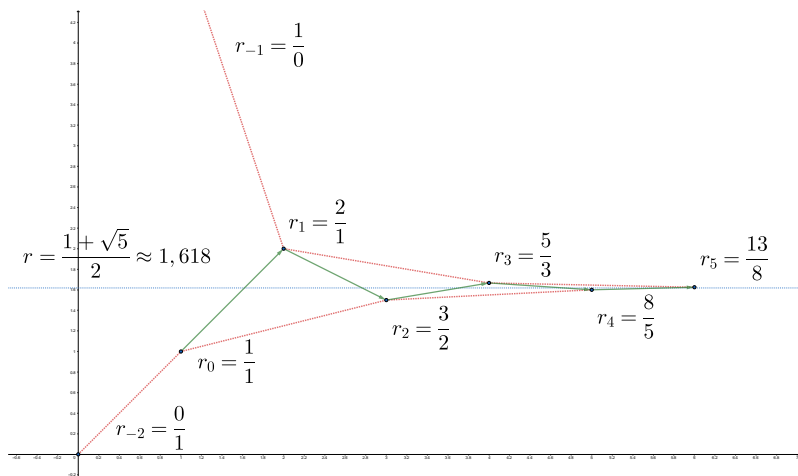
From the recurrent relation it follows that q_k monotonously increases at least as fast as Fibonacci numbers, thus

$$r = \lim_{k \rightarrow \infty} r_k = a_0 + \sum_{i=1}^{\infty} \frac{(-1)^{i-1}}{q_i q_{i-1}}$$

is always well-defined, as the underlying series always converge. Noteworthy, the residual series

$$r - r_k = \sum_{i=k+1}^{\infty} \frac{(-1)^{i-1}}{q_i q_{i-1}}$$

has the same sign as $(-1)^k$ due to how fast $q_i q_{i-1}$ decreases. Hence even-indexed r_k approach r from below while odd-indexed r_k approach it from above:



Convergents of $r = \phi = \frac{1+\sqrt{5}}{2} = [1; 1, 1, \dots]$ and their distance from r .

From this picture we can see that

$$|r - r_k| = |r_k - r_{k+1}| - |r - r_{k+1}| \leq |r_k - r_{k+1}|,$$

thus the distance between r and r_k is never larger than the distance between r_k and r_{k+1} :

$$r - \frac{p_k}{q_k} \leq \frac{1}{q_k q_{k+1}} \leq \frac{1}{q_k^2}.$$

Extended Euclidean?

You're given $A, B, C \in \mathbb{Z}$. Find $x, y \in \mathbb{Z}$ such that $Ax + By = C$.

Solution

Although this problem is typically solved with the [extended Euclidean algorithm](#), there is a simple and straightforward solution with continued fractions.

Let $\frac{A}{B} = [a_0; a_1, \dots, a_k]$. It was proved above that $p_k q_{k-1} - p_{k-1} q_k = (-1)^{k-1}$. Substituting p_k and q_k with A and B , we get

$$Aq_{k-1} - Bp_{k-1} = (-1)^{k-1}g,$$

where $g = \gcd(A, B)$. If C is divisible by g , then the solution is $x = (-1)^{k-1} \frac{C}{g} q_{k-1}$ and $y = (-1)^k \frac{C}{g} p_{k-1}$.

Python

```
# return (x, y) such that Ax+By=C
# assumes that such (x, y) exists
def dio(A, B, C):
    p, q = convergents(fraction(A, B))
    C //= A // p[-1] # divide by gcd(A, B)
    t = (-1) if len(p) % 2 else 1
    return t*C*q[-2], -t*C*p[-2]
```

Linear fractional transformations

Another important concept for continued fractions are the so-called [linear fractional transformations](#).

Definition

A **linear fractional transformation** is a function $f: \mathbb{R} \rightarrow \mathbb{R}$ such that $f(x) = \frac{ax+b}{cx+d}$ for some $a, b, c, d \in \mathbb{R}$.

A composition $(L_0 \circ L_1)(x) = L_0(L_1(x))$ of linear fractional transforms $L_0(x) = \frac{a_0x+b_0}{c_0x+d_0}$ and $L_1(x) = \frac{a_1x+b_1}{c_1x+d_1}$ is itself a linear fractional transform:

$$\frac{a_0 \frac{a_1x+b_1}{c_1x+d_1} + b_0}{c_0 \frac{a_1x+b_1}{c_1x+d_1} + d_0} = \frac{a_0(a_1x+b_1) + b_0(c_1x+d_1)}{c_0(a_1x+b_1) + d_0(c_1x+d_1)} = \frac{(a_0a_1 + b_0c_1)x + (a_0b_1 + b_0d_1)}{(c_0a_1 + d_0c_1)x + (c_0b_1 + d_0d_1)}.$$

Inverse of a linear fractional transform, is also a linear fractional transform:

$$y = \frac{ax + b}{cx + d} \iff y(cx + d) = ax + b \iff x = -\frac{dy - b}{cy - a}.$$

DMOPC '19 Contest 7 P4 - Bob and Continued Fractions

You're given an array of positive integers $a_1, \dots, a_n$. You need to answer m queries. Each query is to compute $[a_1; a_{1+1}, \dots, a_r]$.

Solution

We can solve this problem with the segment tree if we're able to concatenate continued fractions.

It's generally true that $[a_0; a_1, \dots, a_k, b_0, b_1, \dots, b_k] = [a_0; a_1, \dots, a_k, [b_1; b_2, \dots, b_k]]$.

Let's denote $L_k(x) = [a_k; x] = a_k + \frac{1}{x} = \frac{a_k x + 1}{1 \cdot x + 0}$. Note that $L_k(\infty) = a_k$. In this notion, it holds that

$$[a_0; a_1, \dots, a_k, x] = [a_0; a_1; [\dots; [a_k; x]]] = (L_0 \circ L_1 \circ \dots \circ L_k)(x) = \frac{p_k x + p_{k-1}}{q_k x + q_{k-1}}.$$

Thus, the problem boils down to the computation of

$$(L_1 \circ L_{1+1} \circ \dots \circ L_r)(\infty).$$

Composition of transforms is associative, so it's possible to compute in each node of a segment tree the composition of transforms in its subtree.

Linear fractional transformation of a continued fraction

Let $L(x) = \frac{ax+b}{cx+d}$. Compute the continued fraction representation $[b_0; b_1, \dots, b_m]$ of $L(A)$ for $A = [a_0; a_1, \dots, a_n]$.

This allows to compute $A + \frac{p}{q} = \frac{aA+p}{q}$ and $A \cdot \frac{p}{q} = \frac{pA}{q}$ for any $\frac{p}{q}$.

Solution

As we noted above, $[a_0; a_1, \dots, a_k] = (L_{a_0} \circ L_{a_1} \circ \dots \circ L_{a_k})(\infty)$, hence $L([a_0; a_1, \dots, a_k]) = (L \circ L_{a_0} \circ L_{a_1} \circ \dots \circ L_{a_k})(\infty)$.

Hence, by consequentially adding L_{a_0}, L_{a_1} and so on we would be able to compute

$$(L \circ L_{a_0} \circ \dots \circ L_{a_k})(x) = L\left(\frac{p_k x + p_{k-1}}{q_k x + q_{k-1}}\right) = \frac{a_k x + b_k}{c_k x + d_k}.$$

Since $L(x)$ is invertible, it is also monotonous in x . Therefore, for any $x \geq 0$ it holds that $L(\frac{p_k x + p_{k-1}}{q_k x + q_{k-1}})$ is between $L(\frac{p_k}{q_k}) = \frac{a_k}{c_k}$ and

$$L(\frac{p_{k-1}}{q_{k-1}}) = \frac{b_k}{d_k}.$$

Moreover, for $x = [a_{k+1}; \dots, a_n]$ it is equal to $L(A)$. Hence, $b_0 = \lfloor L(A) \rfloor$ is between $\lfloor L(\frac{p_k}{q_k}) \rfloor$ and $\lfloor L(\frac{p_{k-1}}{q_{k-1}}) \rfloor$. When they're equal, they're also equal to b_0 .

Note that $L(A) = (L_{b_0} \circ L_{b_1} \circ \dots \circ L_{b_m})(\infty)$. Knowing b_0 , we can compose $L_{b_0}^{-1}$ with the current transform and continue adding $L_{a_{k+1}}, L_{a_{k+2}}$ and so on, looking for new floors to agree, from which we would be able to deduce b_1 and so on until we recover all values of $[b_0; b_1, \dots, b_m]$.

Continued fraction arithmetics

Let $A = [a_0; a_1, \dots, a_n]$ and $B = [b_0; b_1, \dots, b_m]$. Compute the continued fraction representations of $A + B$ and $A \cdot B$.

Solution

Idea here is similar to the previous problem, but instead of $L(x) = \frac{ax+b}{cx+d}$ you should consider bilinear fractional transform

$$L(x, y) = \frac{axy+bx+cy+d}{exy+fx+gy+h}.$$

Rather than $L(x) \mapsto L(L_{a_k}(x))$ you would change your current transform as $L(x, y) \mapsto L(L_{a_k}(x), y)$ or $L(x, y) \mapsto L(x, L_{b_k}(y))$.

Then, you check if $\lfloor \frac{a}{e} \rfloor = \lfloor \frac{b}{f} \rfloor = \lfloor \frac{c}{g} \rfloor = \lfloor \frac{d}{h} \rfloor$ and if they all agree, you use this value as c_k in the resulting fraction and change the transform as

$$L(x, y) \mapsto \frac{1}{L(x, y) - c_k}.$$

Definition

A continued fraction $x = [a_0; a_1, \dots]$ is said to be **periodic** if $x = [a_0; a_1, \dots, a_k, x]$ for some k .

A continued fraction $x = [a_0; a_1, \dots]$ is said to be **eventually periodic** if $x = [a_0; a_1, \dots, a_k, y]$, where y is periodic.

For $x = [1; 1, 1, \dots]$ it holds that $x = 1 + \frac{1}{x}$, thus $x^2 = x + 1$. There is a generic connection between periodic continued fractions and quadratic equations. Consider the following equation:

$$x = [a_0; a_1, \dots, a_k, x].$$

On one hand, this equation means that the continued fraction representation of x is periodic with the period $k + 1$.

On the other hand, using the formula for convergents, this equation means that

$$x = \frac{p_k x + p_{k-1}}{q_k x + q_{k-1}}.$$

That is, x is a linear fractional transformation of itself. It follows from the equation that x is a root of the second degree equation:

$$q_k x^2 + (q_{k-1} - p_k)x - p_{k-1} = 0.$$

Similar reasoning stands for continued fractions that are eventually periodic, that is $x = [a_0; a_1, \dots, a_k, y]$ for $y = [b_0; b_1, \dots, b_k, y]$. Indeed, from first equation we derive that $x = L_0(y)$ and from second equation that $y = L_1(y)$, where L_0 and L_1 are linear fractional transformations. Therefore,

$$x = (L_0 \circ L_1)(y) = (L_0 \circ L_1 \circ L_0^{-1})(x).$$

One can further prove (and it was first done by Lagrange) that for arbitrary quadratic equation $ax^2 + bx + c = 0$ with integer coefficients, its solution x is an eventually periodic continued fraction.

Quadratic irrationality

Find the continued fraction of $\alpha = \frac{x+y\sqrt{n}}{z}$ where $x, y, z, n \in \mathbb{Z}$ and $n > 0$ is not a perfect square.

 Solution

For the k -th complete quotient s_k of the number it generally holds that

$$\alpha = [a_0; a_1, \dots, a_{k-1}, s_k] = \frac{s_k p_{k-1} + p_{k-2}}{s_k q_{k-1} + q_{k-2}}.$$

Therefore,

$$s_k = -\frac{\alpha q_{k-1} - p_{k-1}}{\alpha q_k - p_k} = -\frac{q_{k-1} y \sqrt{n} + (x q_{k-1} - z p_{k-1})}{q_k y \sqrt{n} + (x q_k - z p_k)}.$$

Multiplying the numerator and denominator by $(x q_k - z p_k) - q_k y \sqrt{n}$, we'll get rid of $\sqrt{n}$ in the denominator, thus the complete quotients are of form

$$s_k = \frac{x_k + y_k \sqrt{n}}{z_k}.$$

Let's find s_{k+1} , assuming that s_k is known.

First of all, $a_k = \lfloor s_k \rfloor = \lfloor \frac{x_k + y_k \sqrt{n}}{z_k} \rfloor$. Then,

$$s_{k+1} = \frac{1}{s_k - a_k} = \frac{z_k}{(x_k - z_k a_k) + y_k \sqrt{n}} = \frac{z_k (x_k - y_k a_k) - y_k z_k \sqrt{n}}{(x_k - y_k a_k)^2 - y_k^2 n}.$$

Thus, if we denote $t_k = x_k - y_k a_k$, it will hold that

$$\begin{aligned} x_{k+1} &= z_k t_k, \\ y_{k+1} &= -y_k z_k, \\ z_{k+1} &= t_k^2 - y_k^2 n. \end{aligned}$$

Nice thing about such representation is that if we reduce $x_{k+1}, y_{k+1}, z_{k+1}$ by their greatest common divisor, the result would be unique. Therefore, we may use it to check whether the current state has already been repeated and also to check where was the previous index that had this state.

Below is the code to compute the continued fraction representation for $\alpha = \sqrt{n}$:

Python

```
# compute the continued fraction of sqrt(n)
def sqrt(n):
    n0 = math.floor(math.sqrt(n))
    x, y, z = 1, 0, 1
    a = []
    def step(x, y, z):
        a.append((x * n0 + y) // z)
        t = y - a[-1] * z
        x, y, z = -z * x, z * t, t ** 2 - n * x ** 2
        g = math.gcd(x, math.gcd(y, z))
        return x // g, y // g, z // g

    used = dict()
    for i in range(n):
        used[x, y, z] = i
        x, y, z = step(x, y, z)
        if (x, y, z) in used:
            return a
```

Using the same `step` function but different initial x, y and z it is possible to compute it for arbitrary $\frac{x + y \sqrt{n}}{z}$.

Tavrida NU Akai Contest - Continued Fraction

You're given x and k , x is not a perfect square. Let $\sqrt{x} = [a_0; a_1, \dots]$, find $\frac{p_k}{q_k} = [a_0; a_1, \dots, a_k]$ for $0 \leq k \leq 10^9$.

Solution

After computing the period of $\sqrt{x}$, it is possible to compute a_k using binary exponentiation on the linear fractional transformation induced by the continued fraction representation. To find the resulting transformation, you compress the period of size T into a single transformation and repeat it $\lfloor \frac{k-1}{T} \rfloor$ times, after which you manually combine it with the remaining transformations.

Python

```
x, k = map(int, input().split())

mod = 10**9+7

# compose (A[0]*x + A[1]) / (A[2]*x + A[3]) and (B[0]*x + B[1]) / (B[2]*x + B[3])
def combine(A, B):
    return [t % mod for t in [A[0]*B[0]+A[1]*B[2], A[0]*B[1]+A[1]*B[3], A[2]*B[0]+A[3]*B[2], A[2]*B[1]+A[3]*B[3]]]

A = [1, 0, 0, 1] # (x + 0) / (0*x + 1) = x

a = sqrt(x)

T = len(a) - 1 # period of a

# apply ak + 1/x = (ak*x+1)/(1x+0) to (Ax + B) / (Cx + D)
for i in reversed(range(1, len(a))):
    A = combine([a[i], 1, 1, 0], A)

def bpow(A, n):
    return [1, 0, 0, 1] if not n else combine(A, bpow(A, n-1)) if n % 2 else bpow(combine(A, A), n // 2)

C = (0, 1, 0, 0) # = 1 / 0
while k % T:
    i = k % T
    C = combine([a[i], 1, 1, 0], C)
    k -= 1

C = combine(bpow(A, k // T), C)
C = combine([a[0], 1, 1, 0], C)
print(str(C[1]) + '/' + str(C[3]))
```

Geometric interpretation

Let $r_k = (q_k; p_k)$ for the convergent $r_k = \frac{p_k}{q_k}$. Then, the following recurrence holds:

$$r_k = a_k r_{k-1} + r_{k-2}.$$

Let $r = (1; r)$. Then, each vector $(x; y)$ corresponds to the number that is equal to its slope coefficient $\frac{y}{x}$.

With the notion of **pseudoscalar product** $(x_1; y_1) \times (x_2; y_2) = x_1 y_2 - x_2 y_1$, it can be shown (see the explanation below) that

$$s_k = -\frac{r_{k-2} \times r}{r_{k-1} \times r} = \frac{r_{k-2} \times r}{r_{k-1} \times r}.$$

The last equation is due to the fact that r_{k-1} and r_{k-2} lie on the different sides of r , thus pseudoscalar products of r_{k-1} and r_{k-2} with r have distinct signs. With $a_k = \lfloor s_k \rfloor$ in mind, formula for r_k now looks like

$$r_k = r_{k-2} + \left\lfloor \frac{r \times r_{k-2}}{r \times r_{k-1}} \right\rfloor r_{k-1}.$$

Note that $r_{k-1} \times r = (q; p) \times (1; r) = qr - p$, thus

$$a_k = \left\lfloor \frac{q_{k-1}r - p_{k-1}}{q_{k-2}r - p_{k-2}} \right\rfloor.$$

Explanation

As we have already noted, $a_k = \lfloor s_k \rfloor$, where $s_k = [a_k; a_{k+1}, a_{k+2}, \dots]$. On the other hand, from the convergent recurrence we derive that

$$r = [a_0; a_1, \dots, a_{k-1}, s_k] = \frac{s_k p_{k-1} + p_{k-2}}{s_k q_{k-1} + q_{k-2}}.$$

In vector form, it rewrites as

$$r \rightarrow \| s_k r_{k-1} + r_{k-2},$$

meaning that $r \rightarrow$ and $s_k r_{k-1} + r_{k-2}$ are collinear (that is, have the same slope coefficient). Taking the [pseudoscalar product](#) of both parts with $r_{k-1} \rightarrow$, we get

$$0 = s_k (r_{k-1} \times r_{k-1}) + (r_{k-2} \times r_{k-1}),$$

which yields the final formula

$$s_k = -\frac{r_{k-2} \times r_{k-1}}{r_{k-1} \times r_{k-1}}.$$

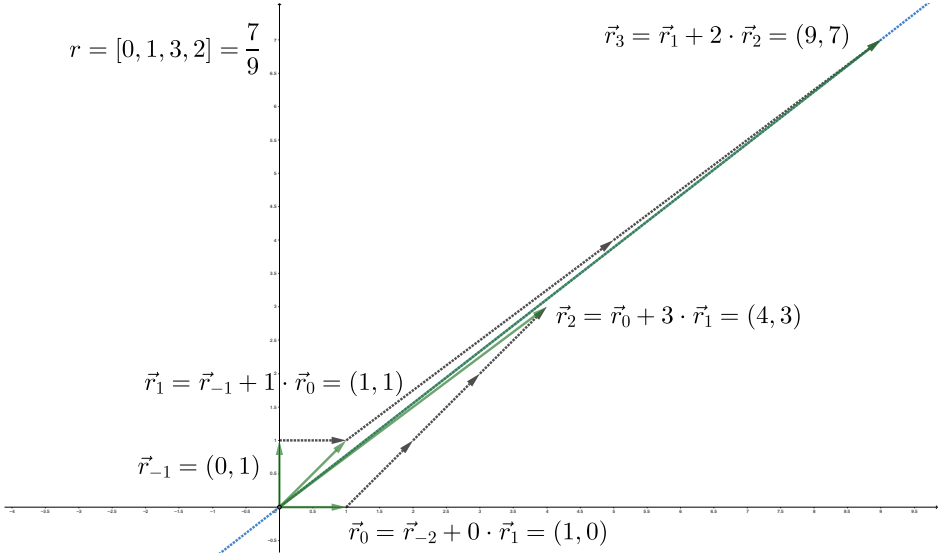
Nose stretching algorithm

Each time you add r_{k-1} to the vector p , the value of $p \times r$ is increased by $r_{k-1} \times r$.

Thus, $a_k = \lfloor s_k \rfloor$ is the maximum integer number of r_{k-1} vectors that can be added to r_{k-2} without changing the sign of the cross product with r .

In other words, a_k is the maximum integer number of times you can add r_{k-1} to r_{k-2} without crossing the line defined by r .

$r = [0, 1, 3, 2] = \frac{7}{9}$



$\vec{r}_3 = \vec{r}_1 + 2 \cdot \vec{r}_2 = (9, 7)$
 $\vec{r}_2 = \vec{r}_0 + 3 \cdot \vec{r}_1 = (4, 3)$
 $\vec{r}_1 = \vec{r}_{-1} + 1 \cdot \vec{r}_0 = (1, 1)$
 $\vec{r}_{-1} = (0, 1)$
 $\vec{r}_0 = \vec{r}_{-2} + 0 \cdot \vec{r}_1 = (1, 0)$

Convergents of $r = \frac{7}{9} = [0; 1, 3, 2]$. Semiconvergents correspond to intermediate points between gray arrows.

On the picture above, $r_2 = (4; 3)$ is obtained by repeatedly adding $r_1 = (1; 1)$ to $r_0 = (1; 0)$.

When it is not possible to further add r_1 to r_0 without crossing the $y = rx$ line, we go to the other side and repeatedly add r_2 to r_1 to obtain $r_3 = (9; 7)$.

This procedure generates exponentially longer vectors, that approach the line.

For this property, the procedure of generating consequent convergent vectors was dubbed the **nose stretching algorithm** by Boris Delaunay.

If we look on the triangle drawn on points r_{k-2} , r_k and O we will notice that its doubled area is

$$|r_{k-2} \times r_k| = |r_{k-2} \times (r_{k-2} + a_k r_{k-1})| = a_k |r_{k-2} \times r_{k-1}| = a_k.$$

Combined with the [Pick's theorem](#), it means that there are no lattice points strictly inside the triangle and the only lattice points on its border are O and $r_{k-2} + t \cdot r_{k-1}$ for all integer t such that $0 \leq t \leq a_k$. When joined for all possible k it means that there are no integer points in the space between polygons formed by even-indexed and odd-indexed convergent vectors.

This, in turn, means that r_k with odd coefficients form a convex hull of lattice points with $x \geq 0$ above the line $y = rx$, while r_k with even coefficients form a convex hull of lattice points with $x > 0$ below the line $y = rx$.

Definition

These polygons are also known as **Klein polygons**, named after Felix Klein who first suggested this geometric interpretation to the continued fractions.

Problem examples

file:///tmp/tmp.eaEVB9acNV/page_027_files/localhost:8000/algebra/continued-fractions.html

21/29

Now that the most important facts and concepts were introduced, it is time to delve into specific problem examples.

Convex hull under the line

Find the convex hull of lattice points $(x; y)$ such that $0 \leq x \leq N$ and $0 \leq y \leq rx$ for $r = [a_0; a_1, \dots, a_k] = \frac{p_k}{q_k}$.

Solution

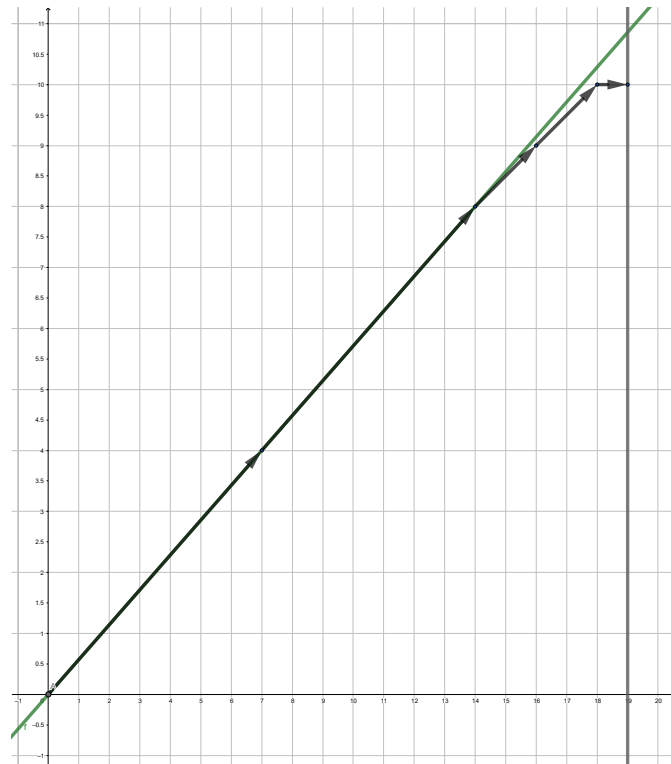
If we were considering the unbounded set $0 \leq x$, the upper convex hull would be given by the line $y = rx$ itself.

However, with additional constraint $x \leq N$ we'd need to eventually deviate from the line to maintain proper convex hull.

Let $t = \lfloor \frac{N}{q_k} \rfloor$, then first t lattice points on the hull after $(0; 0)$ are $\alpha \cdot (q_k; p_k)$ for integer $1 \leq \alpha \leq t$.

However $(t+1)(q_k; p_k)$ can't be next lattice point since $(t+1)q_k$ is greater than N .

To get to the next lattice points in the hull, we should get to the point $(x; y)$ which diverges from $y = rx$ by the smallest margin, while maintaining $x \leq N$.



Convex hull of lattice points under $y = \frac{4}{7}x$ for $0 \leq x \leq 19$ consists of points $(0; 0), (7; 4), (14; 8), (16; 9), (18; 10), (19; 10)$.

Let $(x; y)$ be the last current point in the convex hull. Then the next point $(x'; y')$ is such that $x' \leq N$ and $(x'; y') - (x; y) = (\Delta x; \Delta y)$ is as close to the line $y = rx$ as possible. In other words, $(\Delta x; \Delta y)$ maximizes $r\Delta x - \Delta y$ subject to $\Delta x \leq N - x$ and $\Delta y \leq r\Delta x$.

Points like that lie on the convex hull of lattice points below $y = rx$. In other words, $(\Delta x; \Delta y)$ must be a lower semiconvergent of r .

That being said, $(\Delta x; \Delta y)$ is of form $(q_{i-1}; p_{i-1}) + t \cdot (q_i; p_i)$ for some odd number i and $0 \leq t < a_i$.

To find such i , we can traverse all possible i starting from the largest one and use $t = \lfloor \frac{N-x-q_{i-1}}{q_i} \rfloor$ for i such that $N - x - q_{i-1} \geq 0$.

With $(\Delta x; \Delta y) = (q_{i-1}; p_{i-1}) + t \cdot (q_i; p_i)$, the condition $\Delta y \leq r\Delta x$ would be preserved by semiconvergent properties.

And $t < a_i$ would hold because we already exhausted semiconvergents obtained from $i+2$, hence $x + q_{i-1} + a_i q_i = x + q_{i+1}$ is greater than N .

Now that we may add $(\Delta x; \Delta y)$, to $(x; y)$ for $k = \lfloor \frac{N-x}{\Delta x} \rfloor$ times before we exceed N , after which we would try the next semiconvergent.

C++

```

// returns [ah, ph, qh] such that points r[i]=(ph[i], qh[i]) constitute upper convex hull
// of lattice points on  $0 \leq x \leq N$  and  $0 \leq y \leq r * x$ , where  $r = [a_0; a_1, a_2, \dots]$ 
// and there are ah[i]-1 integer points on the segment between r[i] and r[i+1]
auto hull(auto a, int N) {
    auto [p, q] = convergents(a);
    int t = N / q.back();
    vector ah = {t};
    vector ph = {0, t*p.back()};
    vector qh = {0, t*q.back()};

    for(int i = q.size() - 1; i >= 0; i--) {
        if(i % 2) {
            while(qh.back() + q[i - 1] <= N) {
                t = (N - qh.back() - q[i - 1]) / q[i];
                int dp = p[i - 1] + t * p[i];
                int dq = q[i - 1] + t * q[i];
                int k = (N - qh.back()) / dq;
                ah.push_back(k);
                ph.push_back(ph.back() + k * dp);
                qh.push_back(qh.back() + k * dq);
            }
        }
    }
    return make_tuple(ah, ph, qh);
}

```

Python

```

# returns [ah, ph, qh] such that points r[i]=(ph[i], qh[i]) constitute upper convex hull
# of lattice points on  $0 \leq x \leq N$  and  $0 \leq y \leq r * x$ , where  $r = [a_0; a_1, a_2, \dots]$ 
# and there are ah[i]-1 integer points on the segment between r[i] and r[i+1]
def hull(a, N):
    p, q = convergents(a)
    t = N // q[-1]
    ah = [t]
    ph = [0, t*p[-1]]
    qh = [0, t*q[-1]]
    for i in reversed(range(len(q))):
        if i % 2 == 1:
            while qh[-1] + q[i-1] <= N:
                t = (N - qh[-1] - q[i-1]) // q[i]
                dp = p[i-1] + t*p[i]
                dq = q[i-1] + t*q[i]
                k = (N - qh[-1]) // dq
                ah.append(k)
                ph.append(ph[-1] + k * dp)
                qh.append(qh[-1] + k * dq)
    return ah, ph, qh

```

Timus - Crime and Punishment

You're given integer numbers A , B and N . Find $x \geq 0$ and $y \geq 0$ such that $Ax + By \leq N$ and $Ax + By$ is the maximum possible.

Solution

In this problem it holds that $1 \leq A, B, N \leq 2 \cdot 10^9$, so it can be solved in $O(\sqrt{N})$. However, there is $O(\log N)$ solution with continued fractions.

For our convenience, we will invert the direction of x by doing a substitution $x \mapsto \lfloor \frac{N}{A} \rfloor - x$, so that now we need to find the point $(x; y)$ such that $0 \leq x \leq \lfloor \frac{N}{A} \rfloor$, $By - Ax \leq N \pmod A$ and $By - Ax$ is the maximum possible. Optimal y for each x has a value of $\lfloor \frac{Ax + (N \pmod A)}{B} \rfloor$.

To treat it more generically, we will write a function that finds the best point on $0 \leq x \leq N$ and $y = \lfloor \frac{Ax+B}{C} \rfloor$.

Core solution idea in this problem essentially repeats the previous problem, but instead of using lower semiconvergents to diverge from line, you use upper semiconvergents to get closer to the line without crossing it and without violating $x \leq N$. Unfortunately, unlike the previous problem, you need to make sure that you don't cross the $y = \frac{Ax+B}{C}$ line while getting closer to it, so you should keep it in mind when calculating semiconvergent's coefficient t .

Python

```
# (x, y) such that y = (A*x+B) // C,
# Cy - Ax is max and 0 <= x <= N.
def closest(A, B, C, N):
    # y <= (A*x + B)/C <=> diff(x, y) <= B
    def diff(x, y):
        return C*y - A*x
    a = fraction(A, C)
    p, q = convergents(a)
    ph = [B // C]
    qh = [0]
    for i in range(2, len(q) - 1):
        if i % 2 == 0:
            while diff(qh[-1] + q[i+1], ph[-1] + p[i+1]) <= B:
                t = 1 + (diff(qh[-1] + q[i-1], ph[-1] + p[i-1]) - B - 1) // abs(diff(q[i], p[i]))
                dp = p[i-1] + t*p[i]
                dq = q[i-1] + t*q[i]
                k = (N - qh[-1]) // dq
                if k == 0:
                    return qh[-1], ph[-1]
                return qh[-1], ph[-1]
            k = min(k, (B - diff(qh[-1], ph[-1])) // diff(dq, dp))
            qh.append(qh[-1] + k*dq)
            ph.append(ph[-1] + k*dp)
    return qh[-1], ph[-1]

def solve(A, B, N):
    x, y = closest(A, N % A, B, N // A)
    return N // A - x, y
```

June Challenge 2017 - Euler Sum

Compute $\sum_{x=1}^N \lfloor ex \rfloor$, where $e = [2; 1, 2, 1, 1, 4, 1, 1, 6, 1, \dots, 1, 2n, 1, \dots]$ is the Euler's number and $N \leq 10^{4000}$.

 Solution

This sum is equal to the number of lattice point $(x; y)$ such that $1 \leq x \leq N$ and $1 \leq y \leq ex$.

After constructing the convex hull of the points below $y = ex$, this number can be computed using [Pick's theorem](#):

C++

```
// sum floor(k * x) for k in [1, N] and x = [a0; a1, a2, ...]
int sum_floor(auto a, int N) {
    N++;
    auto [ah, ph, qh] = hull(a, N);

    // The number of lattice points within a vertical right trapezoid
    // on points (0; 0) - (0; y1) - (dx; y2) - (dx; 0) that has
    // a+1 integer points on the segment (0; y1) - (dx; y2).
    auto picks = [](int y1, int y2, int dx, int a) {
        int b = y1 + y2 + a + dx;
        int A = (y1 + y2) * dx;
        return (A - b + 2) / 2 + b - (y2 + 1);
    };

    int ans = 0;
    for(size_t i = 1; i < qh.size(); i++) {
        ans += picks(ph[i - 1], ph[i], qh[i] - qh[i - 1], ah[i - 1]);
    }
    return ans - N;
}
```

Python

```
# sum floor(k * x) for k in [1, N] and x = [a0; a1, a2, ...]
def sum_floor(a, N):
    N += 1
    ah, ph, qh = hull(a, N)

    # The number of lattice points within a vertical right trapezoid
    # on points (0; 0) - (0; y1) - (dx; y2) - (dx; 0) that has
    # a+1 integer points on the segment (0; y1) - (dx; y2).
    def picks(y1, y2, dx, a):
        b = y1 + y2 + a + dx
        A = (y1 + y2) * dx
        return (A - b + 2) // 2 + b - (y2 + 1)

    ans = 0
    for i in range(1, len(qh)):
        ans += picks(ph[i-1], ph[i], qh[i]-qh[i-1], ah[i-1])
    return ans - N
```

 NAIPC 2019 - It's a Mod, Mod, Mod, Mod World

Given p , q and n , compute $\sum_{i=1}^n [p \cdot i \bmod q]$.

Solution

This problem reduces to the previous one if you note that $a \bmod b = a - \lfloor \frac{a}{b} \rfloor b$. With this fact, the sum reduces to

$$\sum_{i=1}^n (p \cdot i - \lfloor \frac{p \cdot i}{q} \rfloor q) = \frac{pn(n+1)}{2} - q \sum_{i=1}^n \lfloor \frac{p \cdot i}{q} \rfloor.$$

However, summing up $\lfloor rx \rfloor$ for x from 1 to N is something that we're capable of from the previous problem.

C++

```
void solve(int p, int q, int N) {
    cout << p * N * (N + 1) / 2 - q * sum_floor(fraction(p, q), N) << "\n";
}
```

Python

```
def solve(p, q, N):
    return p * N * (N + 1) // 2 - q * sum_floor(fraction(p, q), N)
```

Library Checker - Sum of Floor of Linear

Given N, M, A and B , compute $\sum_{i=0}^{N-1} \lfloor \frac{Ai+B}{M} \rfloor$.

 Solution

This is the most technically troublesome problem so far.

It is possible to use the same approach and construct the full convex hull of points below the line $y = \frac{Ax+B}{M}$.

We already know how to solve it for $B = 0$. Moreover, we already know how to construct this convex hull up to the closest lattice point to this line on $[0, N - 1]$ segment (this is done in the "Crime and Punishment" problem above).

Now we should note that once we reached the closest point to the line, we can just assume that the line in fact passes through the closest point, as there are no other lattice points on $[0, N - 1]$ in between the actual line and the line moved slightly below to pass through the closest point.

That being said, to construct the full convex hull below the line $y = \frac{Ax+B}{M}$ on $[0, N - 1]$, we can construct it up to the closest point to the line on $[0, N - 1]$ and then continue as if the line passes through this point, reusing algorithm for constructing convex hull with $B = 0$:

Python

```
# hull of lattice (x, y) such that C*y <= A*x+B
def hull(A, B, C, N):
    def diff(x, y):
        return C*y-A*x
    a = fraction(A, C)
    p, q = convergents(a)
    ah = []
    ph = [B // C]
    qh = [0]

    def insert(dq, dp):
        k = (N - qh[-1]) // dq
        if diff(dq, dp) > 0:
            k = min(k, (B - diff(qh[-1], ph[-1])) // diff(dq, dp))
        ah.append(k)
        qh.append(qh[-1] + k*dq)
        ph.append(ph[-1] + k*dp)

    for i in range(1, len(q) - 1):
        if i % 2 == 0:
            while diff(qh[-1] + q[i+1], ph[-1] + p[i+1]) <= B:
                t = (B - diff(qh[-1] + q[i+1], ph[-1] + p[i+1])) // abs(diff(q[i], p[i]))
                dp = p[i+1] - t*p[i]
                dq = q[i+1] - t*q[i]
                if dq < 0 or qh[-1] + dq > N:
                    break
                insert(dq, dp)
            insert(q[-1], p[-1])

    for i in reversed(range(len(q))):
        if i % 2 == 1:
            while qh[-1] + q[i-1] <= N:
                t = (N - qh[-1] - q[i-1]) // q[i]
                dp = p[i-1] + t*p[i]
                dq = q[i-1] + t*q[i]
                insert(dq, dp)
    return ah, ph, qh
```

 OKC 2 - From Modular to Rational

There is a rational number $\frac{p}{q}$ such that $1 \leq p, q \leq 10^9$. You may ask the value of pq^{-1} modulo $m \sim 10^9$ for several prime numbers m . Recover $\frac{p}{q}$.

Equivalent formulation: Find x that delivers the minimum of $Ax \bmod M$ for $1 \leq x \leq N$.

 Solution


Due to Chinese remainder theorem, asking the result modulo several prime numbers is the same as asking it modulo their product. Due to this, without loss of generality we'll assume that we know the remainder modulo sufficiently large number m .

There could be several possible solutions (p, q) to $p \equiv qr \pmod{m}$ for a given remainder r . However, if (p_1, q_1) and (p_2, q_2) are both the solutions then it also holds that $p_1 q_2 \equiv p_2 q_1 \pmod{m}$. Assuming that $\frac{p_1}{q_1} \neq \frac{p_2}{q_2}$ it means that $|p_1 q_2 - p_2 q_1|$ is at least m .

In the statement we were told that $1 \leq p, q \leq 10^9$, so if both p_1, q_1 and p_2, q_2 are at most 10^9 , then the difference is at most 10^{18} . For $m > 10^{18}$ it means that the solution $\frac{p}{q}$ with $1 \leq p, q \leq 10^9$ is unique, as a rational number.

So, the problem boils down, given r modulo m , to finding any q such that $1 \leq q \leq 10^9$ and $qr \pmod{m} \leq 10^9$.

This is effectively the same as finding q that delivers the minimum possible $qr \pmod{m}$ for $1 \leq q \leq 10^9$.

For $qr = km + b$ it means that we need to find a pair (q, m) such that $1 \leq q \leq 10^9$ and $qr - km \geq 0$ is the minimum possible.

Since m is constant, we can divide by it and further restate it as find q such that $1 \leq q \leq 10^9$ and $\frac{r}{m}q - k \geq 0$ is the minimum possible.

In terms of continued fractions it means that $\frac{k}{q}$ is the best diophantine approximation to $\frac{r}{m}$ and it is sufficient to only check lower semiconvergents of $\frac{r}{m}$.

Python

```
# find Q that minimizes Q*r mod m for 1 <= k <= n < m
def mod_min(r, n, m):
    a = fraction(r, m)
    p, q = convergents(a)
    for i in range(2, len(q)):
        if i % 2 == 1 and (i + 1 == len(q) or q[i+1] > n):
            t = (n - q[i-1]) // q[i]
            return q[i-1] + t*q[i]
```

Practice problems

- [UVA OJ - Continued Fractions](#)
- [ProjectEuler+ #64: Odd period square roots](#)
- [Codeforces Round #184 \(Div. 2\) - Continued Fractions](#)
- [Codeforces Round #201 \(Div. 1\) - Doodle Jump](#)
- [Codeforces Round #325 \(Div. 1\) - Alice, Bob, Oranges and Apples](#)
- [POJ Founder Monthly Contest 2008.03.16 - A Modular Arithmetic Challenge](#)
- [2019 Multi-University Training Contest 5 - fraction](#)
- [SnackDown 2019 Elimination Round - Election Bait](#)
- [Code Jam 2019 round 2 - Continued Fraction](#)

Contributors:

[Oleksandr Kulkov](#) (99.91%) [JATIN GAUR](#) (0.09%)

Original

Binary Exponentiation by Factoring

Consider a problem of computing $ax^y \pmod{2^d}$, given integers a, x, y and $d \geq 3$, where x is odd.

The algorithm below allows to solve this problem with $O(d)$ additions and binary operations and a single multiplication by y .

Due to the structure of the multiplicative group modulo 2^d , any number x such that $x \equiv 1 \pmod{4}$ can be represented as

$$x \equiv b^{L(x)} \pmod{2^d},$$

where $b \equiv 5 \pmod{8}$. Without loss of generality we assume that $x \equiv 1 \pmod{4}$, as we can reduce $x \equiv 3 \pmod{4}$ to $x \equiv 1 \pmod{4}$ by substituting $x \mapsto -x$ and $a \mapsto (-1)^y a$. In this notion, ax^y is represented as

$$ax^y \equiv ab^{yL(x)} \pmod{2^d}.$$

The core idea of the algorithm is to simplify the computation of $L(x)$ and $b^{yL(x)}$ using the fact that we're working modulo 2^d . For reasons that will be apparent later on, we'll be working with $4L(x)$ rather than $L(x)$, but taken modulo 2^d instead of 2^{d-2} .

In this article, we will cover the implementation for 32-bit integers. Let

- `mbin_log_32(r, x)` be a function that computes $r + 4L(x) \pmod{2^d}$;
- `mbin_exp_32(r, x)` be a function that computes $rb^{\frac{x}{4}} \pmod{2^d}$;
- `mbin_power_odd_32(a, x, y)` be a function that computes $ax^y \pmod{2^d}$.

Then `mbin_power_odd_32` is implemented as follows:

```
uint32_t mbin_power_odd_32(uint32_t rem, uint32_t base, uint32_t exp) {
    if (base & 2) {
        /* divider is considered negative */
        base = -base;
        /* check if result should be negative */
        if (exp & 1) {
            rem = -rem;
        }
    }
    return (mbin_exp_32(rem, mbin_log_32(0, base) * exp));
}
```

Computing $4L(x)$ from x

Let x be an odd number such that $x \equiv 1 \pmod{4}$. It can be represented as

$$x \equiv (2^{a_1} + 1) \dots (2^{a_k} + 1) \pmod{2^d},$$

where $1 < a_1 < \dots < a_k < d$. Here $L(\cdot)$ is well-defined for each multiplier, as they're equal to 1 modulo 4. Hence,

$$4L(x) \equiv 4L(2^{a_1} + 1) + \dots + 4L(2^{a_k} + 1) \pmod{2^d}.$$

So, if we precompute $t_k = 4L(2^{a_k} + 1)$ for all $1 < k < d$, we will be able to compute $4L(x)$ for any number x .

For 32-bit integers, we can use the following table:

```
const uint32_t mbin_log_32_table[32] = {
    0x00000000, 0x00000000, 0xd3cfd984, 0x9ee62e18,
    0xe83d9070, 0xb59e81e0, 0xa17407c0, 0xce601f80,
    0xf4807f00, 0xe701fe00, 0xbe07fc00, 0xfc1ff800,
    0xf87ff000, 0xf1ffe000, 0xe7ffc000, 0xdfff8000,
    0xffff0000, 0xfffe0000, 0xfffc0000, 0xff800000,
    0xff000000, 0xfe000000, 0xfc000000, 0xf8000000,
    0xf0000000, 0xe0000000, 0xc0000000, 0x80000000,
};
```

On practice, a slightly different approach is used than described above. Rather than finding the factorization for x , we will consequently multiply x with $2^n + 1$ until we turn it into 1 modulo 2^d . In this way, we will find the representation of x^{-1} , that is

$$x(2^{a_1} + 1) \dots (2^{a_k} + 1) \equiv 1 \pmod{2^d}.$$

To do this, we iterate over n such that $1 < n < d$. If the current x has n -th bit set, we multiply x with $2^n + 1$, which is conveniently done in C++ as `x = x + (x << n)`. This won't change bits lower than n , but will turn the n -th bit to zero, because x is odd.

With all this in mind, the function `mbin_log_32(r, x)` is implemented as follows:

```
uint32_t mbin_log_32(uint32_t r, uint32_t x) {
    uint8_t n;

    for (n = 2; n < 32; n++) {
        if (x & (1 << n)) {
            x = x + (x << n);
            r -= mbin_log_32_table[n];
        }
    }

    return r;
}
```

Note that $4L(x) = -4L(x^{-1})$, so instead of adding $4L(2^n + 1)$, we subtract it from r , which initially equates to 0.

Computing x from $4L(x)$

Note that for $k \geq 1$ it holds that

$$(a2^k + 1)^2 = a^2 2^{2k} + a2^{k+1} + 1 = b2^{k+1} + 1,$$

from which (by repeated squaring) we can deduce that

$$(2^a + 1)^{2^b} \equiv 1 \pmod{2^{a+b}}.$$

Applying this result to $a = 2^n + 1$ and $b = d - k$ we deduce that the multiplicative order of $2^n + 1$ is a divisor of 2^{d-n} .

This, in turn, means that $L(2^n + 1)$ must be divisible by 2^n , as the order of b is 2^{d-2} and the order of b^y is 2^{d-2-y} , where 2^y is the highest power of 2 that divides y , so we need

$$2^{d-k} \equiv 0 \pmod{2^{d-2-y}},$$

thus y must be greater or equal than $k - 2$. This is a bit ugly and to mitigate this we said in the beginning that we multiply $L(x)$ by 4. Now if we know $4L(x)$, we can uniquely decomposing it into a sum of $4L(2^n + 1)$ by consequentially checking bits in $4L(x)$. If the n -th bit is set to 1, we will multiply the result with $2^n + 1$ and reduce the current $4L(x)$ by $4L(2^n + 1)$.

Thus, `mbin_exp_32` is implemented as follows:

```
uint32_t mbin_exp_32(uint32_t r, uint32_t x) {
    uint8_t n;

    for (n = 2; n < 32; n++) {
        if (x & (1 << n)) {
            r = r + (r << n);
            x -= mbin_log_32_table[n];
        }
    }

    return r;
}
```

Further optimizations

It is possible to halve the number of iterations if you note that $4L(2^{d-1} + 1) = 2^{d-1}$ and that for $2k \geq d$ it holds that

$$(2^n + 1)^2 \equiv 2^{2n} + 2^{n+1} + 1 \equiv 2^{n+1} + 1 \pmod{2^d},$$

which allows to deduce that $4L(2^n + 1) = 2^n$ for $2n \geq d$. So, you could simplify the algorithm by only going up to $\frac{d}{2}$ and then use the fact above to compute the remaining part with bitwise operations:

```
uint32_t mbin_log_32(uint32_t r, uint32_t x) {
    uint8_t n;

    for (n = 2; n != 16; n++) {
        if (x & (1 << n)) {
            x = x + (x << n);
            r -= mbin_log_32_table[n];
        }
    }

    r -= (x & 0xFFFF0000);

    return r;
}

uint32_t mbin_exp_32(uint32_t r, uint32_t x) {
    uint8_t n;

    for (n = 2; n != 16; n++) {
        if (x & (1 << n)) {
            r = r + (r << n);
            x -= mbin_log_32_table[n];
        }
    }

    r *= 1 - (x & 0xFFFF0000);

    return r;
}
```

Computing logarithm table

To compute log-table, one could modify the [Pohlig–Hellman algorithm](#) for the case when modulo is a power of 2.

Our main task here is to compute x such that $g^x \equiv y \pmod{2^d}$, where $g = 5$ and y is a number of kind $2^n + 1$.

Squaring both parts k times we arrive to

$$g^{2^k x} \equiv y^{2^k} \pmod{2^d}.$$

Note that the order of g is not greater than 2^d (in fact, than 2^{d-2} , but we will stick to 2^d for convenience), hence using $k = d - 1$ we will have either g^1 or g^0 on the left hand side which allows us to determine the smallest bit of x by comparing y^{2^k} to g . Now assume that $x = x_0 + 2^k x_1$, where x_0 is a known part and x_1 is not yet known. Then

$$g^{x_0 + 2^k x_1} \equiv y \pmod{2^d}.$$

Multiplying both parts with g^{-x_0} , we get

$$g^{2^k x_1} \equiv (g^{-x_0} y) \pmod{2^d}.$$

Now, squaring both sides $d - k - 1$ times we can obtain the next bit of x , eventually recovering all its bits.

References

- [M30, Hans Petter Selasky, 2009](#)

Contributors:

[Oleksandr Kulkov](#) (80.18%) [Hans Petter Selasky](#) (19.82%)